

10. 动态规划

10.1 背包DP

10.1.1 0-1背包

题意

有 n ($1 \leq n \leq 1000$)件物品和一个容量为 V ($1 \leq V \leq 1000$)的背包, 每件物品只能用一次. 第 i ($1 \leq i \leq n$)件物品的体积为 v_i ($1 \leq v_i \leq 1000$), 价值为 w_i ($1 \leq v_i \leq 1000$). 问将哪些物品装入背包可使得这些物品的总体积不超过背包容量, 且总价值最大, 输出最大总价值.

思路I

$dp[i][j]$ 表示只考虑前 i 个物品、当前总体积为 j 时的最大总价值, 初始条件 $dp[0][0] = 0$, 最终答案 $ans = \max_{0 \leq v \leq V} dp[n][v]$.

按是否选第 i 个物品分类:

①若不选第 i 个物品, 则 $dp[i][j] = dp[i-1][j]$.

②若选第 i 个物品, 则 $dp[i][j] = dp[i-1][j-v[i]] + w[i]$.

综上, 状态转移方程 $dp[i][j] = \max\{dp[i-1][j], dp[i-1][j-v[i]] + w[i]\} (j \geq v[i])$.

代码I

```
1 void solve() {
2     int n, V; cin >> n >> V;
3     vector<int> v(n+1), w(n+1);
4     for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
5
6     // dp[i][j]表示只考虑前i个物品、体积之和为j时的最大价值
7     vector<vector<int>> dp(n+1, vector<int>(V+1));
8     for (int i = 1; i <= n; i++) {
9         for (int j = 0; j <= V; j++) {
10             dp[i][j] = dp[i-1][j];
11             if (j >= v[i])
12                 dp[i][j] = max(dp[i][j], dp[i-1][j-v[i]] + w[i]);
13         }
14     }
15     cout << *max_element(all(dp[n])) << endl;
16 }
17
18 int main() {
19     solve();
20 }
```

思路II

$dp[i][j] = dp[i-1][j]$ 表明: $dp[i][j]$ 只与 $dp[0][j]$ 有关, 则无需记录第一个维度. 若直接去掉第一个维度, 则 $dp[i-1][j-v[i]]$ 表示 $dp[i][j-v[i]]$ 而不是 $dp[i-1][j-v[i]]$, 故需调整内层循环的次序, 使得原第一维是 $(i-1)$ 的状态是没算过的.

最终答案是 $dp[V]$ 是因为 $dp[]$ 初始值都为0. 若只有 $dp[0] = 0$, 则需要扫一遍 $dp[]$ 来得到最大值.

代码II

```

1 void solve() {
2     int n, V; cin >> n >> V;
3     vector<int> v(n + 1), w(n + 1);
4     for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
5
6     // dp[i][j]表示只考虑前i个物品、体积之和为j时的最大价值
7     vector<int> dp(V + 1);
8     for (int i = 1; i <= n; i++) {
9         for (int j = V; j >= v[i]; j--)
10             dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
11     }
12     cout << dp[V] << endl;
13 }
14
15 int main() {
16     solve();
17 }

```

推广

求解总体积恰为 V 时的最大值, 则初始化 $dp[0] = 0, dp[i] = -\text{INF}$ ($1 \leq i \leq n$).

10.1.2 完全背包

题意

有 n ($1 \leq n \leq 1000$)件物品和一个容量为 V ($1 \leq V \leq 1000$)的背包, 每件物品能用无限次. 第 i ($1 \leq i \leq n$)件物品的体积为 v_i ($1 \leq v_i \leq 1000$), 价值为 w_i ($1 \leq w_i \leq 1000$). 问将哪些物品装入背包可使得这些物品的总体积不超过背包容量, 且总价值最大, 输出最大总价值.

思路

$$dp[i][j] = \max\{dp[i-1][j], dp[i-1][j-v] + w, dp[i-1][j-2v] + 2w, \dots\} \cdots \textcircled{1}$$

朴素想法: 在0-1背包的基础上, 内层循环变为:

```

1 for (int j = V; j >= v[i]; j--) {
2     for (int k = 0; k * v[i] <= j; k++)
3         dp[j] = max(dp[j], dp[j - k * v[i]] + k * w[i]);
4 }

```

①中令 $j \mapsto j - v$ 得:

$$dp[i][j-v] = \max\{dp[i-1][j-v], dp[i-1][j-2v] + w, dp[i-1][j-3v] + 2w, \dots\} \cdots \textcircled{2}$$

注意到①②中 $dp[i-1][j-v], dp[i-1][j-2v], \dots$ 的项相差一个 w , 则 $dp[i][j] = \max\{dp[i-1][j], dp[i][j-v] + w\}$.

故只需将内层循环的顺序反过来, 这样 $dp[j-v[i]]$ 是被算过的, 即:

```

1 for (int j = v[i]; j <= V; j++)
2     dp[j] = max(dp[j], dp[j - v[i]] + w[i]);

```

最终答案 $ans = dp[n][V]$.

代码I

```

1 void solve() {
2     int n, V; cin >> n >> V;
3     vector<int> v(n + 1), w(n + 1);
4     for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
5
6     // dp[i][j]表示只考虑前i个物品、体积之和为j时的最大价值
7     vector<vector<int>> dp(n + 1, vector<int>(V + 1));
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= V; j++) {
10             dp[i][j] = dp[i - 1][j];
11             if (j >= v[i])
12                 dp[i][j] = max(dp[i][j], dp[i][j - v[i]] + w[i]);
13         }
14     }
15     cout << dp[n][V] << endl;
16 }
17
18 int main() {
19     solve();
20 }

```

代码II

```

1 void solve() {
2     int n, V; cin >> n >> V;
3     vector<int> v(n + 1), w(n + 1);
4     for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
5
6     // dp[i][j]表示只考虑前i个物品、体积之和为j时的最大价值
7     vector<int> dp(V + 1);
8     for (int i = 1; i <= n; i++) {
9         for (int j = v[i]; j <= V; j++)
10             dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
11     }
12     cout << dp[V] << endl;
13 }
14
15 int main() {
16     solve();
17 }

```

扩展

求解总体积恰为 V 时的最大值, 则初始化 $dp[0] = 0, dp[i] = -\text{INF}$.

对比0 — 1背包和完全背包

0 — 1背包: $dp[i][j] = \max\{dp[i - 1][j], dp[i - 1][j - v] + w\}$, 内层循环从大到小.

完全背包: $dp[i][j] = \max\{dp[i - 1][j], dp[i][j - v] + w\}$, 内层循环从小到大.

10.1.3 多重背包I

题意

有 N ($1 \leq N \leq 100$)件物品和一个容量为 V ($1 \leq V \leq 100$)的背包.第 i 件物品的体积为 v_i ($1 \leq v_i \leq 100$),价值为 w_i ($1 \leq w_i \leq 100$),能用 s_i ($1 \leq s_i \leq 100$)次.问将哪些物品装入背包可使得这些物品的总体积不超过背包容量,且总价值最大,输出最大总价值.

思路

因数据范围 ≤ 100 ,则可跑 $O(n^3)$ 的朴素算法.

代码

```
1  const int MAXN = 105;
2  int dp[MAXN];
3
4  int main() {
5      for (int i = 0; i < N; i++) {
6          int v, w, s;
7          cin >> v >> w >> s;
8          for (int j = v; j >= 0; j--) {
9              for (int k = 1; k <= s && k * v <= j; k++)
10                 dp[j] = max(dp[j], dp[j - k * v] + k * w);
11          }
12      }
13      cout << dp[V];
14  }
```

10.1.4 多重背包II

题意

有 N ($1 \leq N \leq 1000$)件物品和一个容量为 V ($1 \leq V \leq 2000$)的背包.第 i 件物品的体积为 v_i ($1 \leq v_i \leq 2000$),价值为 w_i ($1 \leq w_i \leq 2000$),能用 s_i ($1 \leq s_i \leq 2000$)次.问将哪些物品装入背包可使得这些物品的总体积不超过背包容量,且总价值最大,输出最大总价值.

思路

数据范围不能跑 $O(n^3)$ 的算法.

考虑将第 i 个物品拆成 s_i 份,这样每个物品只能用1次,转化为0-1背包问题.最坏的情况是 $1000 \times 2000 \times 2000$,依旧是 $1e9$ 的数量级,会T.

考察问题:给定数 n ,至少用哪些数可将 $0 \sim n$ 都拆成这些数之和.

$n = 7$ 时,考察其二进制 $(111)_2$.取 $2^0, 2^1, 2^2$ 三个数,则 $0 = 0$ (一个都不取),
 $1 = 2^0, 2 = 2^1, 3 = 2^0 + 2^1, 4 = 2^2, 5 = 2^0 + 2^2, 6 = 2^1 + 2^2, 7 = 2^0 + 2^1 + 2^2$.

显然,对给定的 n ,至少需要 $\lceil \log_2 n \rceil$ 个数.

但 $n = 10$ 时,取1, 2, 4, 8不仅可表示出 $0 \sim 10$,还可表示出11 ~ 15,即过量了.

考虑10依次减去取的数直至不能再减: $10 - 1 = 9, 9 - 2 = 7, 7 - 4 = 3, 3 - 8 < 0$,故取1, 2, 4, 3即可表示出1 ~ 10.事实上,取1, 2, 4可表出 $0 \sim 7$,再取3即可表示 $0 \sim 10$.

用二进制的思想,可将 s_i 分成 $\lceil \log_2 s_i \rceil$ 份,最坏的情况是 $1000 \times (\log_2 2000) \times 2000 \approx 1000 \times 11 \times 2000$,是 $2e7$ 的数量级,不会T.

代码:二进制优化

```

1  const int MAXN = 2005;
2  int dp[MAXN];
3  struct Good {
4      int v, w;
5  };
6  vector<Good> goods; // 物品组
7
8  int main() {
9      for (int i = 0; i < N; i++) {
10         int v, w, s;
11         cin >> v >> w >> s;
12         for (int k = 1; k <= s; k *= 2) { // 将整的 $2^k$ 放进物品组
13             s -= k;
14             goods.push_back({ k * v, k * w });
15         }
16         if (s > 0) goods.push_back({ s * v, s * w }); // 不整的放进物品组
17     }
18
19     for (auto good : goods) {
20         for (int j = v; j >= good.v; j--)
21             dp[j] = max(dp[j], dp[j - good.v] + good.w);
22     }
23     cout << dp[V];
24 }

```

10.1.5 多重背包III

题意

有一容量为 V 的背包和 n 件物品,第 i 件物品最多有 s_i 件,每件体积为 v_i ,价值为 w_i .求将哪些物品装入背包,可使得物品体积之和不超过背包容量,且价值之和最大,输出最大价值.

第一行输入两个整数 n, V ($1 \leq n \leq 1000, 1 \leq V \leq 2e4$).接下来 n 行每行输入三个整数 v, w, s ($1 \leq v, w, s \leq 2e4$),分别表示物品的体积、价值和数量.

思路

$dp[i][j]$ 表示只选择前 i 个物品且总体积不超过 j 的方案的最大价值.

$$dp[i][j] = \max\{dp[i-1][j], dp[i-1][j-v] + w, dp[i-1][j-2v] + 2w, \dots, dp[i-1][j-sv] + sw\}.$$

$$dp[i][j-v] = \max\{dp[i-1][j-v], dp[i-1][j-2v] + w, \dots, dp[i-1][j-sv] + (s-1)w, dp[i-1][j-(s+1)v] + sw\}.$$

观察知,去掉 $dp[i][j]$ 的第一项和 $dp[i][j-v]$ 的最后一项后,上面每一项都比下面多 w .

设 $r = j \% v$,则每个 $dp[i][j]$ 对应数轴上的点 $r, r+v, r+2v, \dots$,且每一项的结果是前 s 项的最大值再加上若干个 w 的结果.用单调队列维护长度为 s 的滑动窗口中的最大值.总时间复杂度 $O(nV)$.

代码

```

1  const int MAXM = 2e4 + 5;
2  int n, m; // 物品数、总体积
3  // dp[i][j]表示只选择前i个物品且总体积不超过j的方案的最大价值,backup[]为其备份
4  int dp[MAXM], backup[MAXM];
5  int que[MAXM]; // deque
6
7  void solve() {
8      cin >> n >> m;

```

```

9   for (int i = 0; i < n; i++) {
10      int v, w, s; cin >> v >> w >> s;
11      memcpy(backup, dp, so(dp)); // 备份
12      for (int r = 0; r < v; r++) { // 枚举j%v
13         int hh = 0, tt = -1;
14         for (int k = r; k <= m; k += v) {
15            if (hh <= tt && que[hh] < k - s * v) que[hh++]; // 队头离开滑动窗口
16            if (hh <= tt) dp[k] = max(dp[k], backup[que[hh]] + (k - que[hh]) / v * w);
17            while (hh <= tt && backup[que[tt]] - (que[tt] - r) / v * w <= backup[k] - (k - r) / v * w)
18               tt--; // 维护单调队列
19            que[++tt] = k;
20         }
21      }
22   }
23   cout << dp[m];
24 }
25
26 int main() {
27     solve();
28 }

```

10.1.6 分组背包

多重背包是分组背包的特殊情况,分组背包是0-1背包的变种.

题意

有 N ($1 \leq N \leq 100$)组物品和一个容量为 V ($1 \leq V \leq 100$)的背包,第 i 组有 S_i ($1 \leq S_i \leq 100$)个物品,每组物品至多只能选1个.第 i 个物品组的第 j 件物品的体积为 v_{ij} ($1 \leq v_{ij} \leq 100$),价值为 w_{ij} ($1 \leq w_{ij} \leq 100$).问将哪些物品装入背包可使得这些物品的总体积不超过背包容量,且总价值最大,输出最大总价值.

思路

0-1背包多一层循环.因数据范围 ≤ 100 ,故可跑 $O(n^3)$.

代码

```

1  const int MAXN = 105;
2  int v[MAXN], w[MAXN];
3  int dp[MAXN];
4
5  int main() {
6      for (int i = 0; i < N; i++) {
7          int s;
8          cin >> s;
9          for (int j = 0; j < s; j++) cin >> v[j] >> w[j];
10         for (int j = V; j >= 0; j--) {
11             for (int k = 0; k < s; k++)
12                 if (j >= v[k]) dp[j] = max(dp[j], dp[j - v[k]] + w[k]);
13         }
14     }
15     cout << dp[V];
16 }

```

10.1.7 采药

题意

有不同的草药,采每一株需花费一些时间,每一株有一定的价值.给定一段时间,求能采到的草药的最大价值.

第一行输入两个整数 t, m ($1 \leq t \leq 1000, 1 \leq m \leq 100$),分别表示可供采药的时间和草药数.接下来 m 行每行输入两个范围为 $[1, 100]$ 的整数,分别表示采摘某株草药的时间和该草药的价值.

输出在给定时间内能采到的草药的最大价值.

代码

```
1  const int MAXN = 1005;
2  int t, n; // 总时间、物品数
3  int dp[MAXN];
4
5  void solve() {
6      cin >> t >> n;
7      for (int i = 1; i <= n; i++) {
8          int v, w; cin >> v >> w;
9          for (int j = t; j >= v; j--) dp[j] = max(dp[j], dp[j - v] + w);
10     }
11     cout << dp[t];
12 }
13
14 int main() {
15     solve();
16 }
```

10.1.8 装箱问题

题意

有一容积为 V 的箱子.现有 n 个物品,每个物品有一个体积(正整数).在 n 个物品中任取若干个装入箱内,求箱子剩余空间的最小值.

第一行输入一个整数 V ($0 < V \leq 2e4$).第二行输入一个整数 n $0 < n \leq 30$.接下来 n 行每行输入一个范围为 $[1, 1e4]$ 的整数,分别表示 n 个物品的体积.

思路

将每个物品的体积视为其价值.

代码

```
1  const int MAXM = 2e4 + 5;
2  int n, m; // 物品数、总体积
3  int dp[MAXM];
4
5  void solve() {
6      cin >> m >> n;
7      for (int i = 0; i < n; i++) {
8          int v; cin >> v;
9          for (int j = m; j >= v; j--) dp[j] = max(dp[j], dp[j - v] + v);
10     }
11     cout << m - dp[m];
12 }
13
14 int main() {
15     solve();
16 }
```

10.1.9 二维费用背包

题意

有一个容量为 V 的背包,其能承受的最大重量为 M .现有 n 个物品,每个物品的体积、重量、价值依次为 v_i, m_i, w_i ,每个物品只能用一次.求将哪些物品装入背包,可使得物品总体积不超过背包容量,且总重量不超过背包最大称重量,且物品的价值之和最大,输出最大价值.

第一行输入三个整数 n, V, M ($1 \leq n \leq 1000, 0 \leq V, M \leq 100$).接下来 n 行每行输入三个字母 v_i, m_i, w_i ($0 \leq v_i, m_i \leq 100, 0 \leq w_i \leq 1000$).

思路

$dp[i][j][k]$ 表示只选前 i 个物品、总体积不超过 j 、总重量不超过 k 的选法中物品价值之和的最大值.

按是否包含物品 i 分类,状态转移方程 $dp[i][j][k] = \max\{dp[i-1][j][k], dp[i-1][j-v_i][k-m_i] + w_i\}$.

代码

```
1  const int MAXN = 105;
2  int n, V, M; // 物品数、背包容量、背包最大承重
3  int dp[MAXN][MAXN];
4
5  void solve() {
6      cin >> n >> V >> M;
7      for (int i = 0; i < n; i++) {
8          int v, m, w; cin >> v >> m >> w;
9          for (int j = V; j >= v; j--)
10             for (int k = M; k >= m; k--) dp[j][k] = max(dp[j][k], dp[j-v][k-m] + w);
11     }
12     cout << dp[V][M];
13 }
14
15 int main() {
16     solve();
17 }
```

10.1.10 收服宠物小精灵

题意

对每个小精灵,小智可能需用很多个精灵球才能收服它,收服过程中小精灵会对皮卡丘造成一定的伤害,进而减少皮卡丘的体力.皮卡丘体力 ≤ 0 时,小智需结束狩猎,使得皮卡丘体力 ≤ 0 的小精灵不会被收服.精灵球用完时,小智需结束狩猎.小智遇到小精灵时有两种选择:收服它或离开.若他选择收服,则会扔出一定能收服该小精灵的精灵球,同时皮卡丘受到相应的伤害;若他选择离开,则他不消耗精灵球,皮卡丘也不损失体力.小智有两个目标:①收服尽可能多的小精灵;②若可收服的小精灵数一样,希望皮卡丘受到的伤害之和最小.

第一行输入三个整数 n, m, k ($1 \leq n \leq 1000, 1 \leq m \leq 500, 1 \leq k \leq 100$),分别表示小智的精灵球数、皮卡丘的初始体力、小精灵数.接下来 k 行每行输入两个整数描述一个小精灵:收服该小精灵所需的精灵球数、收服过程中对皮卡丘造成的伤害.

输出两个整数 c, r ,表示最多能收服 c 个小精灵,收服后皮卡丘剩余体力的最大值为 r .

思路

每个小精灵收服或不收服,属于0-1背包问题.每个小精灵视为一个物品,其花费为精灵球数、皮卡丘的体力,价值为1,转化为二维费用背包.

$dp[i][j][k]$ 表示只选前 i 个小精灵、花费精灵球数不超过 j 、消耗皮卡丘的体力不超过 k 的方案能收服的小精灵数的最大值.

状态转移方程: $dp[i][j][k] = \max\{dp[i-1][j][k], dp[i-1][j-v_1[i]][k-v_2[i]] + 1\}$.

$c = dp[k][n][m]$.所有满足 $dp[k][n][m'] = dp[k][n][m]$ 的最小的 m' 对应的 $m - m'$ 即为剩余体力的最大值.

注意皮卡丘体力不能为0,否则此时的小精灵不能被收服,故枚举体力时最大值为 $m - 1$.

代码

```
1  const int MAXN = 1005, MAXM = 505;
2  int v1, v2, n; // 精灵球数、皮卡丘初始体力、小精灵数
3  int dp[MAXN][MAXM];
4
5  void solve() {
6      cin >> v1 >> v2 >> n;
7      for (int i = 0; i < n; i++) {
8          int v1, v2; cin >> v1 >> v2;
9          for (int j = v1; j >= v1; j--)
10             for (int k = v2 - 1; k >= v2; k--) dp[j][k] = max(dp[j][k], dp[j - v1][k - v2] + 1);
11     }
12
13     cout << dp[v1][v2 - 1] << ' '; // 皮卡丘体力不能为0,下同
14     int k = v2 - 1;
15     while (k > 0 && dp[v1][k - 1] == dp[v1][v2 - 1]) k--;
16     cout << v2 - k;
17 }
18
19 int main() {
20     solve();
21 }
```

10.1.11 数字组合

题意

给定 n ($1 \leq n \leq 100$)个整数 $a_1, \dots, a_n$ ($1 \leq a_i \leq 1000$),从中选出若干个使得它们之和为 m ($1 \leq m \leq 1e4$),求方案数.数据保证答案在int范围内.

思路

将和 m 视为背包的容量,将每个数视为一个物品,其值 a_i 视为体积,问题转化为求总体积恰为 m 的方案数.

$dp[i][j]$ 表示只选前 i 个物品且总体积恰为 j 的方案数.

按是否选 a_i 分类,状态转移方程: $dp[i][j] = dp[i - 1][j] + dp[i - 1][j - a_i]$.

初始条件和为0,什么都不选只有一种方案,即 $dp[0] = 1$.

代码

```
1  const int MAXN = 1e4 + 5;
2  int n, m; // n个数、和为m
3  int dp[MAXN];
4
5  void solve() {
6      cin >> n >> m;
7
8      dp[0] = 1; // 都不选只有一种方案
9      for (int i = 0; i < n; i++) {
10         int v; cin >> v;
11         for (int j = m; j >= v; j--) dp[j] += dp[j - v];
12     }
```

```

13     cout << dp[m];
14 }
15
16 int main() {
17     solve();
18 }

```

10.1.12 买书

题意

某人有 n ($0 \leq n \leq 1000$)元全部用于买书,书的价格有10, 20, 50, 100元四种,每本书可购买多本.求他买书的方案数.

思路

$dp[i][j]$ 表示只选前 i 个物品,且体积之和恰为 j 的方案数.

按第 i 个物品选的个数分类,状态转移方程: $dp[i][j] = dp[i-1][j] + \sum_{k=1}^s dp[i-1][j - kv_i]$.

注意到 $dp[i][j] = dp[i-1][j] + dp[i-1][j-v] + dp[i-1][j-2v] + \dots + dp[i-1][j-sv]$,

而 $dp[i][j-v] = dp[i-1][j-v] + dp[i-1][j-2v] + \dots + dp[i-1][j-sv]$,

故 $dp[i][j] = dp[i-1][j] + dp[i][j-v]$.

代码

```

1  const int MAXN = 1005;
2  int m;    // 和为m
3  int v[4] = { 10,20,50,100 };
4  int dp[MAXN];
5
6  void solve() {
7      cin >> m;
8
9      dp[0] = 1;
10     for (int i = 0; i < 4; i++)
11         for (int j = v[i]; j <= m; j++) dp[j] += dp[j - v[i]];
12     cout << dp[m];
13 }
14
15 int main() {
16     solve();
17 }

```

10.1.13 货币系统I

题意

给定一个包含 n ($1 \leq n \leq 15$)种面值的货币系统,求组成金额 m ($0 \leq m \leq 3000$)的方案数.

思路

同10.1.12 买书.

代码

```

1  const int MAXN = 3005;
2  int n, m; // 物品数、总体积
3  ll dp[MAXN];
4
5  void solve() {
6      cin >> n >> m;
7
8      dp[0] = 1; // 0元的方案为1
9      for (int i = 0; i < n; i++) {
10         int v; cin >> v;
11         for (int j = v; j <= m; j++) dp[j] += dp[j - v];
12     }
13     cout << dp[m];
14 }
15
16 int main() {
17     solve();
18 }

```

10.1.14 货币系统II

题意

有 n 种不同面额的货币,其中第 i 种的面额为 $a[i]$,每种面额的货币有无限张.将货币总数为 n ,面额数组为 $a[1 \cdots n]$ 的货币系统记作 (n, a) .在一完善的货币系统中,任一非负整数金额 x 都应能表示出,即对 $\forall x \in \mathbb{N}, \exists n \text{ 个 } t[i] \in \mathbb{N} \text{ s.t. } \sum a[i] \cdot t[i] = x$.称两个货币系统 (n, a) 和 (m, b) 等价,如果对 $\forall x \in \mathbb{N}$,它要么都能被两货币系统表出,要么都不能被两货币系统表出.

有 t ($1 \leq t \leq 20$)组测试数据.每组测试数据第一行输入一个整数 n ($1 \leq n \leq 100$).第二行输入 n 个整数 $a[1], \dots, a[n]$.

对每组测试数据,求一个货币系统 (m, b) s.t. 其与 (n, a) 等价,且 m 尽量小,输出最小的 m .

思路

$b[]$ 是 $a[]$ 的极大独立集,则 $a[]$ 中的元素都能被 $b[]$ 线性表出,且 $b[]$ 中的每个元素不能被其他 $b[]$ 中的元素表出.

$b[]$ 中的元素都来自 $a[]$.

[证] 若不然,设 $b_i = a_1 + a_2 \notin a[]$.因 a_1, a_2 能被 $b[]$ 线性表出,则 b_i 某能被 $b[]$ 线性表出,则无需有 b_i .

将 $a[]$ 升序排列,依次检查每个 a_i 能否被 $a_1 \sim a_{i-1}$ 表出,若能则不选,否则必选,故方案确定.

下面代码中 $dp[]$ 也可改为bool数组,则内层循环变为:

```

1  for (int j = a[i]; j <= m; j++) dp[j] |= dp[j - a[i]];

```

代码

```

1  const int MAXN = 2.5e4 + 5;
2  int n;
3  int a[MAXN];
4  int dp[MAXN];
5
6  void solve() {

```

```

7   cin >> n;
8   for (int i = 0; i < n; i++) cin >> a[i];
9   sort(a, a + n);
10
11  memset(dp, 0, so(dp));
12  dp[0] = 1; // 0元只有一种方案
13
14  int m = a[n - 1];
15  int ans = 0;
16  for (int i = 0; i < n; i++) {
17      if (!dp[a[i]]) ans++; // a[i]不能被表示出来,则必选
18
19      for (int j = a[i]; j <= m; j++) dp[j] += dp[j - a[i]];
20  }
21  cout << ans << endl;
22 }
23
24 int main() {
25     CaseT // 单测时注释掉该行
26     solve();
27 }

```

10.1.15 庆功会

题意

有 m ($1 \leq m \leq 6000$)元.现有 n ($1 \leq n \leq 500$)种物品,每种物品用三个整数 v, w, s ($1 \leq v \leq 100, 1 \leq w \leq 1000, 1 \leq s \leq 10$)描述,分别表示其价格、价值和最大购买数.求不超过 m 元的购买方案中物品价值之和的最大值.

思路

多重背包.朴素做法时间复杂度 $O(nms)$,最坏 $1e7$,可过.

代码

```

1   const int MAXN = 5005;
2   int n, m;
3   int dp[MAXN];
4
5   void solve() {
6       cin >> n >> m;
7       for (int i = 0; i < n; i++) {
8           int v, w, s; cin >> v >> w >> s;
9           for (int j = m; j >= 0; j--) {
10              for (int k = 0; k <= s && k * v <= j; k++)
11                  dp[j] = max(dp[j], dp[j - k * v] + k * w);
12          }
13      }
14      cout << dp[m];
15  }
16
17  int main() {
18      solve();
19  }

```

10.1.16 混合背包问题

题意

有一个容量为 V 的背包,现有 n 种物品,每种物品的体积为 v_i ,价值为 w_i .物品分为如下三类:①第一类物品只能用1次($0-1$ 背包);②第二类物品可用无限次(完全背包);③第三类物品最多用 s_i 次(多重背包).求将哪些物品装入背包,使得物品总体积不超过背包容量,且物品价值之和最大.

第一行输入两个整数 n, V ($1 \leq n, V \leq 1000$),分别表示物品种数和背包容量.接下来 n 行每行输入三个整数 v_i, w_i, s_i ($1 \leq v_i, w_i \leq 1000, -1 \leq s_i \leq 1000$),分别表示第 i 种物品的体积、价值和数量,其中① $s_i = -1$ 表示该物品只能用1次;② $s_i = 0$ 表示该物品能用无限次;③ $s_i > 0$ 表示该物品最多能用 s_i 次.

思路

注意到三种背包的状态转移方程都只用到了前一层的信息,故只需根据前一层的物品的类型,判断改层应用哪个状态转移方程即可.

代码

```
1  const int MAXN = 1005;
2  int n, m; // 物品数、背包体积
3  int dp[MAXN];
4
5  void solve() {
6      cin >> n >> m;
7      for (int i = 0; i < n; i++) {
8          int v, w, s; cin >> v >> w >> s;
9          if (!s) // 完全背包
10             for (int j = v; j <= m; j++) dp[j] = max(dp[j], dp[j - v] + w);
11         else {
12             if (s == -1) s = 1; // 0-1背包是s=1的多重背包
13
14             for (int k = 1; k <= s; k <= 1) { // 多重背包二进制优化
15                 for (int j = m; j >= k * v; j--) dp[j] = max(dp[j], dp[j - k * v] + k * w);
16                 s -= k;
17             }
18
19             if (s)
20                 for (int j = m; j >= s * v; j--) dp[j] = max(dp[j], dp[j - s * v] + s * w);
21         }
22     }
23     cout << dp[m];
24 }
25
26 int main() {
27     solve();
28 }
```

10.1.17 潜水员

题意

潜水员潜水需要带含两种气体的气缸:氧气和但其.潜水员有一定数量的气缸,每个气缸有各自的重量和气体容量.潜水员为完成其工作需一定量的氧气和但其,求其完成工作所需气缸的总重的最小值.

第一行输入两个整数 m, n ($1 \leq m \leq 21, 1 \leq n \leq 79$),分别表示氧气和氮气的所需量.第二行输入一个整数 k ($1 \leq k \leq 1000$).接下来 k 行每行输入三个整数 a_i, b_i, c_i ($1 \leq a_i \leq 21, 1 \leq b_i \leq 79, 1 \leq c_i \leq 800$),分别代表第 i 个气缸中的氧气含量、氮气含量和气缸重量.

思路

$dp[i][j][k]$ 表示只选前 i 个物品,且氧气含量至少为 j ,氮气含量至少为 k 的选法中所需气缸的总重的最小值.

以是否选第 i 个气缸分类,状态转移方程: $dp[i][j][k] = \min\{dp[i-1][j][k], dp[i-1][j-v_1][k-v_2] + w\}$.

注意转移时若 $j < v_1$ 或 $k < v_2$,因每个气缸的 $a_i, b_i \geq 1$,则有一个气体含量为0时,另一气体含量也为0,

只需在 $j < v_1$ 或 $k < v_2$ 时从 $dp[i][0][k]$ 或 $dp[i][j][0]$ 转移即可.

初始条件:什么都不选时气缸总重为0,即 $dp[0][0][0]$,其余 $dp[0][j][k] = INF$ ($j, k \geq 1$).

代码

```
1  const int MAXN = 22, MAXM = 80;
2  int n, m, k; // 氧气所需量、氮气所需量、物品数
3  int dp[MAXN][MAXM];
4
5  void solve() {
6      cin >> n >> m >> k;
7
8      memset(dp, INF, sizeof(dp));
9      dp[0][0] = 0; // 什么都不选时气缸总重为0
10
11     while (k--) {
12         int v1, v2, w; cin >> v1 >> v2 >> w;
13         for (int i = n; i >= 0; i--) {
14             for (int j = m; j >= 0; j--)
15                 dp[i][j] = min(dp[i][j], dp[max(0, i - v1)][max(0, j - v2)] + w);
16         }
17     }
18     cout << dp[n][m];
19 }
20
21 int main() {
22     solve();
23 }
```

10.1.18 机器分配

题意

由 m 台相同的设备,现要分配给 n 个公司,分配原则:每个公司都可获得任意数目的设备,设备总台数不超过 m .得到设备的公司可获得一定盈利,盈利与分配的设备数有关.如何分配设备可使得盈利最大,求出最大盈利.

第一行输入两个数 n, m ($1 \leq n \leq 10, 1 \leq m \leq 15$).第二行输入一个 $n \times m$ 的矩阵,其中元素 (i, j) 表示第 i 个公司分配到 j 台设备时的盈利.

第一行输出一个整数,表示最大盈利.接下来 n 行每行输出两个数,表示公司编号和该公司获得的设备台数.若答案不唯一,输出任意方案.

思路

将每个公司视为一个物品组,每个物品组为该公司分配到的设备台数,每个物品的体积为该公司分配到的设备的台数,价值为该公司得到对应数目的物品的盈利,转化为分组背包问题.

代码

```

1  const int MAXN = 11, MAXM = 16;
2  int n, m;    // 公司数、设备数
3  int w[MAXN][MAXM]; // w[i][j]表示第i个公司分配j台设备的盈利
4  int dp[MAXN][MAXM];
5  int ans[MAXN]; // ans[i]表示第i个公司分配的设备数
6
7  void solve() {
8      cin >> n >> m;
9      for (int i = 1; i <= n; i++)
10         for (int j = 1; j <= m; j++) cin >> w[i][j];
11
12     for (int i = n; i >= 1; i--) { // 倒序枚举公司
13         for (int j = 0; j <= m; j++) { // 枚举设备数
14             for (int k = 0; k <= j; k++) // 枚举分配给该公司的设备数
15                 dp[i][j] = max(dp[i][j], dp[i + 1][j - k] + w[i][k]);
16         }
17     }
18     cout << dp[1][m] << endl;
19
20     int j = m; // 从最大体积往回推
21     for (int i = 1; i <= n; i++) { // 枚举公司
22         for (int k = 0; k <= j; k++) {
23             if (dp[i][j] == dp[i + 1][j - k] + w[i][k]) {
24                 ans[i] = k; // 记录公司分配到的设备数
25                 j -= k;
26                 break; // 找到一组解即break,保证方案字典序最小
27             }
28         }
29     }
30
31     for (int i = 1; i <= n; i++) cout << i << ' ' << ans[i] << endl;
32 }
33
34 int main() {
35     solve();
36 }

```

10.1.19 开心的金明

题意

金明有 n 元,可购买若干物品.他将每个物品规定了一个重要度,用范围 $[1, 5]$ 的整数描述,数字越大越重要.现他查到了每件物品的价格(整数),他希望在总价格不超过 n 元的前提下使得每件物品的价格与重要度乘积之和最大.设第 j 件物品的价格为 v_j ,重要度为 w_j ,则选择编号为 $j_1, \dots, j_k$ 的物品的价格与重要度乘积之和为 $\sum_{i=1}^k v_{j_i} w_{j_i}$.

第一行输入两个整数 n, m ($1 \leq n < 3e4, 1 \leq m < 25$),分别表示总钱数和物品数.接下来 m 行每行输入两个整数 v, w ($0 \leq v < 1e4, 1 \leq w \leq 5$),分别表示每件物品的价格和重要度.

输出在总价格不超过 n 元的前提下,每件物品的价格与重要度乘积之和的最大值,数据保证答案不超过 $1e8$.

代码

```

1  const int MAXN = 3e4 + 5;
2  int n, m;    // 总钱数、物品数
3  int dp[MAXN];
4

```

```

5 void solve() {
6     cin >> m >> n;
7     for (int i = 0; i < n; i++) {
8         int v, w; cin >> v >> w;
9         w *= v;
10        for (int j = m; j >= v; j--) dp[j] = max(dp[j], dp[j - v] + w);
11    }
12    cout << dp[m];
13 }
14
15 int main() {
16     solve();
17 }

```

10.1.20 有依赖的背包问题

题意

有一容量为 V 的背包.现有 n 个物品,物品间有依赖关系,依赖关系是一棵树,若选择一个物品,则必须选择其父节点.每件物品的编号为 i ($1 \leq i \leq n$),体积为 v_i ,价值为 w_i ,依赖父节点 p_i .求将哪些物品装入背包,在物品总体积不超过背包容量的前提下,物品价值之和最大,输出最大价值.

第一行输入两个整数 n, V ($1 \leq n, V \leq 100$).接下来 n 行每行输入三个整数 v, w, p ($1 \leq v, w \leq 100, -1 \leq p \leq n$),分别表示物品的体积、价值和依赖的物品编号,其中 $p_i = -1$ 表示根节点.数据保证依赖关系构成一棵树.

思路

将每棵子树视为一个物品组,转化为分组背包. $dp[u][j]$ 表示只选以 u 为根节点的子树中的物品且总体积不超过 j 的方案的最大价值.按选择的子树的根节点分类.注意到若每棵子树内部按方案分类,设该子树包含 k 个节点,则最坏需枚举 2^k 种情况,会TLE.考虑按该子树占用的体积分类,这样只需枚举 $(V + 1)$ 种情况.

代码

```

1  const int MAXN = 105;
2  int n, m; // 物品数、背包体积
3  int v[MAXN], w[MAXN];
4  vi edges[MAXN];
5  int dp[MAXN][MAXN];
6
7  void dfs(int u) { // 当前节点
8      for (auto son : edges[u]) { // 枚举物品组
9          dfs(son);
10
11         // 分组背包
12         for (int j = m - v[u]; j >= 0; j--) { // 循环体积,因要选u节点,故j的最大值为m-v[u]
13             for (int k = 0; k <= j; k++)
14                 dp[u][j] = max(dp[u][j], dp[u][j - k] + dp[son][k]);
15         }
16     }
17
18     for (int i = m; i >= v[u]; i--) dp[u][i] = dp[u][i - v[u]] + w[u]; // 放入物品u
19     for (int i = 0; i < v[u]; i++) dp[u][i] = 0; // 当前剩余体积不足v[u],则不能选节点u,价值为0
20 }
21
22 void solve() {
23     cin >> n >> m;
24
25     int root;
26     for (int i = 1; i <= n; i++) {
27         int p; cin >> v[i] >> w[i] >> p;

```



```

28     if (~p) edges[p].push_back(i);
29     else root = i;
30 }
31
32 dfs(root); // 从根节点开始搜
33
34 cout << dp[root][m];
35 }
36
37 int main() {
38     solve();
39 }

```

10.1.21 背包问题求方案数

题意

有一容量为 V 的背包.现有 n 件物品,每件物品有体积和价值,每件物品只能用一次.求将哪些物品装入背包可使得物品的总体积不超过背包容量且总价值最大,输出最优选法的方案数,答案对 $1e9 + 7$ 取模.

第一行输入两个整数 n, V ($1 \leq n, V \leq 1000$).接下来 n 行每行输入两个整数 v, w ($1 \leq v, w \leq 1000$),表示物品的体积和价值.

思路

$dp1[i][j]$ 表示只选前 $(i - 1)$ 个物品且物品体积之和恰为 j 的最大价值, $dp2[i][j]$ 表示 $dp1[i][j]$ 取得最大值的方案数.

状态转移方程: $dp1[i][j] = \max\{dp1[i - 1][j], dp1[i - 1][j - v_i] + w_i\}$.

①若 $dp1[i - 1][j]$ 是最大值,则 $dp2[i][j] += dp2[i - 1][j]$.

②若 $dp1[i - 1][j - v_i] + w_i$ 是最大值,则 $dp2[i][j] = dp2[i - 1][j - v_i]$.

代码

```

1  const int MAXN = 1005;
2  const int MOD = 1e9 + 7;
3  int n, m; // 物品数、背包体积
4  int dp1[MAXN]; // dp1[i][j]表示只选前(i-1)个物品且物品体积之和恰为j的最大价值
5  int dp2[MAXN]; // dp2[i][j]表示dp1[i][j]取得最大值的方案数
6
7  void solve() {
8      cin >> n >> m;
9
10     memset(dp1, -INF, so(dp1));
11     dp1[0] = 0, dp2[0] = 1; // 什么都不选的价值为0,方案数为1
12
13     for (int i = 0; i < n; i++) {
14         int v, w; cin >> v >> w;
15         for (int j = m; j >= v; j--) {
16             int maxvalue = max(dp1[j], dp1[j - v] + w);
17             int cnt = 0; // 取得最大值的方案数
18             if (maxvalue == dp1[j]) cnt = ((1ll)cnt + dp2[j]) % MOD;
19             if (maxvalue == dp1[j - v] + w) cnt = ((1ll)cnt + dp2[j - v]) % MOD;
20             dp1[j] = maxvalue, dp2[j] = cnt;
21         }
22     }
23
24     int ans = 0; // 最大价值
25     for (int i = 0; i <= m; i++) ans = max(ans, dp1[i]); // 枚举用的体积
26
27     int cnt = 0; // 方案数
28     for (int i = 0; i <= m; i++)

```

```

29     if (ans == dp1[i]) cnt = ((1l)cnt + dp2[i]) % MOD;
30     cout << cnt;
31 }
32
33 int main() {
34     solve();
35 }

```

DP求具体方案对应最短路中求具体路径,只需在转移过程中记录当前决策(每个节点的前驱),再从终点倒推即可.

10.1.22 背包问题求具体方案

题意

有一容量为 V 的背包.现有编号 $1 \sim n$ 的 n 件物品,每件物品有体积和价值,每件物品只能用一次.求将哪些物品装入背包可使得物品的总体积不超过背包容量且总价值最大,输出(所选的物品编号构成的序列)字典序最小的方案.

第一行输入两个整数 n, V ($1 \leq n, V \leq 1000$).接下来 n 行每行输入两个整数 v, w ($1 \leq v, w \leq 1000$),表示物品的体积和价值.

输出字典序最小的所选的物品编号构成的序列.

思路

求具体方案即判断每个物品是否被选.

为求字典序最小的方案,可依次考察编号为 $1 \sim n$ 的物品:若最优解中该物品①必选,则选;②必不选,则不选;③可选可不选,则选.为防止最后将序列倒过来,可倒序枚举每个物品.

代码

```

1  const int MAXN = 1005;
2  int n, v; // 物品数、总体积
3  int v[MAXN], w[MAXN];
4  int dp[MAXN][MAXN];
5
6  void solve() {
7      cin >> n >> v;
8      for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
9
10     for (int i = n; i >= 1; i--) { // 倒序枚举物品
11         for (int j = 0; j <= v; j++) {
12             dp[i][j] = dp[i + 1][j]; // 不选该物品
13             if (j >= v[i]) dp[i][j] = max(dp[i][j], dp[i + 1][j - v[i]] + w[i]); // 选该物品
14         }
15     }
16
17     int j = v; // 从最大体积倒推
18     for (int i = 1; i <= n; i++) { // 从前往后枚举每个物品,可以选则必选
19         if (j >= v[i] && dp[i][j] == dp[i + 1][j - v[i]] + w[i]) {
20             cout << i << ' ';
21             j -= v[i];
22         }
23     }
24 }
25
26 int main() {
27     solve();

```

10.1.23 能量石

题意

某怪物想吃 n 块能量石,每一次只能吃一块,其中吃掉第 i 块花费时间 s_i 秒.不同能量石包含的能量可能不同,能量石会随时间流逝逐渐失去能量.第 i 块能量是初始时包含 e_i 单位的能量,每秒失去 l_i 单位的能量,能量至多降到0.怪物开始吃一块能量石时会立刻获得该能量石所含的全部能量,无论实际吃完的时间.求它通过吃能量石可获得的最大能量.

有 t ($1 \leq t \leq 10$)组测试数据.每组测试数据第一行输入一个整数 n ($1 \leq n \leq 100$),表示能量石数.接下来 n 行每行输入三个整数 s_i, e_i, l_i ($1 \leq s_i \leq 100, 1 \leq e_i \leq 1e5, 0 \leq l_i \leq 1e5$),描述第 i ($1 \leq i \leq n$)块能量石.

对每组测试数据,输出"Case #x: y",其中 x 为测试数据的编号(从1开始), y 为怪物通过吃能量石可获得的最大能量.

思路

显然最优解中不会吃能量已降为0的能量石.

考察最优解中吃 i 和 $(i+1)$ 相邻两块能量石的顺序:

①若先吃 i ,再吃 $(i+1)$,获得的能量为 $E'_i + E'_{i+1} - s_i \cdot l_{i+1}$,

其中 E'_i 和 E'_{i+1} 分别表示开始吃第 i 、 $(i+1)$ 块能量石时它们的能量,下同.

②若先吃 $(i+1)$,再吃 i ,获得的能量为 $E'_{i+1} + E'_i - s_{i+1} \cdot l_i$.

故先吃 i 再吃 $(i+1)$ 更优当且仅当 $s_i \cdot l_{i+1} < s_{i+1} \cdot l_i$,贪心策略的证明类似于8.5.1 耍杂技的牛.

$dp[i][j]$ 表示排序后只选前 i 块能量石且总耗时恰为 j 的方案可获得的能量的最大值.

按是否吃第 i 块能量石分类,状态转移方程: $dp[i][j] = \max\{dp[i-1][j], dp[i-1][j-s_i] + e_i - (j-s_i) * l_i\}$.

代码

```
1  const int MAXN = 1005, MAXM = 1e4 + 5;
2  const int MOD = 1e9 + 7;
3  int n;
4  struct Stone {
5      int s, e, l;
6
7      bool operator<(const Stone& b) { return s * b.l < b.s * l; }
8  }stones[MAXN];
9  int dp[MAXN][MAXM]; // dp[i][j]表示排序后只选前i块能量石且总耗时不超过j的方案可获得的能量的最大值
10
11 void solve() {
12     int T; cin >> T;
13     for (int C = 1; C <= T; C++) {
14         cin >> n;
15         int m = 0; // 总体积
16         for (int i = 1; i <= n; i++) {
17             int s, e, l; cin >> s >> e >> l;
18             stones[i] = { s, e, l };
19             m += s;
20         }
21
22         sort(stones + 1, stones + n + 1);
23
24         for (int i = 1; i <= n; i++) {
```

```

25     for (int j = 0; j <= m; j++) {
26         dp[i][j] = dp[i - 1][j];
27         if (j >= stones[i].s) {
28             auto [s, e, l] = stones[i];
29             dp[i][j] = max(dp[i][j], dp[i - 1][j - s] + max(0, e - l * (j - s))); // 注意e-l*(j-s)要与0
取max
30         }
31     }
32 }
33
34 int ans = 0;
35 for (int i = 0; i <= m; i++) ans = max(ans, dp[n][i]);
36 printf("Case #%d: %d\n", C, ans);
37 }
38 }
39
40 int main() {
41     solve();
42 }

```

10.1.24 金明的预算方案

题意

金明有 n 元,可购买若干物品.他将物品分为两类:主件和附件,其中每个主件有0, 1, 2个附件,每个附件从属于某个主件,附件雾从属于自己的附件.若要买附件,则需现买其所属的主件.他将每个物品规定了一个重要度,用范围 $[1, 5]$ 的整数描述,数字越大越重要.现他查到了每件物品的价格(整数),他希望在总价格不超过 n 元的前提下使得每件物品的价格与重要度乘积之和最大.设第 j 件物品的价格为 v_j ,重要度

为 w_j ,则选择编号为 $j_1, \dots, j_k$ 的物品的价格与重要度乘积之和为 $\sum_{i=1}^k v_{j_i} w_{j_i}$.

第一行输入两个整数 n, m ($1 \leq n < 3.2e4, 1 \leq m < 60$),分别表示总钱数和物品数.接下来 m 行每行输入两个整数 v, p, q ($0 \leq v < 1e4, 1 \leq p \leq 5, q \geq 0$),分别表示每件物品的价格、重要度、该物品是主件还是附件,其中 $q = 0$ 表示该物品是主件; $q > 0$ 表示该物品是从属于主件 q 的附件.

输出在总价格不超过 n 元的前提下,每件物品的价格与重要度乘积之和的最大值,数据保证答案不超过 $2e5$.

思路

将每个主件及其附件视为一个物品组,其中是物品是主件及其附件(若有)的所有子集,转化为分组背包问题.

代码

```

1  const int MAXN = 60, MAXM = 3.2e4 + 5;
2  int n, m; // 总钱数、物品数
3  pii master[MAXN]; // 主件的价格、价格与重要度之积
4  vii attachment[MAXN]; // 附件的价格、价格与重要度之积
5  int dp[MAXM];
6
7  void solve() {
8      cin >> m >> n;
9      for (int i = 1; i <= n; i++) {
10         int v, p, q; cin >> v >> p >> q;
11         p *= v; // 价格与重要度之积
12         if (!q) master[i] = { v, p };
13         else attachment[q].push_back({ v, p });
14     }
15
16     for (int i = 1; i <= n; i++) { // 枚举每个物品
17         for (int u = m; u >= 0; u--) {
18             for (int j = 0; j < 1 << attachment[i].size(); j++) { // 枚举所有子集

```

```

19     auto [v, w] = master[i];
20     for (int k = 0; k < attachment[i].size(); k++) { // 枚举每个二进制数位
21         if (j >> k & 1)
22             v += attachment[i][k].first, w += attachment[i][k].second;
23     }
24
25     if (u >= v) dp[u] = max(dp[u], dp[u - v] + w);
26 }
27 }
28 }
29 cout << dp[m];
30 }
31
32 int main() {
33     solve();
34 }

```

10.1.25 Chip Move

原题指路:<https://codeforces.com/contest/1716/problem/D>

题意

有一数轴,初始时你在数轴的原点,向数轴正方向出发,每一步走的长度都是 k 的倍数,每走一步 $k++$.给定整数 n, k ($1 \leq k \leq n \leq 2e5$),对 $\forall x \in [1, n]$,求从原点走遇到 x 处的方案数,答案对998244353取模.

思路

设至多走 cnt 步,则 $k + (k + 1) + (k + 2) + \dots + (k + cnt - 1) \leq n$,解得 $cnt \sim \sqrt{2n - 2k}$.

$dp[i]$ 表示从原点走到 $x = i$ 处的方案数.

根据上一步是 $i - k, i - 2k, \dots$ 分类,状态转移方程: $dp[i] = dp[i - k] + dp[i - 2k] + \dots$.

显然RHS是一个前缀和的形式,可用数组 $pre[i]$ 表示 $dp[i - k] + dp[i - 2k] + \dots$,在转移时同时维护即可.

总时间复杂度 $O(n\sqrt{n})$.

代码

```

1  const int MAXN = 4e5 + 5;
2  const int MOD = 998244353;
3  int n, k;
4  int dp[MAXN]; // dp[i]表示从原点走到x=i处的方案数
5  int pre[MAXN]; // pre[i]表示dp[i-k], dp[i-2k], ...的前缀和
6  int ans[MAXN];
7
8  void solve() {
9      cin >> n >> k;
10
11     dp[0] = 1; // 原点只有一种方案
12     int tot = 0; // 走过的路径的总长
13     while (tot <= n) {
14         for (int i = 0; i <= n; i++) {
15             int tmp = i - k >= 0 ? pre[i - k] : 0; // 上一阶段的前缀和
16             pre[i] = ((1ll)tmp + dp[i]) % MOD; // 更新前缀和
17             dp[i] = tmp;
18             ans[i] = ((1ll)ans[i] + dp[i]) % MOD;
19         }
20         tot += k++;
21     }
22 }

```

```

23     for (int i = 1; i <= n; i++) cout << ans[i] << " \n"[i == n];
24 }
25
26 int main() {
27     solve();
28 }

```

10.1.26 Byfibonacci

原题指路:<https://ac.nowcoder.com/acm/contest/35493/B>

题意

Fibonacci数列: $f_0 = 1, f_1 = 1, f_i = f_{i-1} + f_{i-2} \ (i \geq 2)$.

Fibonacci表示: $\forall x \in \mathbb{N}^*$ 可表示为不同的Fibonacci数之和(f_0 与 f_1 视为不同的Fibonacci数),记作 $F(x)$.对 $\forall x \in \mathbb{N}^*$,定义 $F(x)$ 的权值 $w(F(x))$ 为 x 的Fibonacci表示 $F(x)$ 所含的Fibonacci数之积.给定 $x \in \mathbb{N}^*$,求其所有Fibonacci表示的权值之和.如 $7 = 5 + 2 = 5 + 1 + 1 = 3 + 2 + 1 + 1$,则权值之和 $5 \times 2 + 5 \times 1 \times 1 + 3 \times 2 \times 1 \times 1 = 21$.

有 $t \ (1 \leq t \leq 1e6)$ 组测试数据.每组测试数据输入一个整数 $x \ (1 \leq x \leq 1e7)$.

对每组测试数据,输出 x 的所有Fibonacci表示的权值之和,答案对998244353取模.

思路

先预处理出 $1e7$ 内的Fibonacci数,显然数量不会很多,事实上只有 $n = 34$ 个.

显然0-1背包. $dp[i][j]$ 表示只选前 i 个Fibonacci数且和恰为 j 的权值和.

注意到 x 最大 $1e7$,则对每个物品都枚举 $f_i \sim 1e7$ 的体积时间复杂度 $O(nx)$,会TLE.考虑优化,注意到枚举到第 i 个物品时所能凑出的最大数即第 $1 \sim i$ 个Fibonacci数之和,可用变量 pre 记录第 $1 \sim i$ 个Fibonacci数的前缀和,对每个物品 i ,只需枚举体积 $f_i \sim pre$.

代码

```

1  const int MAXN = 1e7 + 5;
2  const int MOD = 998244353;
3  int fib[MAXN];
4  int dp[MAXN]; // dp[i][j]表示只选前i个Fibonacci数且和恰为j的权值和
5
6  int init() { // 预处理1e7内的Fibonacci数
7      fib[0] = fib[1] = 1;
8      for (int i = 2;; i++) {
9          fib[i] = fib[i - 1] + fib[i - 2];
10         if (fib[i] > 1e7) return i;
11     }
12 }
13
14 void solve() {
15     int n = init(); // 物品数
16
17     dp[0] = 1; // 和为0只能都不选,但后面要做乘法,故初始化为1
18     int pre = 0; // fib[]的前缀和
19     for (int i = 0; i < n; i++) { // 枚举物品
20         pre = min(pre + fib[i], (int)1e7); // 更新前缀和
21         for (int j = pre; j >= fib[i]; j--) // 枚举体积
22             dp[j] = ((1ll)dp[j] + (1ll)dp[j - fib[i]] * fib[i] % MOD) % MOD;
23     }
24
25     CaseT{
26         int x; cin >> x;
27         cout << dp[x] << endl;
28     }

```

```

29 }
30
31 int main() {
32     solve();
33 }

```

10.1.27 Steadily Growing Steam

题意 (512 MB)

有 n ($1 \leq n \leq 100$)张牌,每张牌有一个价值 v_i ($|v_i| \leq 1e9$)和点数 t_i ($1 \leq t_i \leq 13$).先选择至多 k ($0 \leq k \leq n$)张牌将其点数翻倍,再从 n 张牌中选若干张牌分成两堆,要求两堆内点数之和相等,求选出的牌的价值之和的最大值.

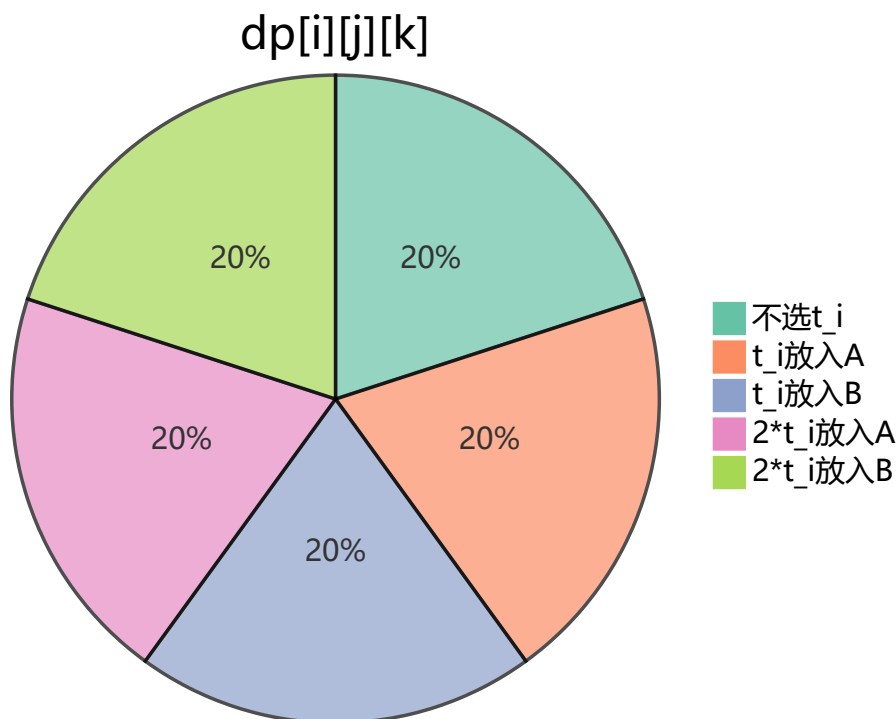
思路

将牌视为物品,点数视为体积,转化为背包.

设两堆的点数之和分别为 A 和 B .注意到不关心 A 和 B 的具体数值,只关心它们是否相等,则可用 $A - B$ 作为 $0 - 1$ 背包的体积,即视放入 A 堆的体积为正,放入 B 堆的体积为负.初始点数 t 最大13,翻倍后最大26,最多100张牌,则体积范围 $(-2600, 2600)$.因数组的下标非负,可加一个2600的偏移量.

$dp[i][j][w]$ 表示只考虑前 i 张牌、还可以翻倍 j 次、当前背包体积为 w 的价值之和的最大值, $dp[100][100][5200]$,最终答案
 $ans = \max_{0 \leq j \leq k} dp[n][j][2600]$.

考虑最后一个不同点,即已考虑完前 $(i - 1)$ 张牌,现考虑第 i 张牌.



$$\text{状态转移方程 } dp[i][j][w] = \max \begin{cases} dp[i-1][j][w], \text{ 不选 } t_i \\ dp[i-1][j][w+t_i] + v_i, t_i \text{ 放入 } A \\ dp[i-1][j][w-t_i] + v_i, t_i \text{ 放入 } B \\ dp[i-1][j-1][w+2*t_i] + v_i, 2t_i \text{ 放入 } A \\ dp[i-1][j-1][w-2*t_i] + v_i, 2t_i \text{ 放入 } B \end{cases}, \text{ 第一维 } O(n), \text{ 第二维 } O(k), \text{ 第三维 } O(4nt), \text{ 转移 } O(1),$$

总时间复杂度 $O(n \cdot k \cdot 4nt) \approx O(4n^3t)$,看起来1s不够,但事实上很多方案不合法,故实际方案数远小于 $O(4n^3t)$.

物品价值数量级 $1e9$,最多100个物品,故dp数组要开ll.空间复杂度

$100 \times 100 \times 5200 \times 8 \div 1024 \div 1024 \approx 397 \text{ MB} < 512 \text{ MB}$,空间足够.

代码I

```

1  const int MAXN = 105, MAXM = 5205;
2  int n, k; // 牌数、最多翻倍次数
3  int v[MAXN], t[MAXN]; // 价值、点数
4  // dp[i][j][w]表示只考虑前i个物品、还可翻倍j次、当前背包体积为w的价值之和最大值
5  ll dp[MAXN][MAXN][MAXM];
6
7  int main() {
8      memset(dp, -INFF, so(dp)); // 置为-INFF,表示未更新
9      dp[0][0][2600] = 0; // k=2600为答案
10
11     cin >> n >> k;
12     for (int i = 1; i <= n; i++) cin >> v[i] >> t[i];
13
14     for (int i = 1; i <= n; i++) { // 枚举物品
15         for (int j = 0; j <= k; j++) { // 枚举翻倍次数
16             for (int w = 0; w <= 5200; w++) { // 枚举背包体积
17                 if (dp[i - 1][j][w] != -INFF) dp[i][j][w] = dp[i - 1][j][w]; // 不选t[i]
18
19                 if (w >= t[i] && dp[i - 1][j][w - t[i]] != -INFF) // t[i]放入A
20                     dp[i][j][w] = max(dp[i][j][w], dp[i - 1][j][w - t[i]] + v[i]);
21
22                 if (w <= 5200 - t[i] && dp[i - 1][j][w + t[i]] != -INFF) // t[i]放入B
23                     dp[i][j][w] = max(dp[i][j][w], dp[i - 1][j][w + t[i]] + v[i]);
24
25                 if (j && w >= 2 * t[i] && dp[i - 1][j - 1][w - 2 * t[i]] != -INFF) // 2*t[i]放入A
26                     dp[i][j][w] = max(dp[i][j][w], dp[i - 1][j - 1][w - 2 * t[i]] + v[i]);
27
28                 if (j && w <= 5200 - 2 * t[i] && dp[i - 1][j - 1][w + 2 * t[i]] != -INFF) // 2*t[i]放入B
29                     dp[i][j][w] = max(dp[i][j][w], dp[i - 1][j - 1][w + 2 * t[i]] + v[i]);
30             }
31         }
32     }
33
34     ll ans = -INFF;
35     for (int j = 0; j <= k; j++) ans = max(ans, dp[n][j][2600]);
36     cout << ans;
37 }

```

思路II

dp数组的第一维滚动.

代码II

```

1  const int MAXN = 105, MAXM = 2605;
2  int n, k; // 牌数、最多翻倍次数
3  int v[MAXN], t[MAXN]; // 价值、点数
4  // dp[i][j][w]表示只考虑前i个物品、还可翻倍j次、当前背包体积为w的价值之和最大值
5  ll dp[2][MAXN][MAXM]; // 第一维滚动
6  int T = 1300; // 偏移量
7  int idx; // 滚动数组下标
8
9  int main() {
10     memset(dp, -INFF, so(dp)); // 置为-INFF,表示未更新
11     dp[0][0][T] = 0; // 初始条件,都不选
12     idx = 1; // 滚动数组下标从1开始
13
14     cin >> n >> k;
15     for (int i = 1; i <= n; i++) cin >> v[i] >> t[i];

```



```

16
17     for (int i = 1; i <= n; i++) { // 枚举物品
18         memset(dp[idx], -INFF, so(dp[idx])); // 初始化将要更新的部分
19
20         for (int j = 0; j <= k; j++) { // 枚举翻倍次数
21             for (int w = -1300; w <= 1300; w++) { // 枚举背包体积
22                 for (int p = -2; p <= 2; p++) { // 枚举选择的物品,分别为-2*t[i],-t[i],0,t[i],2*t[i]
23                     int curw = w + p * t[i]; // 当前背包体积
24                     if (curw < -1300 || curw > 1300) continue; // 越界
25
26                     if (j) { // 可以翻倍
27                         dp[idx][j][curw + T] = max(dp[idx][j][curw + T], dp[idx ^ 1][j - (abs(p) == 2)][w + T] +
28 (p == 0 ? 0 : v[i]));
29                     }
30                     else if (abs(p) <= 1) // 只能转移不翻倍的
31                         dp[idx][j][curw + T] = max(dp[idx][j][curw + T], dp[idx ^ 1][j][w + T] + (p == 0 ? 0 :
32 v[i]));
33                 }
34             }
35             idx ^= 1;
36         }
37     }
38     ll ans = -INFF;
39     for (int i = 0; i <= k; i++) ans = max(ans, dp[n & 1][i][T]);
40     cout << ans;
41 }

```

10.1.28 Chevonne's Necklace

题意

n 颗珍珠串成一串项链,珍珠顺时针编号 $1 \sim n$.每颗珍珠上有一个非负整数 c_i ,每次操作可选择一颗满足 $c_i \geq 1$ 的珍珠 i ,移除从它顺时针开始数的 c_i 颗珍珠.若剩下的珍珠数不足 c_i ,则不能选择珍珠 i .操作后剩下的珍珠重新串成项链.求最多能移除多少颗珍珠及其方案数.用每次操作选择的珍珠的编号构成的集合来描述一个方案,两个方案不同当且仅当它们对应的集合不同.

第一行输入整数 n ($1 \leq n \leq 2000$).第二行输入 n 个整数 $c_1, \dots, c_n$ ($0 \leq c_i \leq 2000$).

输出两个整数,第一个表示最多能移除多少颗珍珠,第二个表示其方案数,答案对998244353取模.

思路

将珍珠视为物品,以 n 为背包容量、 c_i 为体积、0为价值做背包,则方案数即背包的方案数.

[析] 可疑点:删除一个选中的珍珠时顺带将下一步要选择的珍珠也移除了,导致无法构造出背包的解.

[证] 只需对背包选出的方案构造出原题的一组解.设背包选中的珍珠下标为 $p_1, \dots, p_k$,则 $\sum_{i=1}^k c_{p_i} \leq n$.

$dis(p_i)$ 表示珍珠 p_i 与顺时针方向的下一个选中的珍珠(即珍珠 $p_{(i \bmod n)+1}$)间的距离,

则只有 $c_{p_i} \leq dis(p_i)$ 时才能删去珍珠 p_i .

下证每次操作时至少 $\exists$ 一个珍珠 $s.t. c_{p_i} \leq dis(p_i)$.若不然,设所有珍珠都满足 $c_{p_i} > dis(p_i)$,

$$\text{则 } \sum_{i=1}^k c_{p_i} > \sum_{i=1}^k dis(p_i) = n, \text{ 矛盾.}$$

故每次操作时都选中一个满足 $c_{p_i} \leq dis(p_i)$ 的珍珠即可,而方案用集合描述,故移除顺序不影响方案数.

显然最终选中的珍珠是从珍珠1开始的连续编号.

代码

```

1  const int MAXN = 2005;
2  const int MOD = 998244353;
3  int n;
4  int c[MAXN];
5  bool removed[MAXN]; // 记录每颗珍珠是否被移除
6  int dp[MAXN]; // dp[i]表示移除i颗珍珠的方案数
7
8  void solve() {
9      cin >> n;
10     for (int i = 0; i < n; i++) cin >> c[i];
11
12     removed[0] = true;
13     dp[0] = 1;
14     for (int i = 0; i < n; i++) { // 枚举物品
15         if (c[i]) { // c[i]≥1才能选中
16             for (int j = n; j >= c[i]; j--) { // 枚举体积
17                 removed[j] |= removed[j - c[i]]; // 更新珍珠状态
18                 dp[j] = ((1ll)dp[j] + dp[j - c[i]]) % MOD;
19             }
20         }
21     }
22
23     int ans = n;
24     while (!removed[ans]) ans--;
25     cout << ans << ' ' << dp[ans];
26 }
27
28 int main() {
29     solve();
30 }

```

10.1.29 整数拆分

题意

给定一个整数 n ($1 \leq n \leq 1e6$), 将其拆分为2的幂次之和, 不考虑2的幂次间的顺序. 求方案数. 如① $4 = 1 + 1 + 1 + 1$, ② $4 = 1 + 1 + 2$, ③ $4 = 2 + 2$, ④ $4 = 4$.

思路

将2的幂次视为物品, 其体积为其数值大小, 转化为完全背包.

代码

```

1  const int MOD = 1e9;
2
3  void solve() {
4      int n; cin >> n;
5
6      vector<int> v;
7      for (int i = 1; i <= n; i *= 2)
8          v.push_back(i);
9
10     // dp[i][j]表示将j拆分为 $2^0, \dots, 2^i$ 之和的方案数
11     vector<int> dp(n + 1);
12     dp[0] = 1; // 初始条件
13     for (auto vi : v) {
14         for (int j = vi; j <= n; j++)

```

```

15         dp[j] = (dp[j] + dp[j - vi]) % MOD;
16     }
17     cout << dp[n] << endl;
18 }
19
20 int main() {
21     solve();
22 }

```

10.1.30 波动数列

原题指路: https://www.lanqiao.cn/problems/215/learning/?page=1&first_category_id=1&sort=students_count&second_category_id=3&name=%E6%B3%A2%E5%8A%A8

题意

给定四个整数 n, s, a, b ($1 \leq n \leq 1000, -1e9 \leq s \leq 1e9, 1 \leq a, b \leq 1e9$), 求长度为 n 、和为 s , 且后一项要么比前一项增加 a , 要么比前一项减少 b 的整数序列的个数, 答案对 $(1e8 + 7)$ 取模。

思路

设 $d_i \in \{+a, -b\}$ ($1 \leq i \leq n-1$), 序列首项为 x , 则序列形如 $x, x + d_{n-1}, x + d_{n-1} + d_{n-2}, \dots, x + d_{n-1} + \dots + d_1$, 进而 $nx + d_1 + 2 \cdot d_2 + \dots + (n-1)d_{n-1} = s$. 因 $x \in \mathbb{Z}$, 而 $d_i \in \{+a, -b\}$, 考虑用 d_i 表示 x , 即 $x = \frac{s - [d_1 + \dots + (n-1)d_{n-1}]}{n}$, 此时需满足分子是 n 的倍数。

注意到每个合法序列与 $d_1, \dots, d_{n-1}$ 的一组取值——对应, 问题转化为: 求有多少种 $d_1, \dots, d_{n-1}$ 的取法 $s.t. d_1 + \dots + (n-1)d_{n-1}$ 与 s 模 n 同余。

$dp[i][j]$ 表示 $s.t. (d_1 + \dots + i \cdot d_i) \bmod n = j$ 的 $(d_1, \dots, d_i)$ 的对数。初始条件 $dp[0][0] = 1$, 最终答案 $ans = dp[n-1][s \bmod n]$ 。

按最后一个 d_i 取 $+a$ 还是 $-b$ 分类, 状态转移方程 $dp[i][j] = dp[i-1][j-i \cdot a] + dp[i-1][j+i \cdot b]$ 。

代码

```

1  const int MOD = 1e8 + 7;
2
3  int get(int a, int b) { // 求a模b的正余数
4      return (a % b + b) % b;
5  }
6
7  void solve() {
8      int n, s, a, b; cin >> n >> s >> a >> b;
9
10     // dp[i][j]表示 s.t. (d[1] + ... i * d[i]) mod n = j的(d[1], ..., d[i])的对数
11     vector<vector<int>> dp(n, vector<int>(n));
12     dp[0][0] = 1;
13
14     for (int i = 1; i < n; i++) {
15         for (int j = 0; j < n; j++)
16             dp[i][j] =
17                 (dp[i-1][get(j-i*a, n)] + dp[i-1][get(j+i*b, n)]) % MOD;
18     }
19     cout << dp[n-1][get(s, n)] << endl;
20 }
21
22 int main() {
23     solve();
24 }

```

10.1.31 包子凑数

原题指路: https://www.lanqiao.cn/problems/98/learning/?page=1&first_category_id=1&sort=students_count&second_category_id=3&name=%E5%8C%85%E5%AD%90

题意

给定一个长度为 n ($1 \leq n \leq 100$)的整数序列 $a = [a_1, \dots, a_n]$ ($1 \leq a_i \leq 100$). 现从 a 中取任意数量的整数, 求线性组合得到一个整数. 问不能得到的整数的个数, 若有无穷个数不能得到, 输出INF.

思路

设 $d = \gcd(a_1, \dots, a_n)$, 有如下两种情况:

①若 $d > 1$, 则所有非 d 的倍数都无法被表示, 答案为INF.

②若 $d = 1$, 由Bezout定理: $\exists x_1, \dots, x_n$ s.t. $x_1 a_1 + \dots + x_n a_n = 1$.

等式两边同时扩大 m 倍, 则 $\exists x'_1, \dots, x'_n$ s.t. $x'_1 a_1 + \dots + x'_n a_n = m$.

若 $x'_1 < 0$, $x'_2 > 0$, 则可将 $x'_1 a_1 + x'_2 a_2$ 调整为 $(x'_1 + a_2) a_1 + (x'_2 - a_1) a_2$.

类似地可将 $a_1, \dots, a_n$ 前的系数都调整为正, 故 $d = 1$ 时有解.

考察不能被 $a_1, \dots, a_n$ 线性表示的最小数. 对 $x, y \in \mathbb{Z}^+$, 不能被 x, y 线性表示的最小数为 $(x-1)(y-1)-1$, 则直观上不能被 $a_1, \dots, a_n$ 线性表示的最小数很小, 经验值取 $100^2 = 1e4$, 问题转化为求 $1 \sim 1e4$ 中不能被 $a_1, \dots, a_n$ 线性表示的数的个数.

$dp[i][j]$ 表示 j 是否能被 $a_1, \dots, a_i$ 线性表示, 转化为完全背包.

代码

```

1  const int MAXA = 1e4;
2
3  int gcd(int a, int b) {
4      return b ? gcd(b, a % b) : a;
5  }
6
7  void solve() {
8      int n; cin >> n;
9      vector<int> a(n + 1);
10     int d = 0;
11     for (int i = 1; i <= n; i++) {
12         cin >> a[i];
13         d = gcd(d, a[i]);
14     }
15
16     if (d != 1) cout << "INF" << endl;
17     else {
18         // dp[i][j]表示j是否能被a[1], ..., a[i]线性表示
19         vector<int> dp(MAXA);
20         dp[0] = true;
21         for (int i = 1; i <= n; i++) {
22             for (int j = a[i]; j < MAXA; j++)
23                 dp[j] |= dp[j - a[i]];
24         }
25
26         int ans = 0;
27         for (int i = 0; i < MAXA; i++)
28             ans += !dp[i];
29         cout << ans << endl;
30     }
31 }

```

```

32
33 int main() {
34     solve();
35 }

```

10.1.32 砝码称重

原题指路: <https://www.lanqiao.cn/problems/1447/learning/>

题意

给定一个天平和 n ($1 \leq n \leq 100$) 个重量分别为 $w_1, \dots, w_n$ ($\sum_{i=1}^n w_i \leq 1e5$) 的砝码, 砝码可放在天平的两边. 问可称出多少不同的正整数(不含0)重量.

思路

对每个砝码, 不放该砝码对总重量的贡献为0, 放在一边对总重量的贡献为正, 放在另一边对总重量的贡献为负.

$dp[i][j]$ 表示只用前 i 个砝码能否凑出总重量 j . 设 $sum = \sum_{i=1}^n w_i$. 初始条件 $dp[0][0] = true$, 最终答案

$$ans = \sum_{j=1}^{sum} [dp[n][j] = true].$$

以砝码 w_i 的放法分类:

- ① 不放该砝码, 则 $dp[i][j] = dp[i-1][j]$.
- ② 该砝码放在一边, 则 $dp[i][j] = dp[i-1][j-w[i]]$ ($j-w[i] \geq -sum$).
- ③ 该砝码放在另一边, 则 $dp[i][j] = dp[i-1][j+w[i]]$ ($j+w[i] \leq sum$).

因 $(j-w[i])$ 可能为负, 则第二维整体加偏移量 $BASE = 1e5$ 即可.

总时间复杂度 $O\left(n \cdot 2 \sum_{i=1}^n w_i\right) = O\left(n \cdot \sum_{i=1}^n w_i\right)$.

代码

```

1  const int BASE = 1e5, MAXA = 2 * BASE + 5; // 注意空间开两倍
2
3  void solve() {
4      int n; cin >> n;
5      vector<int> w(n + 1);
6      int sum = 0;
7      for (int i = 1; i <= n; i++) {
8          cin >> w[i];
9          sum += w[i];
10     }
11
12     // dp[i][j] 表示只用前 i 个砝码能否凑出总重量 j
13     vector<vector<int>> dp(n + 1, vector<int>(MAXA));
14     dp[0][0 + BASE] = 1; // 初始条件
15     for (int i = 1; i <= n; i++) {
16         for (int j = -sum; j <= sum; j++) {
17             dp[i][j + BASE] = dp[i-1][j + BASE]; // 不放该砝码
18             if (j - w[i] >= -sum) // 该砝码放一边
19                 dp[i][j + BASE] = dp[i-1][j - w[i] + BASE];
20             if (j + w[i] <= sum) // 该砝码放另一边

```

```

21         dp[i][j + BASE] |= dp[i - 1][j + w[i] + BASE];
22     }
23 }
24
25 int ans = 0;
26 for (int j = 1; j <= sum; j++) ans += dp[n][j + BASE];
27 cout << ans << endl;
28 }
29
30 int main() {
31     solve();
32 }

```

10.1.33 Timetable

原题指路: <https://codeforces.com/problemset/problem/946/D>

题意 (2 s)

一周有 n ($1 \leq n \leq 500$) 天, 每天有 m ($1 \leq m \leq 500$) 小时上课, 用 n 个长度为 m 的 0/1 串描述一周的上课情况, 其中 0 表示该小时没课, 1 表示该小时有课. 某人会在每天的第一节课前恰到达教室, 在每天的最后一节课后恰离开教室. 现他可翘掉不超过 $skip$ ($0 \leq skip \leq 500$) 节课, 求他一周待在教室的总时间的最小值.

思路

$dp_1[i][j]$ 表示第 i 天翘掉 j 节课时该天待在教室的最小时间. 初始条件 $dp_1[i][j] = \begin{cases} 0, j = sizes_i \\ INF, 0 \leq j \leq sizes_i - 1 \end{cases}$ ($1 \leq i \leq n$), 其中 $sizes_i$ 为第 i 天的课数.

对第 i ($1 \leq i \leq n$) 天, 将 1 的下标记录在数组 $pos[]$ 中. 状态转移可用区间 DP, 先枚举第 i ($1 \leq i \leq n$) 天的保留的节数 $len \in [1, sizes_i]$, 再枚举保留的区间 $[l, r]$, 则状态转移方程 $dp_1[i][sizes_i - len] = \min_{\substack{[l,r] \subset [1,sizes_i] \\ r-l+1=len}} \{pos[r] - pos[l] + 1\}$.

求每个 $dp_1[i][j]$ 的时间复杂度为 $O(m^2)$, 则求 $dp_1[][]$ 的总时间复杂度为 $O(nm^2)$.

$dp_2[i][j]$ 表示在前 i 天至多翘掉 j 节课时一周待在教室的最小总时间. 初始条件 $dp_2[0][j] = 0$ ($0 \leq j \leq skip$), 最终答案 $ans = dp_2[n][skip]$.

状态转移类似于分组背包, 但此处每个分组恰选一个物品. 先枚举天数 $i \in [1, n]$, 后枚举总翘的节数 $j \in [0, skip]$, 再枚举第 i 天翘的节数 $k \in [0, \min\{j, sizes_i\}]$, 则状态转移方程 $dp_2[i][j] = \min_{0 \leq k \leq \min\{j, sizes_i\}} dp_2[i-1][j-k] + dp_1[i][k]$.

状态数 $O(n \cdot skip)$, 转移 $O(\min\{skip, m\})$, 则求 $dp_2[][]$ 的总时间复杂度为 $O(n \cdot skip \cdot \min\{skip, m\})$.

总时间复杂度 $O(n \cdot (m^2 + skip \cdot \min\{skip, m\}))$, 最坏约 $1.25e8$, 可通过.

代码

```

1 void solve() {
2     int n, m, skip; cin >> n >> m >> skip; // 天数、每天的小时数、最大翘课数
3
4     // dp1[i][j]表示第i天翘掉j节课时该天待在教室的最小时间
5     vector<vector<int>>> dp1(n + 1, vector<int>(m + 1, INF));
6     vector<int> sizes(n + 1); // 每一天的节数
7     for (int i = 1; i <= n; i++) {
8         vector<int> pos = {0}; // 0/1串中1的下标, 下标从1开始
9         for (int j = 1; j <= m; j++) {
10             char ch; cin >> ch;
11             if (ch == '1') pos.push_back(j);

```

```

12     }
13
14     sizes[i] = (int)pos.size() - 1; // 减去初始时的0
15     dp1[i][sizes[i]] = 0; // 初始条件：翘掉该天的所有课
16     for (int len = 1; len <= sizes[i]; len++) { // 枚举保留的课数
17         for (int l = 1; l + len - 1 <= sizes[i]; l++) { // 保留区间[l, r]的课
18             int r = l + len - 1;
19             dp1[i][sizes[i] - len] = min(dp1[i][sizes[i] - len], pos[r] - pos[l] + 1);
20         }
21     }
22 }
23
24 // dp2[i][j]表示在前i天至多翘掉j节课时一周待在教室的最小总时间
25 vector<vector<int>> dp2(n + 1, vector<int>(skip + 1, INF));
26 for (int j = 0; j <= skip; j++) dp2[0][j] = 0; // 初始条件
27
28 for (int i = 1; i <= n; i++) { // 枚举天数
29     for (int j = 0; j <= skip; j++) { // 枚举总翘的节数
30         for (int k = 0; k <= min(j, sizes[i]); k++) // 枚举该天翘的节数
31             dp2[i][j] = min(dp2[i][j], dp2[i - 1][j - k] + dp1[i][k]);
32     }
33 }
34 cout << dp2[n][skip] << endl;
35 }
36
37 int main() {
38     solve();
39 }

```

10.2 线性DP

10.2.2 最长上升子列

题意

给定一个长度为 n ($1 \leq n \leq 1000$)的整数序列 $a = [a_1, \dots, a_n]$ ($-1e9 \leq a_i \leq 1e9$), 求数值严格递增的子列的最长长度.

思路

$dp[i]$ 表示以 $a[i]$ 结尾的最长上升子列的长度.

第 i 个数必取, 则按第 $(i - 1)$ 个数是 $a[1], \dots, a[i - 1]$ 分类, 状态转移方程 $dp[i] = \max_{\substack{j=0,1,\dots,i-1 \\ a[j]<a[i]}} \{dp[j] + 1\}$.

总时间复杂度 $O(n^2)$.

代码

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6     // dp[i]表示以a[i]结尾的LIS的长度
7     vector<int> dp(n + 1);
8     for (int i = 1; i <= n; i++) {
9         dp[i] = 1; // 初始条件
10        for (int j = 1; j < i; j++) {
11            if (a[j] < a[i])
12                dp[i] = max(dp[i], dp[j] + 1);
13        }

```

```

14     }
15     cout << *max_element(all(dp)) << endl;
16 }
17
18 int main() {
19     solve();
20 }

```

题意II

给定一个长度为 n ($1 \leq n \leq 1e5$)的整数序列 $a = [a_1, \dots, a_n]$ ($-1e9 \leq a_i \leq 1e9$), 求数值严格递增的子列的最长长度.

思路II

考察序列3 1 2 1 8 5 6. 注意到若某数可接在3后面形成上升子列, 则也能接到1后面形成上升子列, 故可存前面最长上升子列的结尾数.

显然随 i 增大时, 以 $a[i]$ 结尾的最长上升子列的末尾数的最小值单调增, 欲求以 $a[i]$ 结尾的最长上升子列的长度, 需将 $a[i]$ 接到 $< a[i]$ 的最大数之后. $< a[i]$ 的最大数可二分, 每次查询时间复杂度 $O(\log n)$, 共 n 个数, 故总时间复杂度 $O(n \log n)$.

代码II

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6     vector<int> lis(n + 1);
7     lis[0] = -INF; // 小于a[]中的所有数
8     int maxlen = 0; // LIS的长度
9     for (int i = 1; i <= n; i++) {
10         int pos = lower_bound(lis.begin(), lis.begin() + maxlen + 1, a[i]) - lis.begin() - 1;
11         maxlen = max(maxlen, pos + 1);
12         lis[pos + 1] = a[i];
13     }
14     cout << maxlen << endl;
15 }
16
17 int main() {
18     solve();
19 }

```

10.2.3 最长公共子列

题意

给定两长度分别为 n ($1 \leq n \leq 1000$)和 m ($1 \leq m \leq 1000$)的字符串 A 和 B , 求它们最长公共子列(未必连续)的长度.

思路

极端情况, $A = B$ 时, A 与 B 的共 2^N 个公共子列, 暴搜最大 2^{1000} , 会TLE.

考虑DP. $dp[i][j]$ 表示 $A[1 \dots i]$ 和 $B[1 \dots j]$ 最长公共子列的长度. 按 $A[i]$ 和 $B[j]$ 各自是否在最长公共子列中分类: ①用00表示 $A[i]$ 不在, $B[j]$ 不在; ②用01表示 $A[i]$ 不在, $B[j]$ 在; ③用10表示 $A[i]$ 在, $B[j]$ 不在; ④用11表示 $A[i]$ 在, $B[j]$ 在.

①的情况即 $A[1 \dots (i-1)]$ 和 $B[1 \dots (j-1)]$ 最长公共子列的长度, 亦即 $dp[i-1][j-1]$.

④的情况仅当 $A[i] = B[j]$ 时存在, 此时最长公共子列为 $A[1 \dots (i-1)] + A[i]$ 和 $B[1 \dots (j-1)] + B[j]$, 其中 $A[1 \dots (i-1)]$ 和 $B[1 \dots (j-1)]$ 最长公共子列的长度即 $dp[i-1][j-1]$, 故④的情况的最大值为 $dp[i-1][j-1] + 1$.

②的情况不是 $dp[i-1][j]$, 因为 $dp[i-1][j]$ 未必包含 $B[j]$, 因此难找到一个公式. 注意到求最大值时不能遗漏, 但可以重复, 则②可用 $dp[i-1][j]$ 覆盖, 同理③可用 $dp[i][j-1]$ 覆盖, 虽它们有重复, 但不影响结果.

显然①的情况 $dp[i-1][j-1]$ 包含在 $dp[i-1][j]$ 和 $dp[i][j-1]$ 的并中, 故状态转移方程
 $dp[i][j] = \max\{dp[i-1][j], dp[i][j-1], dp[i-1][j-1] + 1 \mid (A[i] = B[j])\}$, 最终答案 $ans = dp[n][m]$.

代码

```
1 void solve() {
2     int n, m; string a, b; cin >> n >> m >> a >> b;
3     a = " " + a, b = " " + b;
4
5     // dp[i][j]表示a[1...i]与b[1...j]的LCS的长度
6     vector<vector<int>> dp(n + 1, vector<int>(m + 1));
7     for (int i = 1; i <= n; i++) {
8         for (int j = 1; j <= m; j++) {
9             dp[i][j] = max(dp[i-1][j], dp[i][j-1]); // a[i]在, b[j]不在; a[i]不在, b[j]在
10            if (a[i] == b[j])
11                dp[i][j] = max(dp[i][j], dp[i-1][j-1] + 1); // a[i]在, b[j]在
12        }
13    }
14    cout << dp[n][m] << endl;
15 }
16
17 int main() {
18     solve();
19 }
```

10.2.4 最短编辑距离

题意

给定两长度分别为 N ($1 \leq N \leq 1000$)和 M ($1 \leq M \leq 1000$)的只包含大写字母的字符串 A 和 B .现将 A 经若干操作变为 B ,可进行的操作:

- ①删除:删除 A 的某个字符;
- ②插入:在 A 的某个位置插入某字符;
- ③替换:将 A 的某个字符替换为另一字符.

问将 A 变为 B 至少需进行多少次操作.

思路

$dp[i][j]$ 表示将 $A[1 \cdots i]$ 变为 $B[1 \cdots j]$ 的最少操作次数.

考虑最后一步操作:

- ①删除 $A[i]$ 使得 $A[1 \cdots (i-1)]$ 和 $B[1 \cdots j]$ 匹配,则最小步数为 $dp[i-1][j] + 1$.

②插入 $A[i+1]$ 使得 $A[1 \cdots i] + A[i+1]$ 和 $B[1 \cdots j]$ 匹配,则加之前应 $A[1 \cdots i]$ 和 $B[1 \cdots (j-1)]$ 匹配,故最小步数为 $dp[i][j-1] + 1$.

- ③设 $A[1 \cdots (i-1)]$ 和 $B[1 \cdots (j-1)]$ 匹配.

1)若 $A[i] = B[j]$,则 $A[1 \cdots i]$ 和 $B[1 \cdots j]$ 已匹配,故最小步数为 $dp[i-1][j-1]$.

2)若 $A[i] \neq B[j]$,则做一次替换操作将 $A[i]$ 改为 $B[j]$,故最小步数为 $dp[i-1][j-1] + 1$.

状态转移方程 $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + 1/0)$. n^2 个状态,每个状态算3次,总复杂度 $O(n^2)$.

初始状态:

①只用 A 的前0个字母,则只能用添加操作,故 $dp[0][i] = i$ ($i = 0, 1, \dots, M$).

②匹配 B 的前0个字母,则只能用删除操作,故 $dp[i][0] = i$ ($i = 0, 1, \dots, N$).

代码

```
1  const int MAXN = 1005;
2  char s1[MAXN], s2[MAXN];
3  int dp[MAXN][MAXN];
4
5  int main() {
6      for (int i = 0; i <= M; i++) dp[0][i] = i;
7      for (int i = 0; i <= N; i++) dp[i][0] = i;
8
9      for (int i = 1; i <= N; i++) {
10         for (int j = 1; j <= M; j++) {
11             dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1);
12
13             if (s1[i] == s2[j]) dp[i][j] = min(dp[i][j], dp[i - 1][j - 1]);
14             else dp[i][j] = min(dp[i][j], dp[i - 1][j - 1] + 1);
15         }
16     }
17     cout << dp[N][M];
18 }
```

10.2.5 编辑距离 (2 s)

题意

给定 n ($1 \leq n \leq 1000$)个长度不超过10、仅含小写字母的字符串和 m ($1 \leq m \leq 1000$)次询问,每次询问给出一个字符串和一个操作次数上限.对每个询问,求给定的 n 个字符串中有几个能在上限操作次数内经若干次操作变为询问给出的字符串.

单个字符的插入、删除或替换为一次操作.

思路

最坏的情况:单个字母编辑距离 $O(n^2)$,则共需 $1000 \times 1000 \times 10^2 = 1e8$,时间限制2 s可过.

代码

```
1  const int MAXN = 15, MAXM = 1005;
2  char str[MAXN][MAXN];
3  int dp[MAXN][MAXN];
4
5  int edit_distance(char a[], char b[]) {
6      int la = strlen(a + 1), lb = strlen(b + 1);
7
8      for (int i = 0; i <= lb; i++) dp[0][i] = i;
9      for (int i = 0; i <= la; i++) dp[i][0] = i;
10
11     for (int i = 1; i <= la; i++) {
12         for (int j = 1; j <= lb; j++) {
13             dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1);
14             dp[i][j] = min(dp[i][j], dp[i - 1][j - 1] + (a[i] != b[j]));
15         }
16     }
17
18     return dp[la][lb];
19 }
20
21 int main() {
```

```

22     for (int i = 0; i < n; i++) cin >> str[i] + 1;
23
24     while (m--) {
25         char s[MAXN];
26         int limit;
27         cin >> s + 1 >> limit;
28
29         int ans = 0;
30         for (int i = 0; i < n; i++)
31             if (edit_distance(str[i], s) <= limit) ans++;
32
33         cout << ans << '\n';
34     }
35 }

```

10.2.6 Zero Path

原题指路: <https://codeforces.com/contest/1695/problem/C>

题意

有 t ($1 \leq t \leq 1e4$) 组测试数据. 每组测试数据给定一个 $n \times m$ ($1 \leq n, m \leq 1000$) 的网格, 每个方格内填有 1 或 -1, 网格下标从 1 开始. 初始时你在 $(1, 1)$ 处, 目标是到达 (n, m) 处, 每一步能向下或向右移动到相邻的方格. 是否存在一条路径使得经过的方格中的数之和 (称路径和) 为 0, 若存在输出 "YES", 否则输出 "NO". 数据保证所有测试样例的 $n \times m$ 之和不超过 $1e6$.

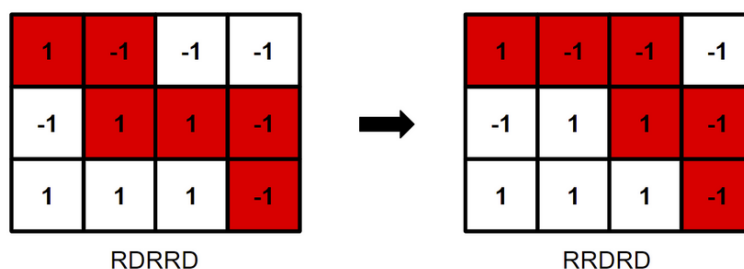
思路

若 $(n + m)$ 是偶数, 因从 $(1, 1)$ 到 (n, m) 需经过 $(n + m - 1)$ 个方格, 则路径和为奇数, 此时无解.

若 $(n + m)$ 是奇数, 则路径和为偶数. $dp1[i][j]$ 表示从 $(1, 1)$ 到 (i, j) 的路径和的最大值, $dp2[i][j]$ 表示从 $(1, 1)$ 到 (i, j) 的路径和的最小值, 则有解的充要条件是 $dp2[n][m] \leq 0 \leq dp1[n][m]$. 显然

$$dp1[i][j] = a[i][j] + \max\{dp1[i-1][j], dp1[i][j-1]\}, dp2[i][j] = a[i][j] + \min\{dp2[i-1][j], dp2[i][j-1]\}.$$

下证 $dp2[n][m] \leq 0 \leq dp1[n][m]$ 时能构造一条从 $(1, 1)$ 到 (i, j) 的路径和为 0 的路径: 设 p_1 是路径和为 $dp2[n][m]$ 的路径, p_2 是路径和为 $dp1[n][m]$ 的路径. 注意到每条路径都可表示为包含 $(m - 1)$ 个 R (向右) 和 $(n - 1)$ 个 D (向下) 的字符串, 则可通过若干次交换字符串中的相邻字符将 p_1 变为 p_2 , 几何上它将路径上的方格替换为其右上角或左下角的方格:



注意到每次交换会让路径和 -2 或不变或 +2, 则若干次操作后可将 p_1 变为 p_2 , 途中会经过路径和为 0 的路径.

代码

```

1 int main() {
2     CaseT{
3         int n, m; cin >> n >> m;
4         vector<vi> a(n, vi(m));
5         for (int i = 0; i < n; i++)
6             for (int j = 0; j < m; j++) cin >> a[i][j];
7
8         if ((n + m) % 2 == 0) {

```

```

9      cout << "NO" << endl;
10     continue;
11 }
12
13 vector<vi> dp1(n, vi(m, 0)); // dp1[i][j]表示从(1,1)到(i,j)的路径和的最大值
14 vector<vi> dp2(n, vi(m, 0)); // dp2[i][j]表示从(1,1)到(i,j)的路径和的最小值
15
16 // 初始化
17 dp1[0][0] = dp2[0][0] = a[0][0]; // 起点
18 for (int i = 1; i < n; i++) // 初始化第0列
19     dp1[i][0] = dp2[i][0] = dp1[i - 1][0] + a[i][0];
20 for (int j = 1; j < m; j++) // 初始化第0行
21     dp1[0][j] = dp2[0][j] = dp1[0][j - 1] + a[0][j];
22
23 // 转移
24 for (int i = 1; i < n; i++) {
25     for (int j = 1; j < m; j++) {
26         dp1[i][j] = max(dp1[i - 1][j], dp1[i][j - 1]) + a[i][j];
27         dp2[i][j] = min(dp2[i - 1][j], dp2[i][j - 1]) + a[i][j];
28     }
29 }
30
31 if (dp2[n - 1][m - 1] <= 0 && dp1[n - 1][m - 1] >= 0) cout << "YES" << endl;
32 else cout << "NO" << endl;
33 }
34 }

```

10.2.7 do NOT a=2b

题意

称一个序列是坏的,如果其中存在两个元素 a, b s.t. $a = 2b$.给定一个长度为 n ($1 \leq n \leq 1e5$)的序列 $a_1, \dots, a_n$ ($1 \leq a_i \leq 1e6$),删除尽量少的元素使得它不是坏的,输出最少要删除的元素的个数.

思路

$dp[a][0]$ 表示不删除值为 a 的元素, $dp[a][1]$ 表示删除所有值为 a 的元素.设值为 a 的元素个数为 $cnt[a]$.

注意到 a 只能是偶数,状态转移方程:

$$\begin{cases} a \text{ 为奇数} & \begin{cases} dp[a][0] = 0, \text{ 因为 } \frac{a}{2} \text{ 不是一个整数} \\ dp[a][1] = cnt[a], \text{ 即要删除所有值为 } a \text{ 的元素} \end{cases} \\ a \text{ 为偶数} & \begin{cases} dp[a][0] = dp[b][1], \text{ 即不删 } a \text{ 就要删除所有值为 } b = \frac{a}{2} \text{ 的元素} \\ dp[a][1] = \min\{dp[b][0], dp[b][1]\} + cnt[a], \text{ 删除 } a \text{ 后 } b \text{ 可删可不删} \end{cases} \end{cases}$$

注意到每个 $dp[a]$ 都包含了 $dp[b]$ 的最优解,故只需对所有 $2a > 1e6$ 的 a 统计答案即可.

代码

```

1  const int MAXN = 1e6 + 5;
2  int n;
3  map<int, int> cnt; // cnt[b]表示值为b的元素个数
4  int dp[MAXN][2]; // dp[a][0]表示不删除值为b的元素,dp[a][1]表示删除所有值为b的元素
5
6  void solve() {
7      cin >> n;
8      while (n--) {
9          int a; cin >> a;
10         cnt[a]++;
11     }
12
13     for (int a = 1; a <= 1e6; a++) {

```

```

14     if (a & 1) {
15         dp[a][0] = 0; // b/2不是整数
16         dp[a][1] = cnt[a]; // 删除所有值为b的元素
17     }
18     else {
19         dp[a][0] = dp[a / 2][1]; // 不删a就要删除所有值为b=a/2的元素
20         dp[a][1] = min(dp[a / 2][0], dp[a / 2][1]) + cnt[a]; // 删除a后b可删可不删
21     }
22 }
23
24 int ans = 0;
25 for (int a = 5e5 + 1; a <= 1e6; a++) // 对所有2a>1e6的a统计答案
26     ans += min(dp[a][0], dp[a][1]);
27 cout << ans;
28 }
29
30 int main() {
31     solve();
32 }

```

10.2.8 小 A 的卡牌游戏

原题指路:<https://codeforces.com/gym/103186/problem/B>

题意 (2 s)

一副 n 张卡的卡组恰包含 a 张A卡、 b 张B卡、 c 张C卡.现给出 n 次三选一的机会,三张卡分别来自三个种类,玩家需从三张卡中选一张加入自己的卡组,使得卡组强度尽量大.每张卡有一个强度值,卡组的强度是所有卡的强度之和.求卡组强度的最大值.

第一行输入四个整数 n, a, b, c ($1 \leq a, b, c \leq n \leq 5000, a + b + c = n$).接下来 n 行每行输入三个整数描述一个三选一的机会,其中第 i 行输入三个整数 a_i, b_i, c_i ($1 \leq a_i, b_i, c_i \leq 1e9$),分别表示该次选择中A卡、B卡、C卡的强度.

思路

先考虑只有A卡和B卡的情况.注意不能贪心地选择强度前 a 大的A卡和强度前 b 大的B卡,因为选择间不独立.考虑先确定B卡的选择,剩下的选A卡.对两次三选一的机会 (a_i, b_i) 和 (a_j, b_j) ,若选 b_i ,则只能再选 a_j ,同理选 b_j 只能再选 a_i ,则选前者更优的充要条件是: $b_i + a_j > b_j + a_i$,即 $b_i - a_i > b_j - a_j$.故将三选一的机会按 $b_i - a_i$ 降序排列后贪心地选前几个即可.

考虑有A、B、C卡的情况. $dp[i][j]$ 表示表示前 i 次选择中有 j 次选择C卡的最大强度,其中的 $(i - j)$ 次选A卡或B卡按照上述贪心策略选即可.总时间复杂度 $O(n^2)$.

代码

```

1  const int MAXN = 5005;
2  int n, A, B, C;
3  ll dp[MAXN][MAXN]; // dp[i][j]表示前i次选择中有j次选择C卡的最大强度
4
5  struct Card {
6      int a, b, c;
7
8      bool operator<(const Card& B) const {
9          if (b - a != B.b - B.a) return b - a > B.b - B.a;
10         else return c > B.c;
11     }
12 } cards[MAXN];
13
14 void solve() {
15     cin >> n >> A >> B >> C;
16
17     for (int i = 1; i <= n; i++) cin >> cards[i].a >> cards[i].b >> cards[i].c;
18 }

```

```

19  sort(cards + 1, cards + n + 1);
20
21  for (int i = 0; i <= n; i++)
22      for (int j = i + 1; j <= C; j++) dp[i][j] = -INFF; // 初始化非法状态
23
24  for (int i = 1; i <= n; i++) {
25      for (int j = 0; j <= min(i, C); j++) { // 枚举选C卡的次数
26          if (j) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + cards[i].c); // 选C卡
27
28          if (i - j <= B) dp[i][j] = max(dp[i][j], dp[i - 1][j] + cards[i].b); // 选B卡
29          else dp[i][j] = max(dp[i][j], dp[i - 1][j] + cards[i].a); // 选A卡
30      }
31  }
32  cout << dp[n][C];
33 }
34
35 int main() {
36     solve();
37 }

```

10.2.9 Mio visits ACGN Exhibition

原题指路:<https://codeforces.com/gym/103366/problem/A>

题意 (2 s, 256 MB)

给定一个 $n \times m$ 的 $0-1$ 矩阵, 从左上角 $(1, 1)$ 出发, 到达右下角 (n, m) , 每次只能向右或向下移动到相邻的各自. 问有几条不同的路径使得路径至少经过 p 个 0 和 q 个 1 , 答案对 998244353 取模.

第一行输入四个整数 n, m, p, q ($1 \leq n, m \leq 500, 0 \leq p, q \leq 1e4$). 接下来输入一个 $n \times m$ 的 $0-1$ 矩阵.

思路

$dp[i][j][k]$ 表示从 $(1, 1)$ 到 (i, j) 恰经过 k 个 0 的路径数, 最终答案为 $\sum_{k=p}^{n+m-1-q} dp[n][m][k]$. 初始条件 $dp[1][1][!a[1][1]] = 1$, 注意不是 $dp[1][1][a[1][1]] = 1$.

一个 `int` 类型的该数组所需空间 $\frac{4 \times 500 \times 500 \times 1e4}{1024 \times 1024} \approx 1e4 \text{ MB} > 256 \text{ MB}$, dp 数组的第一维滚动即可.

代码

```

1  const int MAXN = 505, MAXM = 1e4 + 5;
2  const int MOD = 998244353;
3  int n, m, p, q;
4  int a[MAXN][MAXN];
5  int dp[2][MAXN][MAXM]; // dp[i][j][k] 表示从(1,1)到(i,j)恰经过k个0的路径数
6
7  void solve() {
8      cin >> n >> m >> p >> q;
9      for (int i = 1; i <= n; i++)
10         for (int j = 1; j <= m; j++) cin >> a[i][j];
11
12     dp[1][1][!a[1][1]] = 1; // 初始条件
13     for (int i = 1; i <= n; i++) {
14         for (int j = 1; j <= m; j++) {
15             if (i == 1 && j == 1) continue;
16
17             for (int k = 0; k <= i + j - 1; k++) { // 枚举0的个数

```

```

18     if (a[i][j]) dp[i & 1][j][k] = ((1ll)dp[i & 1][j - 1][k] + dp[i - 1 & 1][j][k]) % MOD;
19     else {
20         dp[i & 1][j][k + 1] = ((1ll)dp[i & 1][j - 1][k] + dp[i - 1 & 1][j][k]) % MOD;
21         dp[i & 1][j][0] = 0; // 清空非法状态
22     }
23 }
24 }
25 }
26
27 int ans = 0;
28 for (int k = p; k <= n + m - 1 - q; k++) ans = ((1ll)ans + dp[n & 1][m][k]) % MOD;
29 cout << ans;
30 }
31
32 int main() {
33     solve();
34 }

```

10.2.10 Count Binary Strings

原题指路:<https://codeforces.com/contest/1767/problem/C>

题意 (2 s)

给定一个整数 n ($1 \leq n \leq 100$), 考察满足如下限制的、长度为 n 的 0/1 串 s (下标从 1 开始) 的个数, 答案对 998244353 取模. 限制有 n 行, 其中第 i ($1 \leq i \leq n$) 行包含 $(n - i + 1)$ 个整数 $a_{i,1}, \dots, a_{i,n}$ ($0 \leq a_{i,j} \leq 2$). 其中, ① $a_{i,j} = 1$ 表示子串 $s[i \dots j]$ 只包含一种字符; ② $a_{i,j} = 2$ 表示子串 $s[i \dots j]$ 包含两种字符; ③ $a_{i,j} = 0$ 表示子串 $s[i \dots j]$ 无限制.

思路

显然只需考虑 $a_{i,j} \neq 0$ 的限制.

注意到对固定的子串右端点 r , 限制只能是 $a[1 \dots (l - 1)][r] = 1, a[l \dots r][r] = 2$ 的形式, $dp[l][r]$ 表示限制

$a[1 \dots (l - 1)][r] = 2, a[l \dots r][r] = 1$ 时前缀 $s[1 \dots r]$ 的方案数, 则 $ans = \sum_{i=1}^n dp[i][n]$. 以 s_{r+1} 是否与 s_r 相同分类: ① 若 $s_{r+1} = s_r$,

状态转移方程 $dp[l][r + 1] += dp[l][r]$; ② 若 $s_{r+1} \neq s_r$, 状态转移方程 $dp[r + 1][r + 1] += dp[l][r]$. 上述状态转移都需在合法的前提下, 注意清空非法状态. 初始条件 $dp[1][1] = 2$, 即首字符可为 0 或 1, 因为 $a[] []$ 只限制了元素的种类数.

代码

```

1 void solve() {
2     int n; cin >> n;
3     vector<vector<int>>> a(n + 1, vector<int>(n + 1));
4     for (int i = 1; i <= n; i++)
5         for (int j = i; j <= n; j++) cin >> a[i][j];
6
7     // dp[l][r] 表示限制 a[1... (l-1)][r]=2, a[l... r][r]=1 时前缀 s[1... r] 的方案数
8     vector<vector<Z>>> dp(n + 5, vector<Z>(n + 5));
9     dp[1][1] = 2; // 初始条件: 首字符可为 0 或 1
10    for (int r = 1; r <= n; r++) {
11        for (int l = 1; l <= r; l++) {
12            bool ok = true; // 记录是否存在合法的转移
13            for (int i = 1; i <= l - 1; i++) ok &= a[i][r] != 1;
14            for (int i = l; i <= r; i++) ok &= a[i][r] != 2;
15
16            if (!ok) dp[l][r] = 0; // 非法状态
17            dp[l][r + 1] += dp[l][r]; // s[r+1]=s[r]
18            dp[r + 1][r + 1] += dp[l][r]; // s[r+1]≠s[r]
19        }
20    }

```

```

21
22     z ans = 0;
23     for (int i = 1; i <= n; i++) ans += dp[i][n];
24     cout << ans << endl;
25 }
26
27 int main() {
28     solve();
29 }

```

思路II

从前往后考察字符串的每个字符.以 $s_i = 0$ 为例,若 s_i 之前有字符1,则前缀 $s[1 \cdots i]$ 中包含两种字符,具体地,子串 $s[j \cdots i]$ 包含两种字符当且仅当区间 $[j, i - 1]$ 中至少存在一个字符1.同理可得 $s_i = 1$ 时的情况.

$dp[i][j]$ 表示使得最后一个与 s_i 相异的字符是 s_j 时前缀 $s[1 \cdots i]$ 的方案数,特别的, $j = 0$ 时表示前缀 $s[1 \cdots i]$ 中的字符都为 s_i .按 s_{i+1} 与 s_i 是否相同分类,状态转移方程:① $dp[i + 1][j] += dp[i][j]$;② $dp[i + 1][i] += dp[i][j]$.初始条件:若 $a[1][1] \neq 2$,则 $dp[2][1] = 2$;否则 $dp[2][1] = 0$.

代码II

```

1 void solve() {
2     int n; cin >> n;
3     vector<vector<int>>> a(n + 1, vector<int>(n + 1));
4     for (int i = 0; i < n; i++)
5         for (int j = i; j < n; j++) cin >> a[i][j];
6
7     auto check = [&](int cur, int last) -> bool { // 当前要填的下标、上一个与要填的字符相异的字符的下标
8         for (int i = 0; i < cur; i++) {
9             if (a[i][cur - 1] == 1 && last > i) return false;
10            if (a[i][cur - 1] == 2 && last <= i) return false;
11        }
12        return true;
13    };
14
15    // dp[i][j]表示使得最后一个与s[i]相异的字符是s[j]时前缀s[1...i]的方案数
16    vector<vector<Z>>> dp(n + 1, vector<Z>(n + 1));
17    if (a[0][0] != 2) dp[1][0] = 2; // 初始条件
18    for (int i = 1; i < n; i++) { // 枚举当前要填的字符的上位
19        for (int j = 0; j < i; j++) { // 枚举与要填的字符相异的字符的下标
20            for (auto k : vector<int>({ j, i }))
21                if (check(i + 1, k)) dp[i + 1][k] += dp[i][j];
22        }
23    }
24
25    z ans = 0;
26    for (int i = 0; i < n; i++) ans += dp[n][i];
27    cout << ans << endl;
28 }
29
30 int main() {
31     solve();
32 }

```


10.2.11 Different Arrays

原题指路:<https://codeforces.com/contest/1783/problem/D>

题意 (2 s)

给定一个长度为 n ($3 \leq n \leq 300$)的序列 $a_1, \dots, a_n$ ($0 \leq a_i \leq 300$).现需做恰 $(n - 2)$ 次操作:第1次操作,要么令 $a_1 + = a_2$ 和 $a_3 - = a_2$,要么令 $a_1 - = a_2$ 和 $a_3 + = a_2$;...第 i 次操作令 a_i 的其中一个相邻的元素 $+ = a_i$,另一个相邻的元素 $- = a_i$;...第 $(n - 2)$ 次操作,令要么令 $a_{n-2} + = a_{n-1}$ 和 $a_n - = a_{n-1}$,要么令 $a_{n-2} - = a_{n-1}$ 和 $a_n + = a_{n-1}$.称一个序列是可达的,如果能从初始序列出发,经上述的恰 $(n - 2)$ 次操作后得到.求可达的序列的个数,答案对998244353取模.

思路

注意到 $a[]$ 的值域很小, $dp[i][j]$ 表示长度为 i 、最后一个元素为 j 的序列的方案数.以第 $(i - 1)$ 个操作时 a_i 左加右减还是左减右加分类,以令 $a_{i-1} - = a_i, a_{i+1} + = a_i$ 为例,设上一个序列为 $a[]$,进行完该变换后得到的序列为 $a'[]$,则 $a'_{i-1} = a'_i - a_i$,即可推得当前序列的 a'_{i-1} ,同理可继续推出 $a'_{i-2}, \dots$,故状态转移方程 $dp[i+1][a[i+1] - j] + = dp[i][j], dp[i+1][a[i+1] + j] + = dp[i][j]$,初始条件 $dp[2][a[2]] = 1$,最终答案 $ans = \sum_{j=-300 \times 300}^{300 \times 300} dp[n][j]$.注意 $j = 0$ 时只有一种转移.

注意状态转移的过程中第二维可能为负,整体加一个偏移量即可.元素的最小值为 -300×300 .

代码

```
1  template<typename T, int OFFSET>
2  struct OffsetArray {
3      T arr[2 * OFFSET];
4
5      OffsetArray() {
6          memset(arr, 0, sizeof(arr));
7      }
8
9      T& operator[](int idx) {
10         return arr[idx + OFFSET];
11     }
12 };
13
14 const int MAXN = 305, MAXA = 300 * 300;
15 OffsetArray<Z, MAXN * MAXN> dp[MAXN]; // dp[i][j]表示长度为i、最后一个元素为j的序列的方案数
16
17 void solve() {
18     int n; cin >> n;
19     vector<int> a(n + 1);
20     for (int i = 1; i <= n; i++) cin >> a[i];
21
22     dp[2][a[2]] = 1; // 初始条件
23     for (int i = 2; i <= n - 1; i++) {
24         for (int j = -MAXA; j <= MAXA; j++) {
25             dp[i + 1][a[i + 1] - j] += dp[i][j];
26             if (j) dp[i + 1][a[i + 1] + j] += dp[i][j];
27         }
28     }
29
30     Z ans = 0;
31     for (int i = -MAXA; i <= MAXA; i++) ans += dp[n][i];
32     cout << ans << endl;
33 }
34
35 int main() {
36     solve();
37 }
```

思路II

类似于背包,压掉第一维.

代码II

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6     int OFFSET = accumulate(a.begin() + 1, a.end(), 0);
7     vector<Z> dp(2 * OFFSET + 1);
8     dp[a[2] + OFFSET] = 1; // 初始条件
9     for (int i = 3; i <= n; i++) {
10        vector<Z> pd(2 * OFFSET + 1);
11        for (int j = 0; j <= 2 * OFFSET; j++) {
12            if (!dp[j].val()) continue; // 非法状态,转移可能因下标为负而越界
13
14            pd[a[i] - (j - OFFSET) + OFFSET] += dp[j];
15            if (j != OFFSET) pd[a[i] + (j - OFFSET) + OFFSET] += dp[j];
16        }
17        dp = pd;
18    }
19
20    Z ans = accumulate(all(dp), Z(0));
21    cout << ans << endl;
22 }
23
24 int main() {
25     solve();
26 }

```

10.2.12 Pokémon Army (easy version)

原题指路:<https://codeforces.com/problemset/problem/1420/C1>

题意 (2 s)

给定一个长度为 n 的序列 $a_1, \dots, a_n$,对其每个子列 $a_{b_1}, \dots, a_{b_k}$ ($1 \leq b_1 < \dots < b_k \leq n$),求交错和 $a_{b_1} - a_{b_2} + a_{b_3} - \dots$ 的最大值.

有 t ($1 \leq t \leq 1000$)组测试数据.每组测试数据第一行输入两个整数 n, q ($1 \leq n \leq 3e5, q = 0$).第二行输入 n 个相异的整数 $a_1, \dots, a_n$ ($1 \leq a_i \leq n$).

思路I

$dp[i][0/1]$ 表示只考虑前 i 个元素时:①若选 a_i ,则它为负/正时交错和的最大值;②若不选 a_i ,则交错和中的最后一项为负/正时交错和的最大值,则 $ans = \max\{dp[n][0], dp[n][1]\}$.按 a_i 是否选择分类,状态转移方程:

$$\begin{cases} dp[i][0] = \max\{dp[i-1][0], dp[i-1][1] - a_i\} \\ dp[i][1] = \max\{dp[i-1][1], dp[i-1][0] + a_i\} \end{cases} \text{初始条件 } dp[1][0] = dp[1][1] = \max\{a_1, 0\} = a_1.$$

代码I

```

1 void solve() {
2     int n, q; cin >> n >> q;
3
4     // dp[i][0/1]表示只考虑前i个元素时:
5     // 若选a[i],则它为负/正时交错和的最大值
6     // 若不选a[i],则交错和中的最后一项为负/正时交错和的最大值

```

```

7   vector<vector<ll>> dp(n + 1, vector<ll>(2));
8   for (int i = 1; i <= n; i++) {
9       int a; cin >> a;
10
11       if (i == 1) dp[1][0] = dp[1][1] = a; // 初始条件
12
13       dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - a);
14       dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] + a);
15   }
16   cout << max(dp[n][0], dp[n][1]) << endl;
17 }
18
19 int main() {
20     CaseT
21     solve();
22 }

```

思路II

注意到思路I中 $dp[i][\]$ 只与 $dp[i - 1][\]$ 有关,则第一维可滚动.注意滚动后无需 $i = 1$ 时的初始条件.

代码II

```

1   void solve() {
2       int n, q; cin >> n >> q;
3
4       // dp[i][0/1]表示只考虑前i个元素时:
5       // 若选a[i],则它为负/正时交错和的最大值
6       // 若不选a[i],则交错和中的最后一项为负/正时交错和的最大值
7       vector<ll> dp(2);
8       for (int i = 1; i <= n; i++) {
9           int a; cin >> a;
10
11           dp[0] = max(dp[0], dp[1] - a);
12           dp[1] = max(dp[1], dp[0] + a);
13       }
14       cout << max(dp[0], dp[1]) << endl;
15   }
16
17 int main() {
18     CaseT
19     solve();
20 }

```

10.2.13 Ice and Fire

原题指路:<https://codeforces.com/contest/1774/problem/C>

题意

有编号 $1 \sim n$ 的 n 个玩家,其中 i ($1 \leq i \leq n$)号玩家的能力值为 i .现有 $(n - 1)$ 场比赛,依次在 $(n - 1)$ 个环境进行,环境用一个长度为 $(n - 1)$ 的0/1串 s 描述,其中0表示在该环境中能力值低者胜,1表示在该环境中能力值高者胜.现前 x ($1 \leq x \leq n$)个玩家用前 $(x - 1)$ 个环境进行 $(x - 1)$ 场比赛,若当前剩余玩家数 > 1 ,则任选两个剩余的球员比赛,胜者留下,负者淘汰,最后留下来的玩家胜.对每个 $x \in [2, n]$,求前 x 个玩家用前 $(x - 1)$ 个环境进行 $(x - 1)$ 场比赛时有多少玩家有机会获胜.

有 t ($1 \leq t \leq 1000$)组测试数据.每组测试数据第一行输入一个整数 n ($2 \leq n \leq 2e5$).第二行输入一个长度为 $(n - 1)$ 的0/1串 s .数据保证所有测试数据的 n 之和不超过 $3e5$.

思路I

设 s 的下标从1开始, cnt_i 表示以 s_i 结尾的字符都相同的最长后缀的长度.

对前 x ($2 \leq x \leq n$)个玩家,有 cnt_{x-1} 个玩家必败,其余玩家都有机会胜,故前 i ($1 \leq i \leq n-1$)场比赛有 $ans_i = i - cnt_i + 1$ 个玩家有机会胜.

[证] 考察前 n 个玩家进行 $(n-1)$ 场比赛时的情况.

WLOG,设 $s_{n-1} = 1$,考察最长连续1后缀的长度为 x 时的情况,此时 $s_{n-x-1} = 0$.

(1)若 $x = n-1$,则只有 n 号玩家能胜.

(2)若 $x < n-1$,

① $1 \sim x$ 号玩家无机会胜.

若不然,若 i ($1 \leq i \leq x$)号玩家胜,则他在最后 x 场比赛中打败能力值比他低的 x 个玩家.

而前 x 个玩家中比 i 号玩家能力低的只有 $(i-1)$ 个,矛盾.

② $(x+1) \sim n$ 号玩家有机会胜.

i ($x+1 \leq i \leq n$)号玩家获胜的策略:

i)前 $(n-x-2)$ 场,令能力值在 $[x+1, n]$ 范围内的除 i 号玩家外的其他玩家比赛,最终剩下一个玩家.

ii)第 $(n-x-1)$ 场,令i)中剩下的玩家与1号玩家比赛,因 $s_{n-x-1} = 0$,故1号玩家胜.

此时只剩下前 x 个玩家与 i 号玩家进行最后 x 场比赛,而 $s_{n-x+1} = \dots = s_n = 1$,故 i 号玩家胜.

代码I

```
1 void solve() {
2     int n; string s; cin >> n >> s;
3     s = " " + s;
4
5     int zero = 0, one = 0; // 后缀连续0、1的个数
6     for (int i = 1; i <= n - 1; i++) {
7         if (s[i] == '0') one = 0, zero++;
8         else zero = 0, one++;
9
10        cout << i - max(zero, one) + 1 << " \n"[i == n - 1];
11    }
12 }
13
14 int main() {
15     caseT
16     solve();
17 }
```

思路II

同思路I,注意到 $ans_i = i - cnt_i + 1$ 中 $i - cnt_i$ 即前缀 $s[1 \dots i]$ 的长度减去后缀连续相同字符的长度,则它等于最后一个与 $s[i]$ 相异的字符的下标.

代码II

```
1 void solve() {
2     int n; string s; cin >> n >> s;
3     s = " " + s;
4
5     vector<int> last(2); // last[0/1]表示最后一个0、1的下标
```

```

6   for (int i = 1; i <= n - 1; i++) {
7       last[s[i] - '0'] = i;
8       cout << (last[s[i] - '0' ^ 1] + 1) << " \n"[i == n - 1];
9   }
10  }
11
12  int main() {
13      CaseT
14      solve();
15  }

```

思路III

$dp[i]$ 表示前 i 场比赛有机会获胜的玩家数.以 s_{i-1} 与 s_i 是否相同分类:

(1) $s_{i-1} = 0, s_i = 0$ 时,前 $(i-1)$ 场比赛的胜者继续胜第 i 场比赛,故获胜玩家数不变.

(2) $s_{i-1} = 1, s_i = 1$ 时,获胜玩家数不变.

[证] 设前 $(i-1)$ 场比赛可能的胜者的集合为 A (包含的下标在范围 $[1, i]$ 中),前 i 场比赛可能的胜者的集合为 B (包含的下标在范围 $[1, i+1]$ 中),下证 $\max |A| = \max |B|$.

设只考虑前 $(i-1)$ 场比赛且 $|A|$ 取得最大值 k 时, $A = \{a_1, \dots, a_k\}$.

①若 $(i+1)$ 号玩家参与第 i 场比赛,因 $s_i = 1$,故 $(i+1)$ 号玩家必胜,此时 $B = \{i+1\}$.

②若 $(i+1)$ 号玩家参与前 $(i-1)$ 场比赛,设 $s[1 \dots (i-1)]$ 中有 cnt_0 个0和 cnt_1 个1.

I)若 $cnt_0 = 0$,则所有比赛都是能力较大者胜,此时 $B = \{i+1\}$.

II)若 $cnt_0 \geq 1$,

i)显然可使得 $(i+1)$ 号玩家是最后的胜者,设最后一轮 $(i+1)$ 号玩家打败 a_l ($1 \leq a_l \leq i$)号玩家.

ii)可用 $(i+1)$ 号玩家可替换前 $(i-1)$ 场比赛中被淘汰的 cnt_0 个能力值较大的玩家之一,使得原本被淘汰的玩家 a_j 成为最后的胜者,设第 i 场比赛中 a_j 号玩家打败 a_m ($1 \leq a_m \leq i$)号玩家.

综上,设 $B_1 = \{a_1, \dots, a_k, (i+1)\} \setminus \{a_l\}$, $B_2 = \{a_1, \dots, a_k, a_j\} \setminus \{a_m\}$, 则 $B = B_1 \cup B_2 \setminus \{a_{lose}\}$, 其中 a_{lose} 为最后一场比赛失败的玩家的编号.

综上, $\max |A| = k = \max |B|$, 故证.

(3) $s_{i-1} = 0, s_i = 1$ 时,

①若 $(i+1)$ 号选手参与第 i 场比赛,则他一定获胜.

②若 $(i+1)$ 号选手参与前 $(i-1)$ 场比赛,设以 s_{i-1} 为右端点的最长连续0的个数为 cnt , 则 $(i+1)$ 号选手可替换前 $(i-1)$ 轮中被淘汰的 cnt 个能力值较大的玩家之一使其成为最后的胜者,故获胜玩家数 $+= cnt - 1$, 其中 -1 是减去原来的胜者.

故获胜玩家数 $+= cnt$.

(4) $s_{i-1} = 1, s_i = 0$ 时,类似于(3),获胜玩家数 $+= cnt$, 其中 cnt 是以 s_{i-1} 为右端点的最长连续1的个数.

综上, 状态转移方程 $\begin{cases} dp[i] = dp[i-1], s[i] = s[i-1] \\ dp[i] = dp[i-1] + cnt, s[i] \neq s[i-1] \end{cases}$

代码III

```

1  void solve() {
2      int n; string s; cin >> n >> s;
3      s = " " + s;
4
5      vector<int> dp(n + 1);
6      dp[1] = 1;
7      for (int i = 1, j; i <= n; i = j) {
8          j = i + 1;
9          int cnt = 1; // 以s[j]结尾的最长连续相同字符的个数

```

```

10     while (j <= n && s[j] == s[i]) {
11         cnt++;
12         dp[j++] = dp[j - 1];
13     }
14
15     if (j <= n) dp[j] = dp[j - 1] + cnt;
16 }
17
18 for (int i = 1; i <= n - 1; i++) cout << dp[i] << " \n"[i == n - 1];
19 }
20
21 int main() {
22     CaseT
23     solve();
24 }

```

10.2.14 Permutation Oddness

原题指路:https://atcoder.jp/contests/abc134/tasks/abc134_f

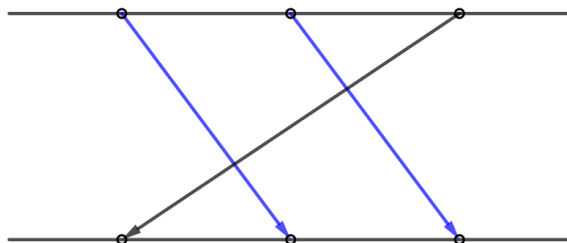
题意 (2 s)

定义一个 $1 \sim n$ ($1 \leq n \leq 50$) 的排列 $p_1, \dots, p_n$ 的权值为 $\sum_{i=1}^n |i - p_i|$. 给定一个整数 m ($0 \leq m \leq n^2$), 求权值为 m 的 $1 \sim n$

的排列的个数, 答案对 $1e9 + 7$ 取模.

思路

将生成排列的过程转化为二分图匹配. 考察上下两根数轴, 上数轴包含 $1 \sim n$ 的 n 个节点, 表示数字, 下数轴包含 $1 \sim n$ 的 n 个节点, 表示盒子, 则生成排列的过程可视为将数字放入盒子中, 若数字 i 放入 j 号盒子, 则上数轴的 i 号节点向下数轴的 j 号节点连边.



$dp[i][j][k]$ 表示只考虑前 i 个数和盒子、有 j 个数和盒子未匹配、预期的权值为 k 的方案数, 其中有 j 个数和盒子未匹配即这些数要放到 i 之后的盒子中, 设数 i 放入 p_i 号盒子, 则 $i < p_i$. 如上图, 这在几何上表现为上数轴从数 i 指向下数轴 p_i 号盒子的边指向右边. k 为预期的权值是因为当前状态 $dp[i][j][k]$ 转移到下一状态 $dp[i+1][j][k]$ 时, 新加入的一个数和一个盒子带来的权值的变化无法提前确定. 初始条件 $dp[0][0][0] = 1$, 即空状态只有一种方案. 最终答案 $ans = dp[n][0][m]$.

考察状态 $dp[i][j][k]$ 的来源. 按新加入的一个数和一个盒子带来的匹配的变化分类, 注意到它们至多增加 2 组匹配:

① 增加 0 组匹配, 即新加入的一个数和一个盒子不与前 $(i-1)$ 个数字和盒子匹配, 则至今未匹配的 j 个数字和盒子只能与 i 之后的数字和盒子匹配, 每个数字匹配盒子将贡献 1 的权值, 每个盒子匹配数字也将贡献 1 的权值, 故权值的增量 $\Delta w = 2j$, 进而 $dp[i][j][k]$ 的上一个状态为 $dp[i-1][j-1][k-2j]$. 显然有该前驱状态的前提是 $j \geq 1$.

显然若新加入的数和盒子不参与匹配只有一种方案.

② 增加 1 组匹配, 权值的增量同 (1), 则 $dp[i][j][k]$ 的上一个状态为 $dp[i-1][j][k-2j]$.

对新加入的数 i 和 i 号盒子, 有如下三种情况: ① 数 i 与 i 号盒子匹配; ② 数 i 与前 j 个盒子匹配; ③ 前 j 个数与 i 号盒子匹配.

故共有 $(2j+1)$ 个方案.

③ 增加 2 组匹配, 权值的增量同 (1), 则 $dp[i][j][k]$ 的上一个状态为 $dp[i-1][j+1][k-2j]$.

此时 $(j+1)$ 个数和 $(j+1)$ 个盒子任意匹配有 $(j+1)^2$ 个方案.

综上,状态转移方程:① $dp[i][j][k] += dp[i-1][j-1][k-2j] \ (j \geq 1)$;② $dp[i][j][k] += (2j+1) \cdot dp[i-1][j][k-2j]$;③ $dp[i][j][k] += (j+1)^2 \cdot dp[i-1][j+1][k-2j]$.总时间复杂度 $O(n^4)$.

代码I

```
1  const int MAXN = 55;
2  const int MOD = 1e9 + 7;
3  Z dp[MAXN][MAXN][MAXN * MAXN];
4
5  void solve() {
6      int n, m; cin >> n >> m;
7
8      dp[0][0][0] = 1; // 初始条件
9      for (int i = 1; i <= n; i++) {
10         for (int j = 0; j <= i; j++) {
11             for (int k = 2 * j; k <= m; k += 2) {
12                 dp[i][j][k] += (j + 1) * (j + 1) * dp[i - 1][j + 1][k - 2 * j]
13                     + (2 * j + 1) * dp[i - 1][j][k - 2 * j];
14                 if (j) dp[i][j][k] += dp[i - 1][j - 1][k - 2 * j];
15             }
16         }
17     }
18     cout << dp[n][0][m] << endl;
19 }
20
21 int main() {
22     solve();
23 }
```

思路II

注意到 $\sum_{i=1}^n (i - p_i) = 0$, 则 $\sum_{i=1}^n |i - p_i|$ 为偶数, 故 k 为奇数时无解.

同**思路I**, 转化为数放入盒子的方案数.

$dp[i][j][k]$ 表示前 i 个数中有 j 个 $> i$ 的数且权值为 k 的方案数, 则初始条件 $dp[1][0][0] = dp[1][1][0] = 1$, 即排列的第一个数是否为

1. 注意到 $\sum_{i < p_i} (i - p_i) = \sum_{i > p_i} (i - p_i) = \frac{\sum_{i=1}^n |i - p_i|}{2}$, 故使得 $\sum_{i=1}^n |i - p_i| = m$ 的方案数即 $dp[n][0][m/2]$.

事实上这样的状态定义与**思路I**中的状态定义等价, 只是第三维变为原来的一半, 故状态转移方程也是类似的.

代码II

```
1  const int MAXN = 55;
2  const int MOD = 1e9 + 7;
3  Z dp[MAXN][MAXN][MAXN * MAXN];
4
5  void solve() {
6      int n, m; cin >> n >> m;
7
8      dp[1][0][0] = dp[1][1][0] = 1; // 初始条件
9      for (int i = 1; i <= n; i++) {
10         for (int j = 0; j <= i; j++) { // 枚举前i个数中比i大的数
11             for (int k = 0; k <= i * i / 2; k++) { // 只需计算最大权值至为i*i/2
12                 dp[i + 1][j][k + j] += (2 * j + 1) * dp[i][j][k];
13                 dp[i + 1][j + 1][k + j] += dp[i][j][k];
14                 if (j) dp[i + 1][j - 1][k + j] += j * j * dp[i][j][k];
15             }
16         }
17     }
18     cout << dp[n][0][m/2] << endl;
19 }
20
21 int main() {
22     solve();
23 }
```

```

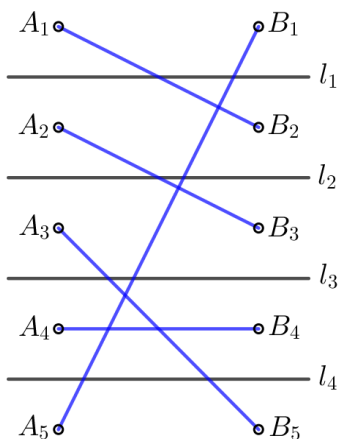
15     }
16     }
17     }
18     cout << (m & 1 ? 0 : dp[n][0][m / 2]) << endl;
19 }
20
21 int main() {
22     solve();
23 }

```

思路III

思路II中 $\sum_{i=1}^n |i - p_i|$ 为偶数的另一证明:考察每新加入一个数和一个盒子时对权值的贡献,注意到权值的变化量为 ± 2 ,故证.

排列的权值的另一解释:考察排列 $\{5, 1, 2, 4, 3\}$.如下图,节点 $A_1 \sim A_5$ 表示数 $1 \sim 5$,节点 $B_1 \sim B_5$ 表示下标 $1 \sim 5$.若数 i 在下标 p_i 位置,则 A_i 向 B_{p_i} 连边.在相邻两行节点间作水平线,则排列的权值即所有线段与水平线的交点数之和(交点重合视为多个交点).交点总是成对出现也表明权值只能为偶数.



状态定义同**思路I**,考察 $dp[i][j][k]$ 的前驱状态.按节点 A_i 与节点 B_j 匹配情况分类:

- ①之前的 A_i 与之后的 B_j 匹配,如上图中的 $A_1 B_2, A_2 B_3, A_3 B_5$,此时至少增加一个交点.
- ②当前的 A_i 与当前的 B_i ($j = i$)匹配,此时不增加交点.
- ③之后的 A_i 与之前的 B_j 匹配,如上图中的 $A_5 A_1$,此时至少增加一个交点.

更准确地,考察新增的数和盒子的匹配情况:

- (1)新增的数和盒子不与之前的数和盒子匹配,如 $i = 1$ 时,新增的数 A_1 和盒子 B_1 不与之前的数和盒子匹配,此时新增0组匹配,对应前驱状态 $dp[i - 1][j][k - 2j]$.

- (2)对新增的数 i 和盒子 i ,有如下三种情况:

- ①数 i 与盒子 i 匹配,如 $i = 4$ 时,新增的数 A_4 与新增的盒子 B_4 匹配.
- ②数 i 与前 j 个盒子匹配,如 $i = 5$ 时,新增的数 A_5 与之前的盒子 B_1 匹配.
- ③前 j 个数与盒子 i 匹配,如 $i = 2$ 时,新增的盒子 B_2 与之前的数 A_1 匹配.

上述情况都新增1组匹配,对应前驱状态 $dp[i - 1][j][k - 2j]$.

- (3)新增的数 i 和盒子 i 连同前面的 j 个数和 j 个盒子任意匹配,此时新增2组匹配,对应前驱状态 $dp[i - 1][j + 1][k - 2j]$.

状态转移方程同**思路I**.

10.2.15 Min Max Sort

原题指路:<https://codeforces.com/contest/1792/problem/C>

题意 (2 s)

给定一个 $1 \sim n$ 的排列 $p_1, \dots, p_n$. 现有操作: 选择两不同的元素 x 和 y , 先将它们都删去, 再将 $\min\{x, y\}$ 插到排列的开头, $\max\{x, y\}$ 插到排列的结尾. 问使得 $p[]$ 升序所需的最小操作次数.

有 t ($1 \leq t \leq 1e4$) 组测试数据. 每组测试数据第一行输入一个整数 n ($1 \leq n \leq 2e5$). 第二行输入一个 $1 \sim n$ 的排列 $p_1, \dots, p_n$. 数据保证所有测试数据的 n -之和不超过 $2e5$.

思路

显然为使得最终序列有序, 应从中间的数往两边操作. 注意到某一步操作数对 (x, y) 后, 之后的 $(x-1, y+1), (x-2, y+2), \dots$ 都需操作, 故只需找到第一个需要操作的数对即可. 具体地, 显然最坏步数不超过 $\left\lfloor \frac{n}{2} \right\rfloor$, 则 ans 从 $\left\lfloor \frac{n}{2} \right\rfloor$ 开始, 考察操作过程中是否有已有序而无需操作的数对, 直至发现第一个需操作的数对.

设当前检查的区间为 $[l, r]$, 数值 x 的下标为 pos_x (下标从 1 开始). 以 n 的奇偶性分类:

(1) n 为奇数时, 初始时 $l = r = pos_{\frac{n}{2}+1}$, 即从中间位置 $\left\lfloor \frac{n}{2} \right\rfloor$ 开始往两边检查, 找到第一个需交换的数对.

具体地, i 从 $\left\lfloor \frac{n}{2} \right\rfloor$ 枚举到 1, 每次检查数对 $(i, n-i-1)$ 是否需操作.

数对 $(i, n-i-1)$ 需操作当且仅当至少满足下列条件之一:

① 两元素构成逆序对, 即 $pos[i] > pos[n-i-1]$.

② 数值 i 不在区间 $[l, r]$ 的两侧 (含端点), 则数值 i 和 $(n-i+1)$ 中至少有一个在区间 (l, r) 中, 即 $pos[i] > l$ 或 $pos[n-i+1] < r$.

(2) n 为偶数时, 不存在中间位置, 则 l 和 r 从第一对检查的数对所确定的区间开始, 即 $l = pos[n/2], r = pos[n-l+1]$.

i 的枚举和数对 $(i, n-i-1)$ 是否需操作的检查同 (1).

代码

```
1 void solve() {
2     int n; cin >> n;
3     vector<int> pos(n + 1);
4     for (int i = 1; i <= n; i++) {
5         int p; cin >> p;
6         pos[p] = i;
7     }
8
9     int ans = n / 2;
10    if (n & 1) {
11        int l = pos[(n + 1) / 2], r = pos[(n + 1) / 2];
12        for (int i = n / 2; i; i--) {
13            if (pos[i] > pos[n - i + 1] || pos[i] > l || pos[n - i + 1] < r)
14                break; // (i, n-i+1) 及之后的数对都需交换
15            else {
16                ans--;
17                l = pos[i], r = pos[n - i + 1];
18            }
19        }
20    }
21    else {
22        int l = -1, r = -1;
23        for (int i = n / 2; i; i--) {
24            if (pos[i] > pos[n - i + 1]) break; // (i, n-i+1) 及之后的数对都需交换
25            else {
26                if (~l && (pos[i] > l || pos[n - i + 1] < r))
27                    break; // (i, n-i+1) 及之后的数对都需交换
```

```

28         else {
29             ans--;
30             l = pos[i], r = pos[n - i + 1];
31         }
32     }
33 }
34 }
35 cout << ans << endl;
36 }
37
38 int main() {
39     CaseT
40     solve();
41 }

```

思路II

同思路I,但无需分奇偶.这种实现采用下标从0开始更方便.

代码II

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> pos(n);
4     for (int i = 0; i < n; i++) {
5         int x; cin >> x;
6         pos[--x] = i;
7     }
8
9     int l = n / 2, r = n - 1;
10    while (l && pos[l - 1] < pos[l] && pos[r] > pos[r - 1])
11        l--, r++;
12    cout << l << endl;
13 }
14
15 int main() {
16     CaseT
17     solve();
18 }

```

思路III

考虑求出序列的所有极大连续上升子列,固定一个极大连续上升子列,使得操作后该子列"自动地"排到正确的位置.固定每个极大上升子列 $[q_1, \dots, q_k]$ 后,还需将 $< q_1$ 的元素和 $> q_k$ 的元素排序,则该极大上升子列所需的操作步数为 $\max\{q_1 - 1, n - q_k\}$,最终答案为所有极大上升子列所需的操作步数的最小值.

[例] $n = 5$,序列 $p = [1, 5, 4, 2, 3]$ 中有三个极大上升子列 $[1, 2, 3]$, $[5]$, $[4]$.

① $[1, 2, 3]$ 所需的操作步数为 $\max\{1 - 1, 5 - 3\} = 2$.

② $[5]$ 所需的操作步数为 $\max\{5 - 1, 5 - 5\} = 4$.

③ $[4]$ 所需的操作步数为 $\max\{4 - 1, 5 - 4\} = 3$.

故 $ans = \min\{2, 4, 3\} = 2$.

考虑如何求序列的所有极大连续上升子列. $dp[x]$ 表示以数值 x 结尾的最长连续上升子列的长度,状态转移方程 $dp[x] = dp[x - 1] + 1$,初始条件 $dp[x] = 0$ ($1 \leq x \leq n$).

预处理出 $dp[x]$ 后,每个 $x \in [1, n]$ 所在的最长连续上升子列对答案的贡献为 $\max\{x - dp[x], n - x\}$.

代码III

```

1 void solve() {
2     int n; cin >> n;
3
4     int ans = n;
5     vector<int> dp(n + 1); // dp[x]表示以数值x结尾的最长连续上升子列的长度
6     for (int i = 1; i <= n; i++) {
7         int p; cin >> p;
8
9         dp[p] = dp[p - 1] + 1;
10        ans = min(ans, max(p - dp[p], n - p));
11    }
12    cout << ans << endl;
13 }
14
15 int main() {
16     CaseT
17     solve();
18 }

```

10.2.16 Almost Triple Deletions

原题指路:<https://codeforces.com/problemset/problem/1699/D>

题意 (2 s)

给定一个长度为 n 的序列 $a = [a_1, \dots, a_n]$. 现有操作: 选择两个相邻的不相等的元素, 将它们删除. 对 $a[]$ 进行若干次(可能为0次)操作, 问最后得到的所有元素都相等的序列的最大长度.

有 t ($1 \leq t \leq 1000$)组测试数据. 每组测试数据第一行输入一个整数 n ($1 \leq n \leq 5000$). 第二行输入 n 个整数 $a_1, \dots, a_n$ ($1 \leq a_i \leq n$). 数据保证所有测试数据的 n 之和不超过 $1e4$.

思路

[定理] 一个区间能被删完当且仅当该区间的长度 len 为偶数, 且其中任一元素的出现次数不超过 $\frac{len}{2}$.

[证] 最优策略: 每次删除区间中出现次数最多的元素和一个相邻的不等元素.

类似于用区间DP判断回文串的思路, $dp1[l][r]$ 表示区间 $[l, r]$ 能否被删完, 显然可 $O(n^2)$ 预处理出该数组.

$dp2[i]$ 表示对前缀 $a[1 \dots i]$ 操作所能得到的所有元素都相等的序列的最大长度, 初始条件 $dp[i] = [i = 1 \vee dp1[1][i - 1]]$, 即初始时 $dp[i] = 1$ 当且仅当 $i = 1$ 或能删完前缀 $a[1 \dots (i - 1)]$, 最终答案 $ans = \max\{dp2[n], \max_{1 \leq i \leq n-1} dp2[i] \text{ if } dp1[i+1][n]\}$.

对每个 $dp2[i]$, 枚举前缀的分界点 $j \in [1, i - 1]$, 若 $a[j] = a[i]$ 且能删完区间 $a[(j + 1) \dots (i - 1)]$ s.t. $a[j]$ 与 $a[i]$ 相邻, 则状态转移方程 $dp2[i] = \max\{dp2[i], dp2[j] + 1\}$, 状态转移条件 $i = j + 1$ 或 $dp1[j + 1][i - 1]$, 且 $dp2[j] > 0$, 即能删完区间 $a[(j + 1) \dots (i - 1)]$ 且能经若干次操作使得前缀 $a[1 \dots j]$ 的所有元素都相等, 其中特判 $i = j + 1$ 是因为此时 $dp1[j + 1][i - 1]$ 为 $dp1[j + 1][j]$, 此处也可通过初始时令 $dp[i + 1][i] = true$ 来避免特判.

代码

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6     // dp1[l][r]表示区间[l, r]能否被删完
7     vector<vector<bool>> dp1(n + 5, vector<bool>(n + 5));

```

```

8   for (int i = 1; i <= n; i++) {
9       vector<int> cnt(n + 1);
10      int maxcnt = 0;
11      for (int j = i; j <= n; j++) {
12          int len = j - i + 1;
13          maxcnt = max(maxcnt, ++cnt[a[j]]);
14          dp1[i][j] = !(len & 1) && maxcnt <= len / 2;
15      }
16  }

17
18  vector<int> dp2(n + 1); // dp2[i]表示对前缀a[1 ... i]操作所能得到的相同元素的最大长度
19  for (int i = 1; i <= n; i++) {
20      dp2[i] = (i == 1) || dp1[1][i - 1]; // 初始条件
21      for (int j = i - 1; j >= 1; j--) { // 枚举分界点
22          if (a[j] == a[i]) { // 可删完区间a[(j + 1) ... (i - 1)] s.t. a[j]与a[i]相邻
23              // 注意特判i = j + 1的情况, 注意dp2[j] > 0时才能转移
24              if ((i == j + 1 || dp1[j + 1][i - 1]) && dp2[j])
25                  dp2[i] = max(dp2[i], dp2[j] + 1);
26          }
27      }
28  }

29
30  int ans = dp2[n]; // 注意初始值不能为0
31  for (int i = 1; i < n; i++)
32      if (dp1[i + 1][n]) ans = max(ans, dp2[i]);
33  cout << ans << endl;
34  }

35
36  int main() {
37      CaseT
38      solve();
39  }

```

10.2.17 Beautiful Array

原题指路: <https://codeforces.com/problemset/problem/1155/D>

题意 (2 s)

给定一个长度为 n ($1 \leq n \leq 3e5$)的序列 $a = [a_1, \dots, a_n]$ ($-1e9 \leq a_i \leq 1e9$)和一个整数 x ($-100 \leq x \leq 100$). 现有操作: 选择 $a[]$ 的一个区间, 令其中的元素 $*$ $= x$. 求至多进行一次操作后 $a[]$ 中区间和的最大值, 区间可为空.

思路

$dp[i][j]$ 表示只考虑前 i 个元素时区间和的最大值, 其中 $j = 0$ 表示不做操作; $j = 1$ 表示 a_i 加入当前选择的区间, 且做操作 $a_i * = x$; $j = 2$ 表示 a_i 加入当前选择的区间, 且不做操作 $a_i * = x$. 初始条件 $dp[0][0/1/2] = 0$, 最终答案 $ans = \max_{1 \leq i \leq n} \{dp[i][0], dp[i][1], dp[i][2]\}$.

状态转移方程:

① $j = 0$ 时, 不做操作即求序列的最大区间和.

按 $a[i]$ 是否加入当前选择的区间分类, 有 $dp[i][0] = \max\{dp[i - 1] + a[i], 0\}$.

② $j = 1$ 时, 能 a_i 加入当前选择的区间且做操作 $a_i * = x$ 的前提是 a_{i-1} 做操作 $a_{i-1} * = x$.

$dp[i][1] = \max\{dp[i][0], dp[i - 1][1] + a[i] * x\}$.

③ $j = 2$ 时, 能 a_i 加入当前选择的区间且不做操作 $a_i * = x$ 的前提是 a_{i-1} 不做操作 $a_{i-1} * = x$.

$dp[i][2] = \max\{dp[i][1], dp[i - 1][2] + a[i]\}$.

代码

```

1 void solve() {
2     int n, x; cin >> n >> x;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6
7     vector<vector<ll>> dp(n + 1, vector<ll>(3));
8     ll ans = 0;
9     for (int i = 1; i <= n; i++) {
10         dp[i][0] = max(dp[i - 1][0] + a[i], (ll)0);
11         dp[i][1] = max(dp[i][0], dp[i - 1][1] + (ll)a[i] * x);
12         dp[i][2] = max(dp[i][1], dp[i - 1][2] + a[i]);
13         ans = max(ans, dp[i][2]);
14     }
15     cout << ans << endl;
16 }
17
18 int main() {
19     solve();
20 }

```

10.2.18 Longest Regular Bracket Sequence

原题指路: <https://codeforces.com/problemset/problem/5/C>

题意 (2 s)

给定一个只包含字符 '(' 和 ')' 的、长度不超过 $1e6$ 的字符串 s , 求其中合法括号序列的最大长度和该长度的合法括号序列在 s 中出现的次数. 若不存在合法括号序列, 输出 "0 1".

思路

$legal[i]$ 表示 $s[i]$ 是否在某个合法括号序列中, 如:

下标 i	1	2	3	4	5	6	7	8	9	10	11	12	13	14
字符 $s[i]$	)	(	(	(	)	)	)	)	(	(	)	(	)	)
$legal[i]$	0	1	1	1	1	1	1	0	1	1	1	1	1	1

$legal[]$ 可用栈做括号匹配预处理出. 问题转化为求 $legal[]$ 中最长连续 1 的长度和该长度的连续 1 在 $legal[]$ 中出现的次数.

$dp[i]$ 表示以 $legal[i]$ 结尾的最长连续 1 的个数. 状态转移方程 $dp[i] = \begin{cases} dp[i-1] + 1, & legal[i] = 1 \\ 0, & legal[i] = 0 \end{cases}$, 则最长连续 1 的长度

$$length = \max_{1 \leq i \leq n} dp[i].$$

求得 $length$ 后, 类似于 DP 的过程, 再扫一遍求得该长度的连续 1 在 $legal[]$ 中出现的次数.

时间复杂度 $O(|s|)$.

代码

```

1 void solve() {
2     string s; cin >> s;
3
4     int len = s.length();
5     vector<int> legal(len); // legal[i] 表示 s[i] 是否在某个合法括号序列中
6     stack<int> stk; // 栈存下标

```

```

7   for (int i = 0; i < len; i++) {
8       if (s[i] == '(') stk.push(i);
9       else if (stk.size()) {
10          legal[stk.top()] = legal[i] = true;
11          stk.pop();
12      }
13  }
14
15  legal.push_back(0); // 哨兵
16  len++;
17
18  int dp = 0; // dp[i]表示以legal[i]结尾的最长连续1的个数
19  int length = 0; // legal[]中最长连续1的长度
20  for (int i = 0; i < len; i++) {
21      if (legal[i]) dp++;
22      else {
23          length = max(length, dp);
24          dp = 0;
25      }
26  }
27
28  dp = 0; // 清空
29  int cnt = 0; // 长度为length的连续1在legal[]中出现的次数
30  for (int i = 0; i < len; i++) {
31      if (legal[i]) dp++;
32      else {
33          if (dp == length) cnt++;
34          dp = 0;
35      }
36  }
37  cout << length << ' ' << (length ? cnt : 1) << endl;
38  }
39
40  int main() {
41      solve();
42  }

```

10.2.19 Maximum sum

原题指路: <http://poj.org/problem?id=2479>

题意

有 t ($1 \leq t \leq 30$)组测试数据. 每组测试数据给定一个长度为 n ($2 \leq n \leq 5e4$)的序列 $a = [a_1, \dots, a_n]$ ($|a_i| \leq 1e4$). 求

$$\max_{1 \leq s_1 \leq t_1 < s_2 \leq t_2 \leq n} \left\{ \sum_{i=s_1}^{t_1} a_i + \sum_{j=s_2}^{t_2} a_j \right\}, \text{ 即求 } a[] \text{ 的两个不交的子段使得它们之和最大.}$$

思路

若只求 $a[]$ 的最大连续子段和, 则可用线性DP在 $O(n)$ 的时间复杂度内求解.

预处理 $a[]$ 的前缀 $a[1 \dots i]$ 的最大连续子段和 $pre[i]$ 和后缀 $a[i \dots n]$ 的最大连续子段和 $suf[i]$. 注意欲求式中 $s_1 \leq t_1$ 且 $s_2 \leq t_2$, 则两子段都非空, 故初始条件 $pre[0] = -INF, suf[n+1] = -INF$. 枚举分界点, 最终答案 $ans = \max_{1 \leq i \leq n} \{pre[i] + suf[i+1]\}$.

考虑如何求 $pre[]$ 和 $suf[]$. $dp1[i]$ 表示以 $a[i]$ 结尾的最大连续字段和, $dp2[i]$ 表示以 $a[i]$ 开始的最大连续子段和, 则 $pre[i] = \max\{pre[i-1], dp1[i]\}, suf[i] = \max\{suf[i+1], dp2[i]\}$.

代码I

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6     vector<int> pre(n + 1); // pre[i]表示前缀a[1...i]的最大连续子段和
7     vector<int> dp(n + 2); // dp[i]表示以a[i]结尾的最大连续子段和, 注意空间
8     pre[0] = dp[0] = -INF; // 初始条件
9     for (int i = 1; i <= n; i++) {
10         dp[i] = max(dp[i - 1], 0) + a[i];
11         pre[i] = max(pre[i - 1], dp[i]);
12     }
13
14     // dp[i]表示以a[i]开始的最大连续子段和
15     vector<int> suf(n + 2); // suf[i]表示后缀a[i...n]的最大连续子段和, 注意空间
16     suf[n + 1] = dp[n + 1] = -INF; // 初始条件
17     for (int i = n; i; i--) {
18         dp[i] = max(dp[i + 1], 0) + a[i];
19         suf[i] = max(suf[i + 1], dp[i]);
20     }
21
22     int ans = -INF;
23     for (int i = 1; i <= n; i++)
24         ans = max(ans, pre[i] + suf[i + 1]);
25     cout << ans << endl;
26 }
27
28 int main() {
29     CaseT
30     solve();
31 }

```

思路II

代码I中的 $dp[i]$ 只与 $dp[i - 1]$ 或 $dp[i + 1]$ 有关, 故可用变量代替.

代码II

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5
6     vector<int> pre(n + 1); // pre[i]表示前缀a[1...i]的最大连续子段和
7     pre[0] = -INF; // 初始条件
8     for (int i = 1, dp = -INF; i <= n; i++) { // dp[i]表示以a[i]结尾的最大连续子段和
9         dp = max(dp, 0) + a[i];
10        pre[i] = max(pre[i - 1], dp);
11    }
12
13    vector<int> suf(n + 2); // suf[i]表示后缀a[i...n]的最大连续子段和, 注意空间
14    suf[n + 1] = -INF; // 初始条件
15    for (int i = n, dp = -INF; i; i--) { // dp[i]表示以a[i]开始的最大连续子段和
16        dp = max(dp, 0) + a[i];
17        suf[i] = max(suf[i + 1], dp);
18    }
19
20    int ans = -INF;
21    for (int i = 1; i <= n; i++)

```

```

22     ans = max(ans, pre[i] + suf[i + 1]);
23     cout << ans << endl;
24 }
25
26 int main() {
27     CaseT
28     solve();
29 }

```

10.2.20 鸡蛋掉落

原题指路: <https://leetcode.cn/problems/super-egg-drop/>

题意

有 m ($1 \leq m \leq 100$) 个相同的鸡蛋, 用从低到高分别为 $1 \sim n$ ($1 \leq n \leq 1e4$) 层楼实验. 已知存在分界楼层 $f \in [0, n]$ s.t. 鸡蛋从 $\forall$ 高于 f 的楼层落下都会打破, 而从 $\forall$ 不高于 f 的楼层落下都不会打破. 每次操作可取一个未打破的鸡蛋并将其从任一楼层 $x \in [1, n]$ 落下, 若鸡蛋打破, 则丢弃; 否则可重复使用该鸡蛋. 求确定 f 所需的最小实验次数.

思路

$dp[i][j]$ 表示用 j 个鸡蛋找到 i 层楼的分界楼层 f 所需的最小实验次数. 初始条件 $dp[i][j] = \begin{cases} i, j = 1 \\ INF, 2 \leq j \leq m \end{cases}$ ($1 \leq i \leq n$). 最终答案 $ans = dp[n][m]$.

按第一个鸡蛋落下的楼层 k 分类:

① 若鸡蛋打破, 则 $f \in [1, k - 1]$, 还需用剩下的 $(j - 1)$ 个鸡蛋确定 $1 \sim (k - 1)$ 层中的分界楼层.

状态转移方程 $dp[i][j] = \min_{1 \leq k \leq i} \{dp[k - 1][j - 1]\} + 1$.

② 若鸡蛋未打破, 则 $f \in [k, n]$, 还需用剩下的 j 个鸡蛋确定 $(k + 1) \sim n$ 层中的分界楼层.

状态转移方程 $dp[i][j] = \min_{1 \leq k \leq i} \{dp[i - k][j]\} + 1$.

综上, 状态转移方程 $dp[i][j] = \min_{1 \leq k \leq i} \{\max\{dp[k - 1][j - 1], dp[i - k][j]\} + 1\}$.

状态数 $O(nm)$, 转移 $O(n)$, 总时间复杂度 $O(n^2m)$.

代码

```

1  class Solution {
2  public:
3      const int INF = 0x3f3f3f3f;
4
5      int superEggDrop(int k, int n) {
6          int m = k;
7
8          // dp[i][j] 表示用 j 个鸡蛋找到 i 层楼的分界楼层 f 所需的最小实验次数
9          vector<vector<int>> dp(n + 1, vector<int>(m + 1));
10         for (int i = 0; i <= n; i++) dp[i][1] = i; // 初始条件
11
12         for (int i = 1; i <= n; i++) { // 枚举楼层
13             for (int j = 2; j <= m; j++) { // 枚举鸡蛋数, 注意从2开始
14                 dp[i][j] = INF; // 初始条件
15                 for (int k = 1; k <= i; k++) // 枚举第一个鸡蛋落下的楼层
16                     dp[i][j] = min(dp[i][j], max(dp[k - 1][j - 1], dp[i - k][j]) + 1);
17             }
18         }
19         return dp[n][m];
20     }

```


思路II

$dp[i][j]$ 表示用 j 个鸡蛋找到 i 层楼的分界楼层 f 所需的最小实验次数. 考察思路I中的状态转移方程

$$dp[i][j] = \min_{1 \leq k \leq i} \{\max\{dp[k-1][j-1], dp[i-k][j]\} + 1\}.$$

注意到 k 从1增大到 i 时,

(1) $dp[k-1][j-1]$ 不减, 因为鸡蛋数 $(j-1)$ 固定时, 楼层越多, 所需的实验次数越多.

① $k=1$ 时, $dp[k-1][j-1]$ 取得最小值 $dp[0][j-1] = 0$.

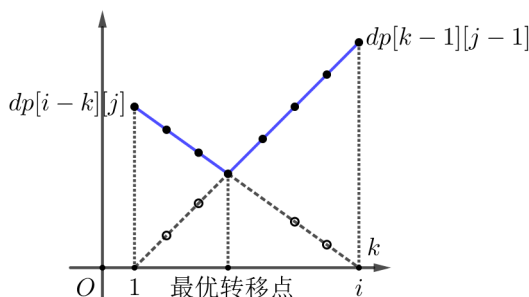
② $k=i$ 时, $dp[k-1][j-1]$ 取得最大值 $dp[i-1][j-1]$.

(2) $dp[i-k][j]$ 不减, 因为鸡蛋数 j 固定时, 楼层越少, 所需的实验次数越少.

① $k=1$ 时, $dp[i-k][j]$ 取得最大值 $dp[i-1][j]$.

② $k=i$ 时, $dp[i-k][j]$ 取得最小值 $dp[0][j] = 0$.

则 $\max\{dp[k-1][j-1], dp[i-k][j]\}$ 的图象如下图蓝色实线所示(事实上函数的图象是离散的点):



则最优转移点为两实线的交点. 因 $dp[k-1][j-1]$ 和 $dp[i-k][j]$ 的自变量和取值都是整数, 故交点必存在.

将代码I中的最内层循环改为二分最佳转移点, 转移时间复杂度 $O(\log n)$, 总时间复杂度 $O(nm \log n)$.

代码II

```
1 class Solution {
2 public:
3     const int INF = 0x3f3f3f3f;
4
5     int superEggDrop(int k, int n) {
6         int m = k;
7
8         // dp[i][j]表示用j个鸡蛋找到i层楼的分界楼层f所需的最小实验次数
9         vector<vector<int>> dp(n + 1, vector<int>(m + 1));
10        for (int i = 0; i <= n; i++) dp[i][1] = i; // 初始条件
11
12        for (int i = 1; i <= n; i++) { // 枚举楼层
13            for (int j = 2; j <= m; j++) { // 枚举鸡蛋数, 注意从2开始
14                // 二分出最优转移点
15                int l = 1, r = i;
16                while (l < r) {
17                    int mid = l + r >> 1;
18                    if (dp[mid - 1][j - 1] >= dp[i - mid][j]) r = mid;
19                    else l = mid + 1;
20                }
21                dp[i][j] = dp[l - 1][j - 1] + 1;
22            }
23        }
24        return dp[n][m];
25    }
26 }
```

```

25     }
26 };

```

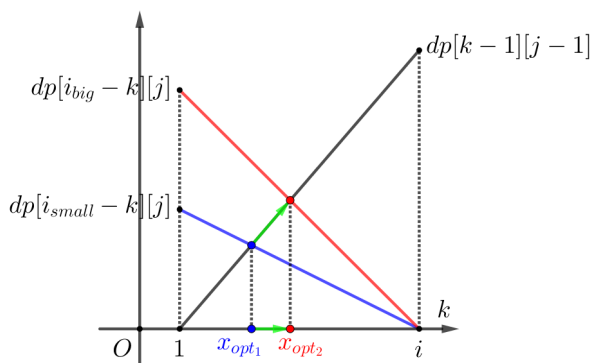
思路III

同思路II, 但用另一种方法求最佳转移点.

$dp[i][j]$ 表示用 j 个鸡蛋找到 i 层楼的分界楼层 f 所需的最小实验次数. 考察思路I中的状态转移方程

$$dp[i][j] = \min_{1 \leq k \leq i} \{ \max\{dp[k-1][j-1], dp[i-k][j]\} + 1 \}.$$

注意到对固定的 j 和 k , 随着 i 的增大, $dp[k-1][j-1]$ 不变, 但 $dp[i-k][j]$ 不减, 而随着 k 的增大, $dp[k-1][j-1]$ 不减, $dp[i-k][j]$ 不减, 则随着 i 增大时, $dp[i-k][j]$ 的最大值不减, 在 $dp[k-1][j-1]$ 不变时, 如下图, 两线段的交点(最优转移点)不左移, 即决策有单调性, 可用记录最后一个最优转移点, 每次转移时尝试右移最优转移点.



因最佳转移点单调右移至 n , 则转移的均摊时间复杂度为 $O(1)$, 总时间复杂度 $O(nm)$.

代码III

```

1  class Solution {
2  public:
3      int superEggDrop(int k, int n) {
4          int m = k;
5
6          // dp[i][j]表示用j个鸡蛋找到i层楼的分界楼层f所需的最小实验次数
7          vector<vector<int>>> dp(n + 1, vector<int>(m + 1));
8          for (int i = 0; i <= n; i++) dp[i][1] = i; // 初始条件
9
10         for (int j = 2; j <= m; j++) { // 枚举鸡蛋数
11             int x = 1; // 最优转移点
12             for (int i = 1; i <= n; i++) { // 枚举步数
13                 while (x < i && max(dp[x - 1][j - 1], dp[i - x][j]) >= max(dp[x][j - 1], dp[i - x -
14                     1][j]))
15                     x++; // 求最优转移点
16                 dp[i][j] = max(dp[x - 1][j - 1], dp[i - x][j]) + 1; // 注意最优转移点为(x - 1)
17             }
18         }
19         return dp[n][m];
20     };
21 }

```

思路IV

思路I中 $dp[i][j]$ 表示用 j 个鸡蛋找到 i 层楼的分界楼层 f 所需的最小实验次数. 考虑将DP的维度与表示的值交换, $dp[i][j]$ 表示用 i 个鸡蛋在 j 步内可确定分界楼层的楼层数的最大值. 问题转化为求 $s.t. dp[m][step] \leq n$ 的 $step$ 的最小值.

考虑如何求 $dp[i][j]$. 初始条件 $dp[i][1] = 1$ ($1 \leq i \leq m$), 即用任意数量的鸡蛋在一步内只能确定1层楼的分界楼层.

按最后一个鸡蛋的情况分类:

①若鸡蛋打破, 则可用剩下的 $(i - 1)$ 个鸡蛋找到往下的 $(j - 1)$ 层楼的分界楼层.

②若鸡蛋未打破, 则可用剩下的 i 个鸡蛋找到往上的 $(j - 1)$ 层楼的分界楼层.

综上, 状态转移方程 $dp[i][j] = dp[i][j] = dp[i - 1][j - 1] + dp[i][j - 1] + 1$. 状态数 $O(nm)$, 转移 $O(1)$, 总时间复杂度 $O(nm)$.

下面的**代码IV**中的 $dp[][]$ 的定义与上述相同, 转移时先枚举第二维, 再枚举第一维, 内存不连续访问, 耗时4 ms; 下面的**代码V**中的 $dp[][]$ 将二维交换, 转移时先枚举第一维, 再枚举第二维, 内存连续访问, 耗时68 ms, 与"内存连续访问时效率高"的经验不同, 可能的原因如下:

通过	68 ms	46.1 MB	C++	2023/04/11 22:58	内存连续访问
通过	4 ms	28.8 MB	C++	2023/04/11 21:26	内存不连续访问

(1)①**代码IV**的第一次转移为 $dp[1][2] = dp[0][1] + dp[1][1] + 1$.

计算等号右边时, 先取出 $dp[0][1]$ 和 $dp[1][1]$, 同时将 $dp[][]$ 的第0、1行放入cache.

计算的结果赋值给 $dp[1][2]$, 发生cache hit.

②**代码V**的第一次转移为 $dp[2][1] = dp[1][0] + dp[1][1] + 1$.

计算等号右边时, 先取出 $dp[1][0]$ 和 $dp[1][1]$, 同时将 $dp[][]$ 的第1行放入cache.

计算的结果复制给 $dp[2][1]$, 发生cache miss.

事实上, 这种解释可能不成立, 正确的解释如下:

(2)本题的数据范围是 $m \leq 100, n \leq 1e4$. **代码VI**定义 $dp[][]$ 时调用了 $(m + 1)$ 次vector的构造函数, **代码V**定义 $dp[][]$ 时调用了 $(n + 1)$ 次vector的构造函数. 进行 $n = 30, m = 3e5$ 的实验, 发现内存连续访问的**代码V**比内存不连续访问的**代码VI**效率高.

综上, 实验结果与"内存连续访问时效率高"的经验不相悖, 造成效率差异的原因是统计了定义 $dp[][]$ 的时间.

代码IV

```

1  class Solution {
2  public:
3      int superEggDrop(int k, int n) {
4          int m = k;
5
6          // dp[i][j]表示用i个鸡蛋在j步内可确定分界楼层的楼层数的最大值
7          vector<vector<int>> dp(m + 1, vector<int>(n + 1));
8          for (int i = 1; i <= m; i++) dp[i][1] = 1; // 初始条件
9
10         for (int j = 2; j <= n; j++){ // 枚举步数
11             for (int i = 1; i <= m; i++) { // 枚举鸡蛋数
12                 dp[i][j] = dp[i - 1][j - 1] + dp[i][j - 1] + 1;
13                 if (dp[i][j] >= n) return j;
14             }
15         }
16         return n;
17     }
18 };

```

代码V

```

1  class Solution {
2  public:
3      int superEggDrop(int k, int n) {
4          int m = k;
5
6          // dp[i][j]表示用j个鸡蛋在i步内可确定分界楼层的楼层数的最大值
7          vector<vector<int>> dp(n + 1, vector<int>(m + 1));

```

```

8         for (int j = 1; j <= m; j++) dp[1][j] = 1; // 初始条件
9
10        for (int i = 2; i <= n; i++) { // 枚举步数
11            for (int j = 1; j <= m; j++) { // 枚举鸡蛋数
12                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j] + 1;
13                if (dp[i][j] >= n) return i;
14            }
15        }
16        return n;
17    }
18 };

```

思路V

注意到 $dp[i][j]$ 只与 $dp[i - 1][j]$ 有关, 可用滚动数组优化.

代码VI

```

1  class Solution {
2  public:
3      int superEggDrop(int k, int n) {
4          int m = k;
5
6          // dp[i][j]表示用j个鸡蛋在i步内可确定分界楼层的楼层数的最大值
7          vector<int> dp(m + 1);
8          for (int j = 1; j <= m; j++) dp[j] = 1; // 初始条件
9
10         for (int i = 2; i <= n; i++) { // 枚举步数
11             vector<int> pd(m + 1);
12             for (int j = 1; j <= m; j++) { // 枚举鸡蛋数
13                 pd[j] = dp[j - 1] + dp[j] + 1;
14                 if (pd[j] >= n) return i;
15             }
16             dp = pd;
17         }
18         return n;
19     }
20 };

```

10.3 区间DP

区间DP一般有四种类型:①环状变链状;②记录方案数;③区间DP+高精;④二维区间DP.

区间DP有两种常用的实现方式:①迭代式,常用于一维;②记搜,常用于二维.

迭代式区间DP的模板:

```

1  for (int len = 1; len <= n; len++){ // 枚举区间长度
2      for (int l = 1; l + len - 1 <= n; l++){ // 枚举区间左端点
3          r = l + len - 1;
4          do sth...
5      }
6  }

```

10.3.1 石子合并

题意

编号为 $1 \sim n$ 的 n ($1 \leq n \leq 300$)堆石子排成一行, 每堆石子有一定的质量, 质量都是不超过1000的整数. 现将这 n 堆石子合并为一堆, 每次只能合并相邻两堆, 合并的代价为两堆石子质量之和, 合并后原来与这两堆石子相邻的石子与新堆相邻. 合并时因顺序不同, 合并的总代价不同. 求最小代价.

思路

不同的合并方法有 $(n - 1)!$ 种, 暴搜最大 $299!$, 会TLE.

$dp[i][j]$ 表示将区间 $[i, j]$ 合并成一堆的最小代价. 初始条件 $dp[i][i] = 0$ ($1 \leq i \leq n$), 即只包含一个元素的区间无需合并. 最终答案 $ans = dp[1][n]$.

为得到 $[i, j]$ 的一堆, 可将 $[i, k]$ 和 $[k + 1, j]$ 合并, 而两者独立, 故应分别取最小. 状态转移方程
 $dp[i][j] = \min(dp[i][k] + dp[k + 1][j] + pre[j] - pre[i - 1])$, 其中 $pre[]$ 为前缀和. 时间复杂度 $O(n^3)$, 最坏的情况 300^3 , 是 $1e7$ 的数量级, 可过.

代码

```
1 void solve() {
2     int n; cin >> n;
3     vector<int> pre(n + 1); // a[]的前缀和
4     for (int i = 1; i <= n; i++) {
5         cin >> pre[i];
6         pre[i] += pre[i - 1];
7     }
8
9     // dp[l][r]表示合并区间[l, r]所需的最小代价
10    vector<vector<int>> dp(n + 1, vector<int>(n + 1));
11    for (int len = 2; len <= n; len++) {
12        for (int l = 1; l + len - 1 <= n; l++) {
13            int r = l + len - 1;
14            dp[l][r] = INF;
15            for (int i = l; i < r; i++) // 枚举分界点
16                dp[l][r] = min(dp[l][r], dp[l][i] + dp[i + 1][r] + (pre[r] - pre[l - 1]));
17        }
18    }
19    cout << dp[1][n] << endl;
20 }
21
22 int main() {
23     solve();
24 }
```

10.3.2 环形石子合并

题意

将编号 $1 \sim n$ 的 n ($1 \leq n \leq 200$)堆石子环形摆放, 其中第 i ($1 \leq i \leq n$)堆石子的质量为 a_i ($1 \leq i \leq n$). 现要将石子有序地合并为一堆, 规定每次只能将相邻的两堆合并成新的一堆, 该次合并的得分为新堆的石子数. 分别求将 n 堆石子合并为1堆的最小得分和最大得分.

思路

朴素做法: 将一堆石子看成一个点, 合并两堆石子即在两点间连一条边. 因连通 n 个点需 $(n - 1)$ 条边, 故最后环会有一个缺口, 可枚举缺口所在位置后将环状展开成链状, 再用**10.3.1**的方法解决. 链状区间DP的时间复杂度为 $O(n^3)$, 枚举缺口的时间复杂度为 $O(n)$, 总时间复杂度 $O(n^4)$, 会TLE.

注意到环上的区间DP本质是 n 条起点不同的、长度为 n 的链上的区间DP, 可将原长度为 n 的环从起点展开成链, 再倍增得到长度为 $2n$ 的链. 与**10.3.1**相同, 先预处理出所有 $dp[i][j]$, 再枚举所有长度为 n 的区间 $dp[i][i + n - 1]$ 并求最值, 时间复杂度 $O((2n)^3) = O(8n^3)$.

代码

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(2 * n + 1);
4     for (int i = 1; i <= n; i++) {
5         cin >> a[i];
6         a[i + n] = a[i];
7     }
8
9     vector<int> pre(2 * n + 2); // a[]的前缀和
10    for (int i = 1; i <= 2 * n; i++)
11        pre[i] = pre[i - 1] + a[i];
12
13    // dp1[l][r]表示合并区间[l, r]的石子的最小得分
14    vector<vector<int>> dp1(2 * n + 1, vector<int>(2 * n + 1, INF)); // 初始条件
15    // dp2[l][r]表示合并区间[l, r]的石子的最大得分
16    vector<vector<int>> dp2(2 * n + 1, vector<int>(2 * n + 1, -INF)); // 初始条件
17    for (int len = 1; len <= n; len++) { // 注意len从1开始枚举
18        for (int l = 1; l + len - 1 <= 2 * n; l++) {
19            int r = l + len - 1;
20
21            if (l == r) dp1[l][l] = dp2[l][l] = 0; // 初始条件
22            else {
23                for (int i = l; i < r; i++) { // 枚举分界点
24                    dp1[l][r] = min(dp1[l][r], dp1[l][i] + dp1[i + 1][r] + (pre[r] - pre[l - 1]));
25                    dp2[l][r] = max(dp2[l][r], dp2[l][i] + dp2[i + 1][r] + (pre[r] - pre[l - 1]));
26                }
27            }
28        }
29    }
30
31    int ans1 = INF, ans2 = -INF;
32    for (int i = 1; i <= n; i++) { // 枚举起点不同的长度为n的区间
33        ans1 = min(ans1, dp1[i][i + n - 1]);
34        ans2 = max(ans2, dp2[i][i + n - 1]);
35    }
36    cout << ans1 << endl;
37    cout << ans2 << endl;
38 }
39
40 int main() {
41     solve();
42 }

```

10.3.3 能量项链

题意

有一串包含 n 颗能量珠的能量项链, 能量珠是一颗有头标记和尾标记的珠子, 这些标记对应着某个正整数, 且相邻的两颗珠子中前一颗的尾标记等于后一颗的头标记. 设前一颗能量珠的头标记为 m , 尾标记为 r , 后一颗的头标记为 r , 尾标记为 n , 则聚合这两颗珠子将释放 $m \times r \times n$ 的能量, 新产生的珠子头标记为 m , 尾标记为 n . 需要能量时, 每次聚合相邻两颗珠子, 直至项链只剩一颗珠子.

输入第一行包含一个正整数 n ($4 \leq n \leq 100$), 表示项链的珠子数. 第二行包含 n 个用空格分隔的正整数(≤ 1000), 其中第 i 个数表示第 i 颗珠子的头标记, $i < N$ 时, 第 i 颗珠子的尾标记等于第 $(i + 1)$ 颗珠子的头标记; 第 n 颗珠子的尾标记等于第1颗珠子的头标记. 珠子的顺序的确定方法: 将项链放在桌子上, 不出现交叉, 随意指定一颗珠子, 按顺时针方向确定其他珠子的顺序.

输出一个正整数 E ($1 \leq E \leq 2.1e9$) 表示聚合释放能量的最大值.

思路

考虑链状的情况. $dp[l][r]$ 表示将区间 $[l, r]$ 的珠子聚合释放的最大能量. 注意到样例的项链 $(2, 3), (3, 5), (5, 10), (10, 2)$ 由序列 $2, 3, 5, 10, 2$ 唯一确定, 则可记聚合该项链释放的最大能量为 $dp[1][5]$. 区间 $[l, r]$ 的分界线依次为 $l + 1, l + 2, \dots, r - 1$. 以要聚合的两堆珠子的分界线分类, 设分界线为 k , 则每个方案可分为三部分: 合并 $[l, k]$, 合并 $[k, r]$, 再将两者合并. 注意到三者独立, 要让释放的能量最大, 需三者都取得最大值, 故状态转移方程 $dp[l][r] = \max_{l+1 \leq k \leq r-1} \{dp[l][k] + dp[k][r] + w[l] \cdot w[k] \cdot w[r]\}$.

考虑环状的情况. 将 n 颗珠子组成的项链展开并倍增为一条长度为 $2n$ 的链, 枚举长度为 $(n + 1)$ 的区间, 每种情况的最后一步是合并一个长度为3的区间.

代码

```
1 void solve() {
2     int n; cin >> n;
3     vector<int> a(2 * n + 1);
4     for (int i = 1; i <= n; i++) {
5         cin >> a[i];
6         a[i + n] = a[i];
7     }
8
9     vector<vector<int>> dp(2 * n + 1, vector<int>(2 * n + 1)); // 初始条件
10    for (int len = 3; len <= n + 1; len++) { // 注意len从3枚举到(n + 1)
11        for (int l = 1; l + len - 1 <= 2 * n; l++) {
12            int r = l + len - 1;
13            for (int i = l + 1; i <= r - 1; i++) // 枚举分界点
14                dp[l][r] = max(dp[l][r], dp[l][i] + dp[i][r] + a[l] * a[i] * a[r]);
15        }
16    }
17
18    int ans = 0;
19    for (int i = 1; i <= n; i++) // 枚举起点不同的长度为n的区间
20        ans = max(ans, dp[i][i + n]);
21    cout << ans << endl;
22 }
23
24 int main() {
25     solve();
26 }
```

10.3.4 加分二叉树

题意

设一颗 n 个节点的二叉树 $tree$ 的中序遍历为 $(1, 2, \dots, n)$, 其中数字 $1, 2, \dots, n$ 为节点编号. 每个节点有一个正整数分数, 节点 i ($1 \leq i \leq n$)的分数为 d_i . $tree$ 及其每个子树都有一个加分, 任一子树 $subtree$ (包含 $tree$ 本身)的加分 = $subtree$ 的左子树加分 $\times$ $subtree$ 的右子树加分 + $subtree$ 的根节点的分数. 若某子树为空, 规定其加分为1. 规定叶子节点的加分是其本身的分数, 不考虑其空子树.

输入第一行包含一个整数 n ($1 < n < 30$), 表示二叉树 $tree$ 的节点个数. 第二行包含 n 个用空格分隔的不超过100的正整数, 表示每个节点的分数.

求一棵中序遍历为 $(1, 2, \dots, n)$ 且加分最高的二叉树 $tree$. 第一行输出一个整数, 表示 $tree$ 的最高加分(结果不超过int). 第二行输出 n 个用空格分隔的整数, 表示该树的前序遍历. 若有多组解, 输出字典序最小的方案.

思路

$dp[l][r]$ 表示中序遍历为 $[l, r]$ 的二叉树的最大分数. 中序遍历 $[l, r]$ 中根节点可能在 $l, l+1, \dots, r-1, r$, 根据根节点位置不同分类. 设根节点为 k , 则二叉树分为三个部分: 根节点 k 、其左子树 $[l, k-1]$ 、其右子树 $[k+1, r]$, 三者相互独立, 为使总分最大, 应分别取最大值, 故状态转移方程 $dp[l][r] = \max_{l \leq k \leq r} \{dp[l][k-1] + w[k] + dp[k+1][r]\}$.

每个方案对应 $[l, r]$ 中的一个子集, $root[]$ 记录中序遍历的区间 $[l, r]$ 的根节点. 进行前序遍历时, 令 $R = root[1][n]$, 则其左子树为 $[1, R-1]$, 右子树为 $[R+1, n]$, 先输出根节点, 再递归遍历左右子树即可.

因要输出字典序最小的方案, 而前序遍历先输出根节点, 则对每个 $dp[l][r]$, 需求得最靠左的最大值.

代码

```
1  const int MAXN = 30;
2  int n;    // 节点数
3  int w[MAXN]; // 每个节点的分数
4  int dp[MAXN][MAXN]; // dp[i][j]表示所有中序遍历为[i,j]的二叉树的最大分数
5  int root[MAXN][MAXN]; // root[i][j]表示区间[i,j]的根节点
6
7  void dfs(int l, int r) { // 前序遍历
8      if (l > r) return; // 没有数
9
10     int v = root[l][r]; // 根节点
11     cout << v << ' ';
12
13     dfs(l, v - 1), dfs(v + 1, r); // 递归输出左右子树
14 }
15
16 int main() {
17     for (int i = 1; i <= n; i++) cin >> w[i];
18
19     for (int len = 1; len <= n; len++) {
20         for (int l = 1; l + len - 1 <= n; l++) {
21             int r = l + len - 1;
22
23             if (len == 1) { // 叶子节点
24                 dp[l][r] = w[l];
25                 root[l][r] = l; // 根节点是自己
26             }
27             else { // 树中结点
28                 for (int k = l; k <= r; k++) {
29                     int left = k == l ? 1 : dp[l][k - 1];
30                     int right = k == r ? 1 : dp[k + 1][r];
31                     int score = left * right + w[k];
32
33                     if (dp[l][r] < score) { // 分数高于根节点时才更新
34                         dp[l][r] = score;
35                         root[l][r] = k;
36                     }
37                 }
38             }
39         }
40     }
41 }
```



```

37     }
38     }
39     }
40 }
41 cout << dp[1][n] << endl;
42
43 dfs(1, n); // 输出先序遍历
44 }

```

10.3.5 凸多边形的划分

题意

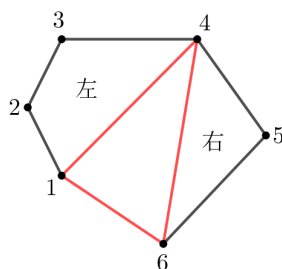
给定一个有 n ($3 \leq n \leq 50$)个顶点的凸多边形,顶点编号 $1 \sim n$,每个顶点的权值都是一个不超过 $1e9$ 正整数.将该多边形划分为 $(n - 2)$ 个互不相交的三角形,对每个三角形,其三个顶点权值之积称为该三角形的分数.求所有三角形的分数之和的最小值.

输入第一行包含一个整数 n ,表示顶点数,第二行包含 n 个整数,依次为 $1 \sim n$ 号节点的权值.

输出一个整数,表示所有三角形的分数之和的最小值.

思路

记顶点 i 和 j 的连边为 (i, j) . $dp[l][r]$ 表示将多边形 $(l, l + 1), (l + 1, l + 2), \dots, (r - 1, r), (r, l)$ 划分为三角形的分数最小值.



如上图,以 $n = 6$ 时为例.注意到中间的三角形将多边形分为三个部分,因划分的三角形间互不相交,则三个部分互相独立,要使得所有三角形的分数之和最小,应让左、右两部分都取最小值,易得 $dp[1][6] = dp[1][4] + dp[4][6] + w[1] * w[4] * w[6]$.

以边 (r, l) 属于哪个三角形分类,设边 (r, l) 所属三角形的另一顶点为 k ,则

$$dp[l][r] = \min\{dp[l][k] + dp[k][r] + w[l] * w[k] * w[r]\}.$$

一个三角形的三个顶点之积最大 $1e27$,最多48个三角形,最终答案不超过30位.

代码

```

1  const int MAXN = 55, MAXM = 35; // 顶点数最大值、答案最大位数
2  int n; // 顶点数
3  int w[MAXN]; // 顶点的权值
4  // dp[l][r][i]表示起点为l、终点为r的多边形分割为三角形的最小分数,最后一维存放分数的各数位
5  ll dp[MAXN][MAXN][MAXM];
6
7  void add(ll a[], ll b[]) { // 高精加,结果放在a[]中
8      static ll c[MAXM]; // 防止每次重新分配内存
9      memset(c, 0, so(c));
10     for (int i = 0, t = 0; i < MAXM; i++) {
11         t += a[i] + b[i];
12         c[i] = t % 10;
13         t /= 10;
14     }
15     memcpy(a, c, so(c)); // 把c中的答案复制到a中
16 }
17

```

```

18 void mul(ll a[], ll b) { // 高精乘低精
19     static ll c[MAXM];
20     memset(c, 0, so(c));
21     ll t = 0;
22     for (int i = 0; i < MAXM; i++) {
23         t += a[i] * b;
24         c[i] = t % 10;
25         t /= 10;
26     }
27     memcpy(a, c, so(c)); // 把c中的答案复制到a中
28 }
29
30 int cmp(ll a[], ll b[]) { // 判断两个大数的大小关系
31     for (int i = MAXM - 1; i >= 0; i--) {
32         if (a[i] > b[i]) return 1;
33         else if (a[i] < b[i]) return -1;
34     }
35     return 0;
36 }
37
38 void print(ll a[]) { // 输出一个大数
39     int k = MAXM - 1;
40     while (k && !a[k]) k--; // 去除前导零
41     while (k >= 0) cout << a[k--];
42     cout << endl;
43 }
44
45 int main() {
46     for (int i = 1; i <= n; i++) cin >> w[i];
47
48     ll tmp[MAXM];
49     for (int len = 3; len <= n; len++) { // 枚举多边形的顶点数
50         for (int l = 1; l + len - 1 <= n; l++) {
51             int r = l + len - 1;
52             dp[l][r][MAXM - 1] = 1; // 100...0,即初始化为INF
53             for (int k = l + 1; k < r; k++) {
54                 memset(tmp, 0, so(tmp));
55
56                 // tmp[]=dp[l][k]+dp[k][r]+w[l]*w[k]*w[r]
57                 tmp[0] = w[l];
58                 mul(tmp, w[k]);
59                 mul(tmp, w[r]);
60                 add(tmp, dp[l][k]);
61                 add(tmp, dp[k][r]);
62                 if (cmp(dp[l][r], tmp) > 0) memcpy(dp[l][r], tmp, so(tmp));
63             }
64         }
65     }
66     print(dp[1][n]);
67 }

```

10.3.6 棋盘分割

题意

将一个 8×8 的棋盘沿网格进行如下分割:将原棋盘割下一块矩形棋盘使得剩下的部分是矩形,再将剩下的部分如此分割,分割 $(n - 1)$ 次后,连通最后剩下的矩形棋盘共 n 块矩形棋盘.原棋盘的每格有一个分值,一块矩形棋盘的总分为其所含的格子的分值之和.

现将给定的棋盘按上述规则分割为 n ($1 < n < 15$)块矩形棋盘,使得各矩形棋盘总分的均方差最小,并输出均方差的最小值,四舍五

入精确到小数点后三位.均方差 $\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$,其中 $\bar{x}$ 为平均值.棋盘用 8×8 的矩阵描述,其中每个元素都是小于100的非负整数,表示棋盘上相应格子的分值.

思路

$$\sigma^2 = \frac{1}{n} \left[\sum_{i=1}^n (x_i^2 - 2x_i\bar{x} + \bar{x}^2) \right] = \frac{1}{n} \left(\sum_{i=1}^n x_i^2 - \bar{x} \sum_{i=1}^n (2x_i) + n\bar{x}^2 \right) = \frac{1}{n} \left(\sum_{i=1}^n x_i^2 - n\bar{x}^2 \right) = \frac{\sum_{i=1}^n x_i^2}{n} - \bar{x}^2$$

要使 σ 最小,只需 $\sum_{i=1}^n x_i^2$ 最小.

$dp[x1][y1][x2][y2][k]$ 表示将以 (x_1, y_1) 为左上角、以 (x_2, y_2) 为右下角的矩形切分成 k 部分的 σ 的最小值.按第一刀的切法不同分为四类:①横切,取上面;②横切,取下面;③纵切,取左边;④纵切,取右边.

本题若dp用迭代的方式实现则可能出现五六层循环,故用记搜.

代码

```
1  const int MAXN = 15, MAXM = 9; // 分割数最大值、棋盘大小最大值
2  int n, m = 8; // 分割数、棋盘大小
3  int pre[MAXM][MAXM]; // 二维前缀和
4  // dp[x1][y1][x2][y2][k]表示以(x1,y1)为左上角、以(x2,y2)为右下角的矩形分割为k份的最小均方差
5  double dp[MAXM][MAXM][MAXM][MAXM][MAXN];
6  double aver; // 平均值
7
8  int get_sum(int x1, int y1, int x2, int y2) { // 求子矩阵的和
9      return pre[x2][y2] - pre[x2][y1 - 1] - pre[x1 - 1][y2] + pre[x1 - 1][y1 - 1];
10 }
11
12 double get_sigma(int x1, int y1, int x2, int y2) { // 求子矩阵的均方差
13     double sum = get_sum(x1, y1, x2, y2) - aver;
14     return sum * sum / n;
15 }
16
17 double solve(int x1, int y1, int x2, int y2, int k) { // 记搜
18     double& v = dp[x1][y1][x2][y2][k];
19     if (v >= 0) return v; // 搜过
20     if (k == 1) return v = get_sigma(x1, y1, x2, y2); // 只有一块矩形
21     v = INF;
22
23     for (int i = x1; i < x2; i++) { // 枚举横切
24         v = min(v, get_sigma(x1, y1, i, y2) + solve(i + 1, y1, x2, y2, k - 1));
25         v = min(v, get_sigma(i + 1, y1, x2, y2) + solve(x1, y1, i, y2, k - 1));
26     }
27
28     for (int i = y1; i < y2; i++) { // 枚举纵切
29         v = min(v, get_sigma(x1, y1, x2, i) + solve(x1, i + 1, x2, y2, k - 1));
30         v = min(v, get_sigma(x1, i + 1, x2, y2) + solve(x1, y1, x2, i, k - 1));
31     }
32     return v;
33 }
34
35 int main() {
36     memset(dp, -1, so(dp)); // 每个字节都为-1即double的-INF
37
38     for (int i = 1; i <= m; i++) {
```

```

39     for (int j = 1; j <= m; j++) {
40         cin >> pre[i][j];
41         pre[i][j] += pre[i - 1][j] + pre[i][j - 1] - pre[i - 1][j - 1]; // 预处理前缀和
42     }
43 }
44 aver = (double)pre[m][m] / n;
45
46 cout << fixed << setprecision(3) << sqrt(solve(1, 1, 8, 8, n));
47 }

```

10.3.7 Doin' Time

题意

有一长度为 n ($1 \leq n \leq 300$)的序列 $a_1, \dots, a_n$ ($1 \leq a_i \leq 1e6$),每次操作可选一个下标 x ,合并 a_x 和 a_{x+1} 为 $a_x \cdot a_{x+1} \bmod 1000003$,并得到 $(a_x - a_{x+1})^2$ 的分数.求合并至只剩下一个元素时的最大得分.

思路

$dp[l][r]$ 表示合并区间 $[l, r]$ 中的数的最大得分.

对区间 $[l, r]$,枚举分界点 k ($l \leq k < r$),

$$\text{状态转移方程: } dp[l][r] = \max_{l \leq k < r} dp[l][k] + dp[k+1][r] + \left(\prod_{i=k+1}^r a[i] - \prod_{i=l}^k a[i] \right)^2.$$

暴力转移时间复杂度 $O(n^4)$,显然可用前缀积优化到 $O(n^3)$.

注意预处理前缀积的逆元.

代码

```

1  const int MAXN = 305;
2  const int MOD = 1e6 + 3;
3  int n;
4  int pre[MAXN]; // 原数组的前缀积
5  int inv[MAXN]; // inva[i]为pre[i]的逆元
6  ll dp[MAXN][MAXN]; // dp[l][r]表示合并区间[l, r]中的数的最大得分
7
8  void solve() {
9      cin >> n;
10
11     pre[0] = inv[0] = 1;
12     for (int i = 1; i <= n; i++) {
13         cin >> pre[i];
14         pre[i] = (ll)pre[i] * pre[i - 1] % MOD;
15         inv[i] = qpow(pre[i], MOD - 2, MOD);
16     }
17
18     for (int len = 2; len <= n; len++) {
19         for (int l = 1; l + len - 1 <= n; l++) {
20             int r = l + len - 1;
21             for (int k = l; k < r; k++) { // 枚举分界点
22                 dp[l][r] = max(dp[l][r], dp[l][k] + dp[k + 1][r]
23                     + ((ll)pre[r] * inv[k] % MOD - (ll)pre[k] * inv[l - 1] % MOD) * ((ll)pre[r] * inv[k] % MOD
24                     - (ll)pre[k] * inv[l - 1] % MOD));
25             }
26         }
27     }
28     cout << dp[1][n];
29 }

```

```

29
30 int main() {
31     solve();
32 }

```

10.3.8 Flood Fill

原题指路: <https://codeforces.com/problemset/problem/1114/D>

题意 (2 s)

编号 $1 \sim n$ 的 n ($1 \leq n \leq 5000$) 个砖块排成一行, i ($1 \leq i \leq n$) 号砖块的颜色为 c_i ($1 \leq c_i \leq 5000$). 在所有操作前, 可选择一砖块作为起始砖块. 每次操作任选一种颜色, 将起始砖块所在的连通块的所有砖块变为该选定的颜色. 问至少需多少次操作可使得所有砖块同色.

思路

因每次操作改变一个连通块的颜色, 则初始时同色的砖块最后也同色, 进而可将初始时同色的连通块视为一个砖块. 将同色的一段缩点, 则初始时所有相邻的砖块异色.

$dp[l][r]$ 表示在区间 $[l, r]$ 中选一个起始砖块, 将 $[l, r]$ 中的砖块变为同色所需的最小操作次数. 初始条件, 最终答案 $ans = dp[1][n]$.

按 $c[l]$ 与 $c[r]$ 是否相同分类:

(1) $c[l] \neq c[r]$ 时, 注意到将区间 $[l, r]$ 变为同色的操作是选择一个连通块并向两边扩展, 则最后一个改变颜色的砖块要么是砖块 l , 要么是砖块 r , 不会是同时改变砖块 l 和砖块 r , 否则与 $c[l] \neq c[r]$ 矛盾.

若最后一个改变颜色的砖块为 l , 则最小操作次数为将 $[l+1, r]$ 变为同色所需的最小操作次数 + 1, 即 $(dp[l+1][r] + 1)$.

同理最后一个改变颜色的砖块为 r 时的最小操作次数为 $(dp[l][r-1] + 1)$.

故状态转移方程 $dp[l][r] = \min\{dp[l+1][r], dp[l][r-1]\} + 1$.

(2) $c[l] = c[r]$ 时, 由对称性, 不妨设最后一个改变颜色的砖块为 r .

将区间 $[l, r-1]$ 变为同色的操作有如下两种情况:

① 最后一个改变颜色的砖块为 $(r-1)$, 则 $c[l] \neq c[r-1]$, 否则有 $c[r-1] = c[r]$, 与初始时相邻的砖块异色矛盾. 此时最小操作次数 $\geq dp[l][r-1] + 1$, 此处为 $\geq$ 是因为 $dp[l][r-1]$ 表示将区间 $[l, r-1]$ 变为同色所需的最小操作次数, 而最后一个改变颜色的砖块为 $(r-1)$ 的方案只是其中的一类.

② 最后一个改变颜色的砖块为 l , 则先将区间 $[l+1, r-1]$ 先变为色 $c[l]$, 再将砖块 l 和砖块 r 连同区间 $[l+1, r-1]$ 变为与区间 $[l, r]$ 的前驱和后继的砖块相同的颜色, 则最小操作次数为 $(dp[l+1][r-1] + 1)$.

注意到 $dp[l+1][r-1] \leq dp[l][r-1]$, 故最小操作次数为 $(dp[l+1][r-1] + 1)$.

综上, 状态转移方程 $dp[l][r] = \begin{cases} \min\{dp[l+1][r], dp[l][r-1]\} + 1, & c[l] \neq c[r] \\ dp[l+1][r-1] + 1, & c[l] = c[r] \end{cases}$.

代码

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5     n = unique(a.begin() + 1, a.end()) - a.begin() - 1; // 将同色的一段缩点
6
7     // dp[l][r] 表示在区间 [l, r] 中选一个起始砖块, 将 [l, r] 中的砖块变为同色所需的最小操作次数
8     vector<vector<int>> dp(n + 1, vector<int>(n + 1));
9     for (int len = 2; len <= n; len++) {
10         for (int l = 1; l + len - 1 <= n; l++) {
11             int r = l + len - 1;

```

```

12         if (a[l] != a[r]) dp[l][r] = min(dp[l + 1][r], dp[l][r - 1]) + 1;
13         else dp[l][r] = dp[l + 1][r - 1] + 1;
14     }
15 }
16 }
17 cout << dp[1][n] << endl;
18 }
19
20 int main() {
21     solve();
22 }

```

10.3.9 inimum Triangulation

原题指路: <https://codeforces.com/problemset/problem/1140/D>

题意 (2 s)

一个多边形的三角剖分指一组三角形, 使得每个三角形的顶点是多边形的顶点, 没有不相交的三角形, 且所有三角形的面积之和等于多边形的面积. 一个三角形的权值定义为其三个顶点的编号之积, 一个三角剖分的权值定义为其包含的所有三角形的权值之和.

给定一个正 n ($3 \leq n \leq 500$) 边形, 顶点逆时针编号 $1 \sim n$. 求该多边形的三角剖分的权值的最小值.

思路

$dp[l][r]$ 表示由编号 $l \sim r$ 的顶点构成的多边形的三角剖分的最小权值, 初始条件 $dp[i][j] = \begin{cases} 0, j = i + 1 \\ INF, j \neq i + 1 \end{cases}$, 最终答案 $ans = dp[1][n]$.

显然边 (l, r) 在某个三角形中, 按该三角形的第三个顶点 k 分类, 状态转移方程:

$$dp[l][r] = \min_{l+1 \leq k \leq r-1} (dp[l][k] + dp[k][r] + l \cdot r \cdot k).$$

总时间复杂度 $O(n^3)$.

代码

```

1  const ll INFF = 0x3f3f3f3f3f3f3f3f;
2
3  void solve() {
4      int n; cin >> n;
5
6      vector<vector<ll>> dp(n + 5, vector<ll>(n + 5, INFF));
7      for (int i = 1; i <= n; i++) dp[i][i + 1] = 0; // 初始条件
8
9      for (int len = 3; len <= n; len++) {
10         for (int l = 1; l + len - 1 <= n; l++) {
11             int r = l + len - 1;
12             for (int k = l + 1; k <= r - 1; k++)
13                 dp[l][r] = min(dp[l][r], dp[l][k] + dp[k][r] + l * r * k);
14         }
15     }
16     cout << dp[1][n] << endl;
17 }
18
19 int main() {
20     solve();
21 }

```

思路II

$dp[i][j]$ 表示由编号 $1 \sim i$ 和编号 $j \sim n$ 的顶点构成的多边形的三角剖分的最小权值, 初始条件

$$dp[1][j] = \begin{cases} 0, & j = 2, n \\ INF, & j \neq 2, n \end{cases}, dp[i][j] = INF \quad (2 \leq i \leq n, 1 \leq j \leq n), \text{ 最终答案 } ans = \min_{i+1=j-1} dp[i+1][j].$$

从两边往中间递推, 升序枚举 $i: 1 \rightarrow n$, 降序枚举 $j: n \rightarrow (i+1)$, 状态转移方程

$$\begin{cases} dp[i+1][j] = \min\{dp[i+1][j], dp[i][j] + i \cdot j \cdot (i+1)\} \\ dp[i][j-1] = \min\{dp[i][j-1], dp[i][j] + i \cdot j \cdot (j-1)\} \end{cases}.$$

总时间复杂度 $O(n^2)$.

代码II

```
1  const ll INFF = 0x3f3f3f3f3f3f3f3f;
2
3  void solve() {
4      int n; cin >> n;
5
6      vector<vector<ll>> dp(n + 5, vector<ll>(n + 5, INFF));
7      dp[1][2] = dp[1][n] = 0; // 初始条件
8
9      ll ans = INFF;
10     for (int i = 1; i <= n; i++) {
11         for (int j = n; j >= i + 1; j--) {
12             dp[i + 1][j] = min(dp[i + 1][j], dp[i][j] + i * j * (i + 1));
13             dp[i][j - 1] = min(dp[i][j - 1], dp[i][j] + i * j * (j - 1));
14
15             if (i + 1 == j - 1)
16                 ans = min(ans, dp[i + 1][j]);
17         }
18     }
19     cout << ans << endl;
20 }
21
22 int main() {
23     solve();
24 }
```

思路III

[定理] 取节点 1 到其余所有节点的连线作为分割线最优, 即 $\sum_{i=2}^{n-1} i \cdot (i+1)$.

[证] 设包含边 $(1, n)$ 的三角形的三个顶点分别为 $1, n, x$, 记作 $\triangle(1, n, x)$.

按 x 的值分类:

(1) $x = n - 1$ 时, 删除该三角形, 递归地考察 $(n - 1)$ -边形.

(2) $1 < x < n - 1$ 时, 考察 $\triangle(n, x, y)$, 其中 $y \in [x + 1, n - 1]$.

将三角形 $\triangle(1, n, x)$ 和 $\triangle(n, x, y)$ 替换为 $\triangle(1, n, y)$ 和 $\triangle(1, x, y)$,

因 $nx > xy, nxy > ny$, 则 $nx + nxy > xy + ny$, 即答案更优.

将 $\triangle(1, n, x)$ 替换为 $\triangle(1, n, y)$, 其中 $y > x$. 重复上述操作, 直至调整至 $x = n - 1$, 故证.

总时间复杂度 $O(n)$.

代码III

```

1  void solve() {
2      int n; cin >> n;
3
4      ll ans = 0;
5      for (int i = 2; i <= n - 1; i++)
6          ans += i * (i + 1);
7      cout << ans << endl;
8  }
9
10 int main() {
11     solve();
12 }
```

思路IV

[引理] $n(n-1) = \frac{1}{3}[(n-1)n(n+1) - (n-2)(n-1)n]$.

[证] 注意到 $1 \times 2 = \frac{1}{3}(1 \times 2 \times 3 - 0 \times 1 \times 2)$, $2 \times 3 = \frac{1}{3}(2 \times 3 \times 4 - 1 \times 2 \times 3)$, 数归即证.

$$\begin{aligned}
 ans &= \sum_{i=2}^{n-1} i \cdot (i+1) = \sum_{i=1}^{n-1} i \cdot (i+1) - 2 \\
 &= \frac{1}{3}[-0 \times 1 \times 2 + 1 \times 2 \times 3 - 1 \times 2 \times 3 + \cdots + (n-1)n(n+1)] - 2 \\
 &= \frac{1}{3}(n-1)n(n+1) - 2.
 \end{aligned}$$

总时间复杂度 $O(1)$.

代码IV

```

1  void solve() {
2      int n; cin >> n;
3      cout << (n - 1) * n * (n + 1) / 3 - 2 << endl;
4  }
5
6  int main() {
7      solve();
8  }
```

10.4 计数DP

10.4.1 整数划分

题意

正整数 n 可表示为若干个正整数之和 $n = n_1 + n_2 + \cdots + n_k$ ($k \geq 1; n_1 \geq n_2 \geq \cdots \geq n_k$), 称这样的一种表示为 n 的一种划分. 给定正整数 n ($1 \leq n \leq 1000$), 输出 n 有几种划分, 答案对 $1e9 + 7$ 取模.

思路1:转化为完全背包

有 n 个体积分别为 $1, 2, \dots, n$ 的物品和一个容量为 n 的背包,求恰好装满背包的方案数,每个数可用无限次,即完全背包.

$dp[i][j]$ 表示从前 i 个物品中选,体积恰好为 j 的方案数.

状态转移方程 $dp[i][j] = dp[i-1][j] + dp[i-1][j-i] + dp[i-1][j-2*i] + \dots + dp[i-1][j-s*i] \dots \textcircled{1}$.状态转移 $O(n^2)$,状态 $O(n)$,总复杂度 $O(n^3)$.事实上总复杂度 $< O(n^3)$: $s=1$ 时为 $\frac{1000}{1}$, $s=2$ 时为 $\frac{1000}{2}$, $\dots$, $s=n$ 时为 $\frac{1000}{n}$,由调和级数知:总复杂度 $O(n^2 \log n)$.

注意到 $dp[i][j-i] = dp[i-1][j-1] + dp[i-1][j-2*i] + \dots + dp[i-1][j-s*i]$,则 $\textcircled{1}$ 可写作 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$.

类似于完全背包,压成一维,初始条件 $dp[0] = 1, dp[i] = 0 (i \neq 0)$.

代码1

```
1  const int MAXN = 1005;
2  const int MOD = 1e9 + 7;
3  int dp[MAXN];
4
5  int main() {
6      dp[0] = 1; // 初始条件
7      for (int i = 1; i <= n; i++) {
8          for (int j = i; j <= n; j++)
9              dp[j] = (dp[j] + dp[j - i]) % MOD;
10     }
11     cout << dp[n];
12 }
```

思路2

$dp[i][j]$ 表示和为 i 的 j 个数的方案数.

按划分中的数的最小数是否为1分类:

①最小数是1:每个划分中都有1,去掉后即和为 $(i-1)$ 的 $(j-1)$ 个数的方案数,即 $dp[i-1][j-1]$.

②最小数 > 1 :每个划分中的最小数都 > 1 ,则全部减1后仍为正整数,此时变为和为 $(i-j)$ 的 j 个数的方案数,即 $dp[i-j][j]$.

状态转移方程 $dp[i][j] = dp[i-1][j-1] + dp[i-j][j]$,最终答案 $ans = \sum_{i=0}^n dp[n][i]$.

代码2

```
1  const int MAXN = 1005;
2  const int MOD = 1e9 + 7;
3  int dp[MAXN][MAXN];
4
5  int main() {
6      dp[0][0] = 1; // 初始条件
7      for (int i = 1; i <= n; i++) {
8          for (int j = 1; j <= i; j++)
9              dp[i][j] = (dp[i-1][j-1] + dp[i-j][j]) % MOD;
10     }
11     int ans = 0;
12     for (int i = 1; i <= n; i++) ans = (ans + dp[n][i]) % MOD;
13     cout << ans;
14 }
```

10.5 数位DP

10.5.1 计数问题

题意

给定整数 a, b ($1 \leq a, b \leq 1e8$), 输出 a 和 b 间所有数字中 $0 \sim 9$ 出现的次数, 用空格分隔.

思路

最坏的情况有 $1e8$ 个数, 每个数枚举8位, 暴搜会T.

考虑实现一个 $count(n, x)$ 的函数用于计算 $1 \sim n$ 中 x 出现的次数, 则最终答案 $ans = count(b, x) - count(a - 1, x)$.

考察 $n = \overline{abcdefg}, x = 1$ 时, 数 $1 \sim n$ 中1在第4位上出现的次数, 即 $[1, n]$ 中形如 $\overline{xxx1yyy}$ 的数的个数:

① $\overline{xxx} \in [000, \overline{abc} - 1]$ 时, $\overline{yyy} \in [000, 999]$, 共 $\overline{abc} * 1000$ 个.

1) 此情况当1在首位时不存在.

2) 枚举 $x = 0$ 时, 因数字不能有前导零, 则 $\overline{xxx} \in [001, \overline{abc} - 1]$.

② $\overline{xxx} = \overline{abc}$ 时:

1) $d < 1$ 时, $\overline{abc1yyy} > \overline{abc0efg}$, 超出范围.

2) $d = 1$ 时, $\overline{yyy} \in [000, \overline{efg}]$, 共 $(\overline{efg} + 1)$ 个.

3) $d > 1$ 时, $\overline{yyy} \in [000, 999]$, 共1000个.

10个数码, 每个数码2种情况, 第二种情况算10次, 每个数有8个数位, 则最坏的情况是 $10 \times 2 \times 10 \times 8 = 1600$.

代码

```

1  int get(vector<int> num, int l, int r) { // [r,l]位构成的数
2      int res = 0;
3      for (int i = l; i >= r; i--)
4          res = res * 10 + num[i];
5
6      return res;
7  }
8
9  int pow10(int x) { // 求10^x
10     int res = 1;
11     while (x--) res *= 10;
12
13     return res;
14 }
15
16 int count(int n, int x) {
17     if (!n) return 0;
18
19     vector<int> num; // 存各个数位
20     while (n) {
21         num.push_back(n % 10);
22         n /= 10;
23     }
24
25     n = num.size();
26     int res = 0;
27     for (int i = n - 1 - !x; i >= 0; i--) { // x=0时从第二位开始枚举
28         if (i < n - 1) { // 第一种情况未必存在
29             res += get(num, n - 1, i + 1) * pow10(i);
30             if (!x) res -= pow10(i);
31         }

```

```

32
33     if (num[i] == x) res += get(num, i - 1, 0) + 1;
34     else if (num[i] > x) res += pow10(i);
35 }
36
37 return res;
38 }
39
40 int main() {
41     while (cin >> a >> b, a || b) {
42         if (a > b) swap(a, b);
43
44         for (int i = 0; i < 10; i++)
45             cout << count(b, i) - count(a - 1, i) << ' ';
46         cout << endl;
47     }
48 }

```

10.5.2 度的数量

题意

求区间 $[x, y]$ 内满足如下条件的整数的个数:该数可表示为 k 个互不相等的 b 的整数次幂之和.

第一行输入两个整数 x, y ($1 \leq x \leq y \leq 2^{31} - 1$).接下来两行分别输入一个整数 k, b ($1 \leq k \leq 20, 2 \leq b \leq 10$).

思路

满足条件的数即 b 进制表示中恰包含 k 个1的整数.

$dp[n]$ 表示 $[0, n]$ 中满足条件的数的个数.

设 $N = \overline{a_{n-1}a_{n-2}\cdots a_0}$.每个 $x \in [0, n]$ 的最高位有 $0 \sim a_{n-1} - 1$ 和 a_{n-1} 两种情况.

(1)对最高位为 $0 \sim a_{n-1} - 1$ 的情况,

①若该位非1,则后 $(n - 1)$ 位中需填 k 个1,方案数为 C_{n-1}^k .

②若该为1,则后 $(n - 1)$ 为中需填 $(k - 1)$ 个1,方案数为 C_{n-1}^{k-1} .

(2)对最高位为 a_{n-1} 的情况,同理考虑次高位的情况.

该过程可看作是一棵偏向一边的树,左边的分支通过组合数计算,右边的分支继续递归.

代码I:迭代写法

```

1  const int MAXN = 35;
2  int k, B;
3  int C[MAXN][MAXN]; // 组合数
4
5  void init() {
6      for (int i = 0; i < MAXN; i++) {
7          for (int j = 0; j <= i; j++) {
8              if (!j) C[i][j] = 1;
9              else C[i][j] = C[i - 1][j] + C[i - 1][j - 1];
10         }
11     }
12 }
13
14 int cal(int n) {
15     if (!n) return 0;

```

```

16
17 vector<int> nums;
18 while (n) nums.push_back(n % B), n /= B; // 将n表示为B进制
19
20 int res = 0;
21 int last = 0; // 已填的1的个数
22 for (int i = nums.size() - 1; i >= 0; i--) { // 从高位开始枚举
23     int x = nums[i];
24     if (x) { // 左分支
25         res += C[i][k - last]; // 当前位不填1
26         if (x > 1) { // x>1时当前位才能填1
27             if (k - last - 1 >= 0) res += C[i][k - last - 1]; // 当前位填1
28             break;
29         }
30         else {
31             last++;
32             if (last > k) break; // 已填完所有1
33         }
34     }
35
36     if (!i && last == k) res++; // 递归到最后一个右分支
37 }
38 return res;
39 }
40
41 void solve() {
42     init();
43
44     int l, r; cin >> l >> r >> k >> B;
45     cout << cal(r) - cal(l - 1) << endl;
46 }
47
48 int main() {
49     solve();
50 }

```

代码II:递归写法

```

1  const int MAXN = 35;
2  int k, B;
3  int C[MAXN][MAXN]; // 组合数
4  int nums[MAXN];
5
6  void init() {
7      for (int i = 0; i < MAXN; i++) {
8          for (int j = 0; j <= i; j++) {
9              if (!j) C[i][j] = 1;
10             else C[i][j] = C[i - 1][j] + C[i - 1][j - 1];
11         }
12     }
13 }
14
15 int dfs(int len, int last, bool lim) { // 当前枚举到n的第len位,还可填last个1,lim表示是否在上界
16     if (!len) return !last; // 枚举到n的最后一位时,还有方案当且仅当还有1可填
17     if (!lim) return C[len][last]; // 不是上界的情况直接计算
18
19     int res = 0;
20     int up = lim ? nums[len] : 9; // 本位可填的最大数码
21     for (int i = 0; i <= min(up, 1); i++) // 枚举该位填的数码,只能填0或1
22         res += dfs(len - 1, last + i, lim && i == up);
23     return res;

```

```

24 }
25
26 int cal(int n) {
27     int len = 0;
28     while (n) nums[++len] = n % B, n /= B; // 将n表示为B进制
29
30     return dfs(len, k, true);
31 }
32
33 void solve() {
34     init();
35
36     int l, r; cin >> l >> r >> k >> B;
37     cout << cal(r) - cal(l - 1) << endl;
38 }
39
40 int main() {
41     solve();
42 }

```

10.5.3 数字游戏I

题意

称从左到右的数码不降的数为不降数.求区间 $[a, b]$ 中有多少个不降数.

有多组测试数据.每组测试数据输入两个整数 a, b ($1 \leq a \leq b \leq 2^{31} - 1$).

思路

$f[n]$ 表示 $[0, n]$ 中不降数的个数.

$dp[i][j]$ 表示 i 位数中最高位为 j 的不降数的个数.按此高位填 $j, j+1, \dots, 9$ 分类.状态转移方程 $dp[i][j] = \sum_{k=j}^9 dp[i-1][k]$.

设 $N = \overline{a_{n-1}a_{n-2}\cdots a_0}$.每个 $x \in [0, n]$ 的最高位有 $0 \sim a_{n-1} - 1$ 和 a_{n-1} 两种情况.

(1)对最高位为 $k \in [0, a_{n-1} - 1]$ 的情况,方案数即 $dp[len][k]$.

(2)对最高位为 a_{n-1} 的情况,同理考虑次高位的情况.

代码I:迭代写法

```

1  const int MAXN = 15;
2  int dp[MAXN][MAXN]; // dp[i][j]表示i位数中最高位为j的不降数的个数
3
4  void init() {
5      for (int i = 0; i <= 9; i++) dp[1][i] = 1; // 初始条件
6      for (int i = 2; i < MAXN; i++) {
7          for (int j = 0; j <= 9; j++) { // 枚举高位
8              for (int k = j; k <= 9; k++) // 枚举低位
9                  dp[i][j] += dp[i-1][k];
10         }
11     }
12 }
13
14 int cal(int n) {
15     if (!n) return 1; // n=0是不降数
16
17     vector<int> nums;

```

```

18 while (n) nums.push_back(n % 10), n /= 10;
19
20 int res = 0;
21 int last = 0; // 上一位填的数
22 for (int i = nums.size() - 1; i >= 0; i--) {
23     for (int j = last; j < nums[i]; j++) res += dp[i + 1][j]; // 左分支
24
25     if (nums[i] < last) break; // 无右分支
26
27     last = nums[i];
28     if (!i) res++; // 右分支
29 }
30 return res;
31 }
32
33 void solve() {
34     init();
35
36     int l, r;
37     while (cin >> l >> r) cout << cal(r) - cal(l - 1) << endl;
38 }
39
40 int main() {
41     solve();
42 }

```

代码II:递归写法

```

1 const int MAXN = 15;
2 int dp[MAXN][MAXN]; // dp[i][j]表示i位数中最高位为j的不降数的个数
3 int nums[MAXN];
4
5 void init() {
6     for (int i = 0; i <= 9; i++) dp[1][i] = 1; // 初始条件
7     for (int i = 2; i < MAXN; i++) {
8         for (int j = 0; j <= 9; j++) { // 枚举高位
9             for (int k = j; k <= 9; k++) // 枚举低位
10                 dp[i][j] += dp[i - 1][k];
11         }
12     }
13 }
14
15 int dfs(int len, int last, bool lim) { // 当前枚举到n的第len位,上一位为last,lim表示是否在上界
16     if (!len) return 1; // n=0是不降数
17     if (!lim) dp[len + 1][last];
18
19     int res = 0;
20     int up = lim ? nums[len] : 9; // 本位可填的最大数码
21     for (int i = last; i <= up; i++) res += dfs(len - 1, i, lim && i == up);
22     return res;
23 }
24
25 int cal(int n) {
26     int len = 0;
27     while (n) nums[++len] = n % 10, n /= 10;
28     return dfs(len, 0, true);
29 }
30
31 void solve() {
32     init();
33 }

```

```

34     int l, r;
35     while (cin >> l >> r) cout << cal(r) - cal(l - 1) << endl;
36 }
37
38 int main() {
39     solve();
40 }

```

10.5.4 Windy数

题意

不含前导零且相邻两个数码之差至少为2的正整数称为Windy数.给定两整数 l, r ($1 \leq l \leq r \leq 2e9$),求区间 $[l, r]$ 中Windy数的个数.

思路

本题与10.5.3的不同之处在于本题中不能随便添加前导零而保持数的性质不变.

$dp[i][j]$ 表示 i 位数中最高位为 j 的Windy数的个数.以次高位 k 分类,状态转移方程 $dp[i][j] = \sum_{k=0}^9 dp[i-1][k] \cdot [|j-k| \geq 2]$.注意到该DP是有前导零的.

设 $N = \overline{a_{n-1}a_{n-2}\cdots a_0}$.每个 $x \in [0, n]$ 的最高位有 $0 \sim a_{n-1} - 1$ 和 a_{n-1} 两种情况.

(1)最高位为 $k \in [0, a_{n-1} - 1]$ 时,方案数即 $dp[len][k]$.

(2)最高位为 a_{n-1} 时,同理考虑次高位的情况:

①次高位为 $k \in [1, a_{n-2} - 1]$ 时,

②次高位为 a_{n-2} 时,同理考虑次次高位的情况.

本题无需考虑0是否为Windy数,因为它不会出现在 $[l, r]$ 中,且用前缀和计算时无论0是不是Windy数都会被抵消.下面代码中认为0不是Windy数.

代码

```

1  const int MAXN = 15;
2  int dp[MAXN][MAXN]; // dp[i][j]表示i位数中最高位为j的windy数的个数
3
4  void init() {
5      for (int i = 0; i <= 9; i++) dp[1][i] = 1; // 初始条件
6      for (int i = 2; i < MAXN; i++) {
7          for (int j = 0; j <= 9; j++) { // 枚举高位
8              for (int k = 0; k <= 9; k++) // 枚举低位
9                  if (abs(j - k) >= 2) dp[i][j] += dp[i-1][k];
10         }
11     }
12 }
13
14 int cal(int n) {
15     if (!n) return 0; // 0不是windy数
16
17     vector<int> nums;
18     while (n) nums.push_back(n % 10), n /= 10;
19

```

```

20  int res = 0;
21  int last = -2; // 上一位填的数
22  for (int i = nums.size() - 1; i >= 0; i--) {
23      for (int j = i == nums.size() - 1; j < nums[i]; j++) // 最高位只能从1开始枚举
24          if (abs(j - last) >= 2) res += dp[i + 1][j];
25
26      if (abs(nums[i] - last) >= 2) last = nums[i];
27      else break;
28
29      if (!i) res++;
30  }
31
32  for (int i = 1; i < nums.size(); i++) // 单独统计有前导零的数
33      for (int j = 1; j <= 9; j++) res += dp[i][j];
34  return res;
35 }
36
37 void solve() {
38     init();
39
40     int l, r; cin >> l >> r;
41     cout << cal(r) - cal(l - 1) << endl;
42 }
43
44 int main() {
45     solve();
46 }

```

10.5.5 数字游戏II

题意

称各个数码之和是 n 的倍数的数为取模数.有多组测试数据,每组测试数据给定三个整数 l, r, n ($1 \leq l, r \leq 2^{31} - 1, 1 \leq n < 100$),求区间 $[l, r]$ 中取模数的个数.

思路

$dp[i][j][k]$ 表示 i 位数中最高位为 j 且各个数码之和模 n 余 k 的数的个数.以次高位 x 分类,状态转移方程

$$dp[i][j][k] = \sum_{x=0}^9 dp[i-1][x][(k-j)\%n].$$

设 $N = \overline{a_{n-1}a_{n-2}\cdots a_0}$.每个 $x \in [0, n]$ 的最高位有 $0 \sim a_{n-1} - 1$ 和 a_{n-1} 两种情况.

(1)最高位为 $k \in [0, a_{n-1} - 1]$ 时,方案数即 $dp[len][k]$.

(2)最高位为 a_{n-1} 时,同理考虑次高位的情况.

代码I:迭代写法

```

1  const int MAXN = 15, MAXM = 105; // 余数最大为100
2  int P;
3  // dp[i][j][k]表示i位数中最高位为j且各个数码之和模n余k的数的个数
4  int dp[MAXN][MAXN][MAXM];
5
6  int get(int x) { // 求x%n
7      return (x % P + P) % P;
8  }
9
10 void init() {

```



```

11  memset(dp, 0, sizeof(dp));
12
13  for (int i = 0; i <= 9; i++) dp[1][i][get(i)]++;
14  for (int i = 2; i < MAXN; i++) { // 枚举位数
15      for (int j = 0; j <= 9; j++) { // 枚举最高位
16          for (int k = 0; k < P; k++) { // 枚举余数
17              for (int x = 0; x <= 9; x++) // 枚举次高位
18                  dp[i][j][k] += dp[i - 1][x][get(k - j)];
19          }
20      }
21  }
22 }
23
24 int cal(int n) {
25     if (!n) return 1; // 0是n的倍数
26
27     vector<int> nums;
28     while (n) nums.push_back(n % 10), n /= 10;
29
30     int res = 0;
31     int last = 0; // 前面的数码之和
32     for (int i = nums.size() - 1; i >= 0; i--) {
33         for (int j = 0; j < nums[i]; j++) // 左分支
34             res += dp[i + 1][j][get(-last)];
35
36         last += nums[i];
37         if (!i && last % P == 0) res++; // 右分支
38     }
39     return res;
40 }
41
42 void solve() {
43     int l, r;
44     while (cin >> l >> r >> P) {
45         init();
46
47         cout << cal(r) - cal(l - 1) << endl;
48     }
49 }
50
51 int main() {
52     solve();
53 }

```

代码II:递归写法

```

1  const int MAXN = 15, MAXM = 105; // 余数最大为100
2  int P;
3  // dp[i][j][k]表示i位数中各个数码之和为j且模数为P的数的个数
4  int dp[MAXN][MAXM][MAXM];
5  int nums[MAXN];
6
7  // 当前枚举到n的第len位,之前的数码之和为sum,lim表示是否达到上界
8  int dfs(int len, int sum, bool lim) {
9      if (!len) return sum % P == 0;
10     if (!lim && ~dp[len][sum][P]) return dp[len][sum][P];
11
12     int res = 0;
13     int up = lim ? nums[len] : 9;
14     for (int i = 0; i <= up; i++)
15         res += dfs(len - 1, sum + i, lim && i == up);

```

```

16
17     if (!lim) dp[len][sum][P] = res;
18     return res;
19 }
20
21 int cal(int n) {
22     int len = 0;
23     while (n) nums[++len] = n % 10, n /= 10;
24     return dfs(len, 0, true);
25 }
26
27 void solve() {
28     memset(dp, -1, sizeof(dp));
29
30     int l, r;
31     while (cin >> l >> r >> P) cout << cal(r) - cal(l - 1) << endl;
32 }
33
34 int main() {
35     solve();
36 }

```

10.5.6 不要62

题意

称含有4或62的正整数为不吉利数字,否则称为吉利数字.有多组测试数据,每组测试数据给定两个整数 l, r ($1 \leq l \leq r \leq 1e9$),表示询问区间 $[l, r]$ 中吉利数字的个数,输入0 0表示输入结束.

思路

$dp[i][j]$ 表示 i 位数中最高位为 j 的吉利数字的个数.根据次高位 k 分类,状态转移方程:

$$dp[i][j] = \sum_{k=1}^9 dp[i-1][k] \cdot [j \neq 4 \wedge k \neq 4 \wedge \overline{jk} \neq 62].$$

设 $n = \overline{a_{m-1}a_{m-2}\cdots a_0}$.每个 $x \in [1, n]$ 的最高位有 $0 \sim a_{m-1} - 1$ 和 a_{m-1} 两种情况.

(1)最高位为 $k \in [0, a_{m-1} - 1]$ 时,方案数为 $dp[len][k]$.

(2)最高位为 a_{m-1} 时,同理考虑次高位的情况.

代码1:迭代写法

```

1  const int MAXN = 15;
2  int dp[MAXN][MAXN]; // dp[i][j]表示i位数中最高位为j的吉利数字的个数
3
4  void init() {
5      for (int i = 0; i <= 9; i++)
6          if (i != 4) dp[1][i] = 1;
7
8      for (int i = 1; i < MAXN; i++) { // 枚举长度
9          for (int j = 0; j <= 9; j++) { // 枚举最高位
10             if (j == 4) continue;
11
12             for (int k = 0; k <= 9; k++) { // 枚举次高位
13                 if (k == 4 || j == 6 && k == 2) continue;
14
15                 dp[i][j] += dp[i-1][k];
16             }
17         }
18     }
19 }

```

```

17     }
18 }
19 }
20
21 int cal(int n) {
22     if (!n) return 1;
23
24     vector<int> digits;
25     while (n) digits.push_back(n % 10), n /= 10;
26
27     int res = 0;
28     int last = 0; // 上一位填的数
29     for (int i = digits.size() - 1; i >= 0; i--) {
30         for (int j = 0; j < digits[i]; j++) { // 左分支
31             if (j == 4 || last == 6 && j == 2) continue;
32
33             res += dp[i + 1][j];
34         }
35
36         if (digits[i] == 4 || last == 6 && digits[i] == 2) break; // 不存在右分支
37
38         last = digits[i];
39         if (!i) res++; // 右分支
40     }
41     return res;
42 }
43
44 void solve() {
45     init();
46
47     int l, r;
48     while (cin >> l >> r, l || r) cout << cal(r) - cal(l - 1) << endl;
49 }
50
51 int main() {
52     solve();
53 }

```

思路II

$dp[i][j]$ 表示未达到上界时,从第 i 位起可任意填的方案数,其中 $j = 0$ 表示前一位非6, $j = 1$ 表示前一位为6.

代码II:递归写法

```

1  const int MAXN = 15;
2  int digits[MAXN];
3  // dp[i][j]表示从第i位起可任意填的方案数,其中j=0表示前一位非6,j=1表示前一位为6
4  int dp[MAXN][2];
5
6  int dfs(int len, bool last6, bool lim) { // 当前枚举到n的第len位、上一位是否为6、是否达到上界
7      if (!len) return 1;
8      if (!lim && ~dp[len][last6]) return dp[len][last6];
9
10     int res = 0;
11     int up = lim ? digits[len] : 9;
12     for (int x = 0; x <= up; x++) { // 枚举该位填的数码
13         if (x != 4 && !(last6 && x == 2))
14             res += dfs(len - 1, x == 6, lim && x == up);
15     }
16
17     if (!lim) dp[len][last6] = res;

```

```

18     return res;
19 }
20
21 int cal(int n) {
22     if (!n) return 1;
23
24     int len = 0;
25     while (n) digits[++len] = n % 10, n /= 10;
26     return dfs(len, false, true);
27 }
28
29 void solve() {
30     memset(dp, -1, sizeof(dp));
31
32     int l, r;
33     while (cin >> l >> r, l || r) cout << cal(r) - cal(l - 1) << endl;
34 }
35
36 int main() {
37     solve();
38 }

```

10.5.7 恨7不成妻

题意

称满足如下三个条件之一的整数为与7有关,否则称与7无关:①包含数码7;②各数码之和是7的倍数;③该数是7的倍数.

有 t ($1 \leq t \leq 50$)组测试数据.每组测试数据输入两个整数 l, r ($1 \leq l \leq r \leq 1e18$),表示询问区间 $[l, r]$ 中与7无关的数的平方和,答案对 $1e9 + 7$ 取模.

思路

$dp[i][j][a][b]$ 表示 i 位数中最高位为 j 、该数模7余 a 、该数各数码之和模7余 b 的所有数的信息之和.根据次高位 k 分类,状态转移方程:

$$dp[i][j][a][b] = \sum_{k=1}^9 dp[i-1][k][(a - j \cdot 10^{i-1}) \% 7][(b - j) \% 7].$$

为求得所有数的平方和,需记录合法的数字个数、合法数字之和、合法数字的平方和.

代码I:迭代写法

```

1  const int MAXN = 25;
2  const int MOD = 1e9 + 7;
3  int pow7[MAXN], powMOD[MAXN]; // 10的幂次模7、模MOD
4  struct Node {
5      int cnt; // 合法的数的个数
6      int sum1; // 所有合法的数之和
7      int sum2; // 所有合法的数的平方和
8  } dp[MAXN][MAXN][7][7];
9  // dp[i][j][a][b]表示i位数中最高位为j、该数模7余a、该数各数码之和模7余b的所有数的信息之和
10
11 int get(ll x, int mod) { // x模mod
12     return (x % mod + mod) % mod;
13 }
14
15 void init() {
16     pow7[0] = 1;
17     for (int i = 1; i < MAXN; i++) pow7[i] = (ll)pow7[i - 1] * 10 % 7;
18     powMOD[0] = 1;
19     for (int i = 1; i < MAXN; i++) powMOD[i] = (ll)powMOD[i - 1] * 10 % MOD;
20 }

```

```

21     for (int i = 0; i <= 9; i++) {
22         if (i != 7) {
23             auto& [cnt, sum1, sum2] = dp[1][i][i % 7][i % 7];
24             cnt++, sum1 += i, sum2 += i * i;
25         }
26     }
27
28     ll pow10 = 10; // 记录10的幂次
29     for (int i = 2; i < MAXN; i++, pow10 *= 10) { // 枚举长度
30         for (int j = 0; j <= 9; j++) { // 枚举最高位
31             if (j == 7) continue;
32
33             for (int a = 0; a < 7; a++) { // 枚举整个数模7的余数
34                 for (int b = 0; b < 7; b++) { // 枚举各数码之和模7的余数
35                     for (int k = 0; k <= 9; k++) { // 枚举次高位
36                         if (k == 7) continue;
37
38                         auto& v1 = dp[i][j][a][b], & v2 = dp[i - 1][k][get((11)a - (11)j * pow10 % 7, 7)][get(b
- j, 7)];
39                         int tmp = pow10 % MOD;
40                         v1.cnt = get((11)v1.cnt + v2.cnt, MOD);
41                         v1.sum1 = get((11)v1.sum1 + v2.sum1 + (11)tmp * j % MOD * v2.cnt % MOD, MOD);
42                         v1.sum2 = get((11)v1.sum2 + (11)tmp * tmp % MOD * j % MOD * j % MOD * v2.cnt % MOD
+ v2.sum2 + (11)2 * j * tmp % MOD * v2.sum1 % MOD, MOD);
43                     }
44                 }
45             }
46         }
47     }
48 }
49 }
50
51 // 求i位数中最高位为j、该数模7不余a、该数各数码之和模7不余b的所有数的信息之和
52 Node cal(int i, int j, int a, int b) {
53     int cnt = 0, sum1 = 0, sum2 = 0;
54     for (int x = 0; x < 7; x++) {
55         for (int y = 0; y < 7; y++) {
56             if (x != a && y != b) {
57                 auto v = dp[i][j][x][y];
58                 cnt = (cnt + v.cnt) % MOD;
59                 sum1 = (sum1 + v.sum1) % MOD;
60                 sum2 = (sum2 + v.sum2) % MOD;
61             }
62         }
63     }
64     return { cnt, sum1, sum2 };
65 }
66
67 int cal(ll n) {
68     if (!n) return 0;
69
70     int tmpn = n % MOD;
71     vector<int> digits;
72     while (n) digits.push_back(n % 10), n /= 10;
73
74     int res = 0;
75     ll last = 0; // 上一位数
76     int sum = 0; // 各个数码之和
77     for (int i = digits.size() - 1; i >= 0; i--) {
78         for (int j = 0; j < digits[i]; j++) { // 左分支
79             if (j == 7) continue;
80
81             int a = get(-last * pow7[i + 1] % 7, 7), b = get(-sum, 7);
82             auto v = cal(i + 1, j, a, b);

```

```

83     int tmp = last % MOD;
84     res = get((ll)res + (ll)tmp * tmp % MOD * powMOD[i + 1] % MOD * powMOD[i + 1] % MOD * v.cnt %
MOD
85         + v.sum2 + (ll)2 * tmp * powMOD[i + 1] % MOD * v.sum1 % MOD, MOD);
86     }
87
88     if (digits[i] == 7) break; // 无右分支
89
90     last = last * 10 + digits[i], sum += digits[i];
91
92     if (!i && last % 7 && sum % 7) res = get((ll)res + (ll)tmpn * tmpn % MOD, MOD);
93 }
94 return res;
95 }
96
97 void solve() {
98     ll l, r; cin >> l >> r;
99     cout << get(cal(r) - cal(l - 1), MOD) << endl;
100 }
101
102 int main() {
103     init();
104     CaseT
105     solve();
106 }

```

代码II:递归写法

```

1  const int MAXN = 25;
2  const int MOD = 1e9 + 7;
3  int powMOD[MAXN]; // 10的幂次模MOD
4  int digits[MAXN];
5  struct Node {
6      int cnt; // 合法的数的个数
7      int sum1; // 所有合法的数之和
8      int sum2; // 所有合法的数的平方和
9  } dp[MAXN][7][7];
10 // dp[i][a][b]表示i位数中,该数各数码之和模7余a、该数模7余b的所有数的信息之和
11
12 void init() {
13     memset(dp, -1, sizeof(dp));
14
15     powMOD[0] = 1;
16     for (int i = 1; i < MAXN; i++) powMOD[i] = (ll)powMOD[i - 1] * 10 % MOD;
17 }
18
19 // 当前枚举到n的第len位、该数所有数码之和模7余sum、该数模7余b、是否达到上界
20 Node dfs(int len, int sum, int num, bool lim) {
21     if (!len) return { sum && num, 0, 0 };
22     if (!lim && ~dp[len][sum][num].cnt) return dp[len][sum][num];
23
24     Node res = { 0, 0, 0 };
25     int up = lim ? digits[len] : 9;
26     for (int x = 0; x <= up; x++) {
27         if (x == 7) continue;
28
29         Node v = dfs(len - 1, (sum + x) % 7, ((ll)num * 10 + x) % 7, lim && x == up);
30         int tmp = (ll)x * powMOD[len - 1] % MOD;
31
32         res.cnt = (res.cnt + v.cnt) % MOD;
33         res.sum1 = ((ll)res.sum1 + (ll)v.cnt * tmp % MOD + v.sum1) % MOD;

```

```

34     res.sum2 = ((ll)res.sum2 + (ll)v.cnt * tmp % MOD * tmp % MOD
35         + v.sum2 + (ll)2 * v.sum1 % MOD * tmp % MOD) % MOD;
36 }
37
38 if (!lim) dp[len][sum][num] = res;
39 return res;
40 }
41
42 int cal(ll n) {
43     int len = 0;
44     while (n) digits[++len] = n % 10, n /= 10;
45     return dfs(len, 0, 0, true).sum2;
46 }
47
48 void solve() {
49     ll l, r; cin >> l >> r;
50     cout << ((cal(r) - cal(l - 1)) % MOD + MOD) % MOD << endl;
51 }
52
53 int main() {
54     init();
55     CaseT
56     solve();
57 }

```

10.5.8 统计特殊整数

原题指路:<https://leetcode.cn/problems/count-special-integers/>

题意

称每个数码都不同的正整数为特殊整数.给定一个整数 n ($1 \leq n \leq 2e9$),求区间 $[1, n]$ 中特殊整数的个数.

思路

设 $n = \overline{a_{m-1}a_{m-2}\cdots a_0}$.每个 $x \in [1, n]$ 的最高位有 $0 \sim a_{m-1} - 1$ 和 a_{m-1} 两种情况.

(1)最高位为 $k \in [0, a_{m-1} - 1]$ 时,方案数为 A_{10-m+i}^i ,其中 i 为当前枚举的 n 的位数.

(2)最高位为 a_{m-1} 时,同理考虑次高位的情况.

对位数不足 m 位的情况,枚举位数 $i \in [1, m - 1]$,首位可填 $1 \sim 9$,其余位任意填不重复的数码,故方案数为 $9A_9^{i-1}$.

用10位二进制数 $state$ 记录当前已用过的数码.

代码

```

1  class Solution {
2  public:
3      int A(int n, int m) { // 排列数A(n,m)
4          int res = 1;
5          for (int i = n - m + 1; i <= n; i++) res *= i;
6          return res;
7      }
8
9      int countSpecialNumbers(int n) {
10         vector<int> digits;
11         while (n) digits.push_back(n % 10), n /= 10;

```

```

12
13     int res = 0;
14     int state = 0; // 记录每个数码是否被用过
15     for (int i = digits.size() - 1; i >= 0; i--) {
16         for (int j = i == digits.size() - 1; j < digits[i]; j++) // 左分支,最高位只能从1开始枚举
17             if (!(state >> j & 1)) res += A(10 - digits.size() + i, i);
18
19         if (state >> digits[i] & 1) break; // 无右分支
20
21         state |= 1 << digits[i];
22         if (!i) res++; // 右分支
23     }
24
25     // 单独统计位数不足的情况
26     for (int i = 1; i < digits.size(); i++) res += 9 * A(9, i - 1);
27     return res;
28 }
29 };

```

10.6 状压DP

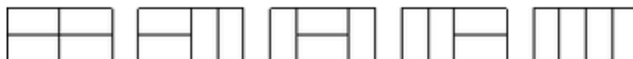
状压DP分为两种:①棋盘式(基于连通性的)DP;②集合式,用于优化暴搜,起到类似于DLX的效果.

10.6.1 蒙德里安的梦想 (1.5 s)

题意

求将 $N \times M$ ($1 \leq N, M \leq 11$) 的棋盘分割成若干个 1×2 的长方形的方案数.

如 $N = 2, M = 4$ 时有如下5种方案:



$N = 2, M = 3$ 时有如下3种方案:



思路

先放横的,放完之后竖的嵌在空位,则总方案数等于只放横的的合法方案数.

核心:如何判断当前方案是否合法,即是否能用竖的塞满棋盘.显然只需每一列中连续的空格有偶数个.

$dp[i][j]$ 表示前 $(i - 1)$ 列已摆好,且从第 $(i - 1)$ 列伸出到第 i 列的状态为 j 的方案数,其中 j 是一个二进制数,某行有伸出则取1,否则取0.

按最后一个不同点划分,因第 $(i - 1)$ 列伸到第 i 列的状态已固定,则按第 $(i - 2)$ 列伸到第 $(i - 1)$ 列的状态划分,共 2^N 种,显然这种划分是不重不漏的.

考察第 $(i - 2)$ 列伸到第 $(i - 1)$ 列的状态为 k 的情况,方案数显然为 $dp[i - 1][k]$,但未必能转移到 $dp[i][j]$,需满足:① j 和 k 不能在同一行都有1,即 $j \& k = 0$;②第 $(i - 1)$ 列的空位能被竖的填满,即状态 $j|k$ 的空格不能是奇数个.

下标从0开始,则最终答案 $ans = dp[M][0]$,即前 M 列(整个棋盘已经摆好),且从第 $(M - 1)$ 列伸到第 M 列的状态为0,即没有凸出来的.

$dp[][]$ 第一维最大值11,第二维最大值 2^{11} ,状态计算最多 2^{11} 种情况,最坏的情况 $4e7$,且有多组测试数据,容易超时.

考虑预处理出哪些状态 k 可转移到状态 j ,开一个 vector $state[]$,用 $state[j]$ 存能转移到状态 j 的状态 k .

代码

```

1  const int MAXN = 12, MAXM = 1 << MAXN;
2  ll dp[MAXN][MAXM];
3  bool sta[MAXM]; // 判断每个状态是否合法,即每列连续的空格有偶数个
4  vector<int> state[MAXM]; // 存能转移到j的状态k
5
6  int main() {
7      while (cin >> n >> m, n || m) {
8          for (int i = 0; i < 1 << n; i++) {
9              int cnt = 0; // 连续空格的个数
10             bool ok = true;
11             for (int j = 0; j < n; j++) {
12                 if (i >> j & 1) { // 若该位置是1
13                     if (cnt & 1) { // 若前面的连续空格的个数不是偶数,则不合法
14                         ok = false;
15                         break;
16                     }
17                     cnt = 0;
18                 }
19                 else cnt++;
20             }
21
22             if (cnt & 1) // 最后一个0
23                 ok = false;
24             sta[i] = ok;
25         }
26
27         for (int i = 0; i < 1 << n; i++) {
28             state[i].clear();
29
30             for (int j = 0; j < 1 << n; j++) {
31                 if ((i & j) == 0 && sta[i | j]) // i|j表示当前位置非空
32                     state[i].push_back(j);
33             }
34         }
35
36         memset(dp, 0, sizeof(dp));
37         dp[0][0] = 1; // 没有从-1列伸到第0列的方案
38         for (int i = 1; i <= m; i++) {
39             for (int j = 0; j < 1 << n; j++) {
40                 for (auto k : state[j])
41                     dp[i][j] += dp[i - 1][k];
42             }
43         }
44         cout << dp[m][0] << endl;
45     }
46 }

```

10.6.2 最短Hamilton路径 (5 s)

题意

给定一张 n ($1 \leq n \leq 20$)个点的带权无向图,节点编号 $0 \sim (n - 1)$.输出起点0到终点 $(n - 1)$ 的最短Hamilton路径(从点0到点 $(n - 1)$ 不重不漏地经过每个点恰一次)距离.

先输入整数 n ,接下来 n 行每行 n 个整数,其中第 i 行的第 j 个整数表示点 i 到 j 的距离 $a[i, j]$ ($0 \leq a[i, j] \leq 1e7$).对任意的 x, y, z ,数据保证 $a[x, x] = 0, a[x, y] = a[y, x], a[x, y] + a[y, z] \geq a[x, z]$.

思路

DFS遍历所有路径有 $20!$ 种, 暴搜会T.

考虑遍历4个节点0, 1, 2, 3的两种路径: ① $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$; ② $0 \rightarrow 2 \rightarrow 1 \rightarrow 3$. 注意到DFS遍历时只关心已遍历了哪些点和最后停在哪个点, 则中间点遍历的次序是无关紧要的, 故只需考虑2点: ①哪些点被遍历过, 最坏有20个点, 每个点有遍历过和没遍历过两种情况, 则为 $2^{20} \times 20 = 2e7$; ②最后停在哪个点上.

$dp[state][j]$ 表示状态为 $state$ (哪些点被用过), 最后停在点 j 处的最短Hamilton距离. 考虑从点 k 转移到 j , 则 $dp[state][k] = dp[state_k][k] + weight[k][j]$, 即 $state_k$ 为 $state$ 除掉 j 后的集合, 且它包含 k .

用一个20位的二进制数来表示 $state$, 1表示对应位置的点被遍历过, 0表示对应位置的点没遍历过, 总复杂度 $2e7 \times 20 = 4e7$, 故5 s可过.

初始状态 $dp[1][0] = 0$, 即一开始在点0, 只遍历过点0的Hamilton距离为0.

代码

```
1  const int INF = 0x3f3f3f3f;
2  const int MAXN = 20, MAXM = 1 << MAXN;
3  int dp[MAXM][MAXN];
4  int weight[MAXN][MAXN];
5
6  int main() {
7      for (int i = 0; i < n; i++) {
8          for (int j = 0; j < n; j++)
9              cin >> weight[i][j];
10     }
11
12     memset(dp, INF, sizeof(dp));
13     dp[1][0] = 0; // 初始条件
14     for (int i = 0; i < 1 << n; i++) {
15         for (int j = 0; j < n; j++) {
16             if (i >> j & 1) { // i的第j位是否为1
17                 for (int k = 0; k < n; k++) {
18                     if (i >> k & 1) // 是否包含k
19                         dp[i][j] = min(dp[i][j], dp[i - (1 << j)][k] + weight[k][j]);
20                 }
21             }
22         }
23     }
24     cout << dp[(1 << n) - 1][n - 1];
25 }
```

10.6.3 小国王

题意

在 $n \times n$ ($1 \leq n \leq 10$)的棋盘上放 k ($0 \leq k \leq n^2$)个国王, 国王可攻击相邻的8个格子. 求使它们无法相互攻击的方案数.

思路

注意到每一行的可行摆法只与上一行有关, $dp[i][j][s]$ 表示已放完前 i 行、已放了 j 个国王、第 i 行的状态为 s 的方案数, 其中 s 是一个 n 位二进制数, 0表示当前位置不放, 1表示当前位置放. 按第 $(i-1)$ 行的状态分类, 该行的摆法至多有 2^n 种, 其状态应满足: ①第 $(i-1)$ 行中不能有两个相邻的1; ②第 $(i-1)$ 行与第 i 行不能互相攻击. 设第 i , $(i-1)$ 行的状态分别为 a , b , 则需满足: ①没有两个1在同一列, 即 $a \& b = 0$; ②第 $(i-1)$ 行与第 i 行不能互相攻击, 即 $a|b = 0$.

考察如何计算每个合法状态的方案数. 考察已放完前 i 行、已放了 j 个国王、第 i 行的状态为 a 、第 $(i-1)$ 行的状态为 b 的方案数. 因第 i 行摆法相同, 则都去掉第 i 行, 考察已放完前 $(i-1)$ 行、已放了 $(j - \text{count}(a))$ 个国王、第 $(i-1)$ 行的状态为 b 的方案数, 其方案数为 $dp[i-1][j - \text{count}(a)][b]$.

最多10行、100个国王,最多 2^{10} 种方案,每种方案最多 2^{10} 个转移,则最坏的情况是 $1e9$ 的数量级,但注意到大部分情况都是不合法的,实际最多只有1365个方案,故最坏 $10 \times 100 \times 1365$,是 $1e6$ 的数量级。

初始条件 $dp[0][0][0] = 1$,即没有放任何国王的方案数为1.最终答案是 $dp[n][k][s]$ 中的最大值,但可假设存在第 $(n+1)$ 行,则最终答案为 $dp[n+1][k][0]$,即已放完前 n 行、已放了 k 个国王、第 $(n+1)$ 行没有放的方案数。

代码

```
1  const int MAXN = 12, MAXM = 1 << 10, MAXK = 105; // 棋盘大小、10位二进制数最大值、国王数量
2  int n, k; // 棋盘大小、国王数
3  vi state; // 存合法状态
4  int cnt[MAXM]; // 存每个状态中1的个数
5  vi head[MAXM]; // 存每个状态可转移到的状态
6  ll dp[MAXN][MAXK][MAXM]; // dp[i][j][s]表示已摆完前(i-1)行、已摆完j个国王、第i行状态为s的方案数
7
8  bool check(int state) { // 判断每个状态是否合法,即是否有相邻的1
9      for (int i = 0; i < n; i++)
10         if ((state >> i & 1) && (state >> i + 1 & 1)) return false;
11     return true;
12 }
13
14 int count(int state) { // 统计每个状态中1的个数
15     int res = 0;
16     for (int i = 0; i < n; i++) res += state >> i & 1;
17     return res;
18 }
19
20 int main() {
21     // 预处理出所有合法的状态
22     for (int i = 0; i < 1 << n; i++) {
23         if (check(i)) {
24             state.push_back(i);
25             cnt[i] = count(i);
26         }
27     }
28
29     // 预处理出每个状态能转移到的状态
30     for (int i = 0; i < state.size(); i++) {
31         for (int j = 0; j < state.size(); j++) {
32             int a = state[i], b = state[j];
33             if ((a & b) == 0 && check(a | b)) head[i].push_back(j); // 状态i可转移到j
34         }
35     }
36
37     dp[0][0][0] = 1; // 什么都不放的方案数为1
38
39     for (int i = 1; i <= n + 1; i++) { // 枚举每行,假设存在第(n+1)行,方便得到最终答案
40         for (int j = 0; j <= k; j++) { // 枚举放的国王数量
41             for (int a = 0; a < state.size(); a++) { // 枚举状态a
42                 for (int b : head[a]) { // 枚举a能转移到的状态b
43                     int c = cnt[state[a]]; // 状态a中1的个数
44                     if (j >= c) dp[i][j][a] += dp[i - 1][j - c][b];
45                 }
46             }
47         }
48     }
49     cout << dp[n + 1][k][0]; // 假设存在第(n+1)行,则无需求第n行的最大值
50 }
```

10.6.4 玉米田

题意

在 $m \times n$ ($1 \leq m, n \leq 12$) 的方格土地上种植玉米, 部分土地不育, 无法种植玉米. 相邻的土地不能同时种植玉米, 即种植玉米的所有方格间无公共边缘. 现给定土地的大小和情况, 求种植方案数, 什么也不种也算一种方案, 答案对 $1e8$ 取模.

土地情况用 $m \times n$ 的矩阵描述, 其中 0 表示该块土地不育, 1 表示该块土地肥沃.

思路

以一块种了玉米的土地为中心的十字形区域不能种其他玉米, 则某一行的摆法只与上一行有关. $dp[i][s]$ 表示摆了前 i 行、第 i 行状态为 s 的方案数. 根据第 $(i-1)$ 行的状态分类. 设第 i 行状态为 a , 第 $(i-1)$ 行状态为 b , 则需满足: ① a 和 b 都不包含两个相邻的 1; ② 同一列没有两个 1, 即 $a \& b = 0$. 第 i 行的状态都相同, 考察已摆完前 $(i-1)$ 行、第 $(i-1)$ 行的状态为 b 的方案数, 即 $dp[i-1][b]$.

最多 12 行、 2^{12} 个状态, 但大部分状态不合法.

将不育的土地置为 1, 方便与运算.

代码

```
1  const int MAXN = 14, MAXM = 1 << 12, MOD = 1e8; // 棋盘大小、12 位二进制数最大值、模数
2  int n, m; // 土地大小
3  int w[MAXN];
4  vi state; // 存合法状态
5  vi head[MAXM]; // 存每个状态可转移到的状态
6  int dp[MAXN][MAXM]; // dp[i][j][s] 表示已摆完前 (i-1) 行、已摆完 j 个国王、第 i 行状态为 s 的方案数
7
8  bool check(int state) { // 判断每个状态是否合法, 即是否有相邻的 1
9      for (int i = 0; i + 1 < m; i++)
10         if ((state >> i & 1) && (state >> i + 1 & 1)) return false;
11     return true;
12 }
13
14 int main() {
15     for (int i = 1; i <= n; i++) {
16         for (int j = 0; j < m; j++) {
17             int t; cin >> t;
18             w[i] += !t << j; // 将不育的土地置为 1, 方便后续做与运算
19         }
20     }
21
22     // 预处理出合法状态
23     for (int i = 0; i < 1 << m; i++) // 枚举每列
24         if (check(i)) state.push_back(i);
25
26     // 预处理出每个状态能转移到的状态
27     for (int i = 0; i < state.size(); i++) {
28         for (int j = 0; j < state.size(); j++) {
29             int a = state[i], b = state[j];
30             if ((a & b) == 0) head[i].push_back(j);
31         }
32     }
33
34     dp[0][0] = 1; // 什么都没种的方案数为 1
35
36     for (int i = 1; i <= n + 1; i++) { // 枚举每行
37         for (int a = 0; a < state.size(); a++) { // 枚举每个状态
38             if (state[a] & w[i]) continue; // 不合法
39
40             for (int b : head[a]) // 枚举状态 a 能转移到的状态
41                 dp[i][a] = (dp[i][a] + dp[i-1][b]) % MOD;
```

```

42     }
43 }
44 cout << dp[n + 1][0];
45 }

```

10.6.5 炮兵阵地 (64 MB)

题意

现在在 $n \times m$ ($1 \leq n \leq 100, 1 \leq m \leq 10$) 的网格地图上部署炮兵不对, 地图上的每一格用一个字母表示, 其中 'H' 表示山地, 'P' 表示平原. 每一格平原地形上至多部署一支炮兵部队, 山地上不能部署炮兵部队. 一支炮兵部队在地图上的攻击范围如下图黑色区域所示, 其他的白色区域该炮兵部队攻击不到, 攻击范围始终为如下图所示的“十”字形, 与地形无关.

P	P	H	P	H	H	P	P
P	H	P	H	P	H	P	P
P	P	P	H	H	P	H	
H	P	H	P	P	P	H	
H	P	P	P	P	H	P	H
H	P	P	H	P	H	H	P
H	H	H	P	P	P	P	H

在保证任意两支炮兵部队都不能互相攻击的前提下, 求地图区域内最多能部署多少支炮兵部队.

思路

注意到每行的摆法与上两行有关, $dp[i][j][k]$ 表示摆完前 i 行、第 $(i-1)$ 行状态为 j 、第 i 行状态为 k 的所有方案的最大值. 根据第 $(i-2)$ 行的状态分类. 设第 $(i-1)$ 行的状态为 a , 第 i 行的状态为 b , 第 $(i-2)$ 行的状态为 c , 则需满足: ① 同一列不能都有 1, 即 $(a \& b) | (b \& c) | (c \& a) = 0$; ② 不能放在山地上, 即 $g[i-1] \& a | g[i] \& b = 0$.

最多 100 行、 $2^{10} \cdot 2^{10}$ 个状态, 每个状态 2^{10} 个转移, 最坏是 $1e11$ 的级别, 但大部分方案不合法.

本题空间限制较小, 需用滚动数组.

代码

```

1  const int MAXN = 10, MAXM = 1 << 10; // 棋盘大小、10位二进制数最大值
2  int n, m; // 地图大小
3  int g[1005]; // 记录每一行山地的位置
4  vi state; // 存所有合法的方案
5  int cnt[MAXM]; // 存每个状态中1的数量
6  int dp[2][MAXM][MAXM]; // 滚动数组, dp[i][j][k]表示摆完前i行, 且第(i-1)行的状态是j, 第i行的状态是k的最大值
7
8  bool check(int state) { // 判断每个状态是否合法, 即是否有相邻的1
9      for (int i = 0; i < m; i++)
10         if ((state >> i & 1) && ((state >> i + 1 & 1) || (state >> i + 2 & 1))) return false;
11     return true;
12 }
13
14 int count(int state) { // 统计状态中1的个数
15     int res = 0;
16     for (int i = 0; i < m; i++)
17         if (state >> i & 1) res++;
18     return res;
19 }
20
21 int main() {
22     for (int i = 1; i <= n; i++) {
23         for (int j = 0; j < m; j++) {
24             char c; cin >> c;
25             g[i] += (c == 'H') << j; // 将山地置为1
26         }
27     }

```

```

28
29 // 预处理出合法状态
30 for (int i = 0; i < 1 << m; i++) {
31     if (check(i)) {
32         state.push_back(i);
33         cnt[i] = count(i);
34     }
35 }
36
37 for (int i = 1; i <= n + 2; i++) { // 枚举每行
38     for (int j = 0; j < state.size(); j++) { // 枚举第(i-1)行的状态
39         for (int k = 0; k < state.size(); k++) { // 枚举第i行的状态
40             for (int u = 0; u < state.size(); u++) { // 枚举第(i-2)行的状态
41                 int a = state[j], b = state[k], c = state[u];
42                 if (a & b | a & c | b & c) continue;
43                 if (g[i] & b | g[i - 1] & a) continue;
44
45                 dp[i & 1][j][k] = max(dp[i & 1][j][k], dp[i - 1 & 1][u][j] + cnt[b]);
46             }
47         }
48     }
49 }
50 cout << dp[n + 2 & 1][0][0];
51 }

```

10.6.6 愤怒的小鸟 (2 s)

题意

愤怒的小鸟这款游戏在一个平面上进行。一架弹弓位于 $(0, 0)$ 处,每次玩家可用其向第一象限发射一只小鸟,小鸟的飞行轨迹方程为 $y = ax^2 + bx$,其中 a, b 是玩家指定的参数,且 $a < 0$ 。小鸟落到 x 轴上时瞬间消失。游戏的目的是用小鸟消灭小猪。某关卡中,平面的第一象限有 n 只小猪,其中第 i 只小猪的坐标为 (x_i, y_i) 。若某只小鸟的飞行轨迹经过 (x_i, y_i) ,则第 i 只小猪被消灭,同时小鸟沿原轨迹继续飞行;若某只小鸟的飞行轨迹不经过 (x_i, y_i) ,则其飞行过程不影响第 i 只小猪。

该游戏对某玩家很难,于是他输入了神秘指令,使得他能轻松的通关。设有 T ($1 \leq T \leq 30$)个关卡,每个关卡的第一行输入两个非负整数 n, m ($1 \leq n \leq 18, 0 \leq m \leq 2, 0 < x_i, y_i < 10$),分别表示该关卡中小猪的数量和某玩家输入的神秘指令类型。接下来 n 行每行输入两个正实数 x_i, y_i ,表示第 i 只小猪的坐标为 (x_i, y_i) ,数据保证同一关卡中不存在两只坐标相同的小猪。神秘指令有如下三种类型:① $m = 0$ 时,某玩家输入了一条没有任何作用的指令;② $m = 1$ 时,该关卡将满足:至多用 $\left\lceil \frac{n}{3} + 1 \right\rceil$ 只小鸟即可消灭所有小猪;③ $m = 2$ 时,该关卡将满足:必存在一种最优解,其中一只小鸟消灭了至少 $\left\lfloor \frac{n}{3} \right\rfloor$ 只小猪。对每个关卡,问至少需要发射多少只小鸟才能消灭所有小猪。输入的实数均保留小数点后两位。

思路

注意到抛物线过原点,则只需 x 不相等的两个点即可确定 a, b 。先预处理出 n^2 条抛物线及它们穿过的点,问题转化为至少取多少条抛物线可穿过所有点,这是一个重复覆盖问题。

暴搜伪代码:

```

1 void dfs(int state, int cnt) { // state记录当前哪些列已被覆盖,cnt记录当前已用了几条抛物线
2     if(state已包含所有列) ans = min(ans, cnt), return;
3
4     任选没被覆盖的一列x
5     枚举能覆盖x的抛物线
6     更新state->new_state
7     dfs(new_state);
8 }

```

考虑如何优化,可用记搜防止状态被重复计算. $dp[state]$ 表示状态为覆盖 $state$ 所需的最少抛物线数.

代码

```

1  #define x first
2  #define y second
3  const int MAXN = 18, MAXM = 1 << 18; // 小猪数最大值、18位二进制数最大值
4  int n, m; // 小猪数、神秘指令
5  pdd points[MAXN]; // 存点
6  int path[MAXN][MAXN]; // path[i][j]记录穿过第i、j个点的抛物线有哪些
7  int dp[MAXM]; // dp[state]表示覆盖状态state所需的最少抛物线数
8
9  int cmp(double x, double y) { // 比较两浮点数的大小
10     if (fabs(x - y) < eps) return 0;
11     if (x < y) return -1;
12     return 1;
13 }
14
15 void init() {
16     memset(path, 0, so(path));
17     memset(dp, INF, so(dp));
18     dp[0] = 0;
19 }
20
21 int main() {
22     while (T--) {
23         init();
24
25         for (int i = 0; i < n; i++) cin >> points[i].x >> points[i].y;
26
27         // 预处理出n^2条抛物线及它们穿过的点
28         for (int i = 0; i < n; i++) {
29             path[i][i] = 1 << i; // 将只穿过一个点的抛物线置为1,因为有些点无法与其他点共一条抛物线
30
31             for (int j = 0; j < n; j++) {
32                 double x1 = points[i].x, y1 = points[i].y;
33                 double x2 = points[j].x, y2 = points[j].y;
34
35                 if (!cmp(x1, x2)) continue; // 两点x相同,不合法
36
37                 double a = (y1 / x1 - y2 / x2) / (x1 - x2), b = y1 / x1 - a * x1; // 抛物线的系数
38                 if (cmp(a, 0) >= 0) continue; // 需满足a<0
39
40                 int state = 0;
41                 for (int k = 0; k < n; k++) {
42                     double x = points[k].x, y = points[k].y;
43                     if (!cmp(a * x * x + b * x, y)) state += 1 << k; // 枚举所有能穿过x的抛物线
44                 }
45                 path[i][j] = state; // 记录能穿过点i和j的抛物线
46             }
47         }
48
49         for (int i = 0; i + 1 < 1 << n; i++) { // 枚举所有状态.state已包含所有列时无需更新,故为i+1
50             int x = 0;
51             for (int j = 0; j < n; j++) { // 找到一个当前还未被穿过的点
52                 if (!(i >> j & 1)) {
53                     x = j;
54                     break;
55                 }
56             }
57
58             for (int j = 0; j < n; j++) // 枚举穿过x的抛物线
59                 dp[i | path[x][j]] = min(dp[i | path[x][j]], dp[i] + 1);

```

```

60     }
61     cout << dp[(1 << n) - 1] << endl;
62 }
63 }

```

10.6.7 宝藏

题意

有一份藏宝图,图上有编号 $1 \sim n$ 的 n 个深埋在底下的宝藏屋和 n 个宝藏屋间可供开发的 m 条道路和它们的长度.小明想挖掘所有宝藏.每个宝藏离地面很远,赞助商可以免费帮助他打通一条从地面到某个宝藏屋的通道,通往哪个宝藏屋由小明决定.在此基础上,小明需考虑如何开发宝藏屋间的道路.每开发一条新道路,小明会挖掘由该道路到达的宝藏屋的宝藏.已开发的道路可容易通行而无需代价,开发一条新道路的代价定义为该道路的长度 $\times$ 从赞助商帮小明打通的宝藏屋到该道路起点的宝藏屋的路径上经过的宝藏屋的数量(含端点).求小明能挖掘到所有宝藏的最小代价.

第一行输入两整数 n, m ($1 \leq n \leq 12, 0 \leq m \leq 1000$),分别表示宝藏屋的个数和道路数.接下来 m 行每行输入三个整数 u, v, w ($1 \leq u, v \leq n, 0 \leq w \leq 5e5$),表示存在一条连接宝藏屋 u 和 v 的道路,长度为 w .

思路

用一个 n 位二进制数 i 的每个数位表示每个宝藏屋节点是否连通, $dp[i][j]$ 表示建立所有包含状态 i 中的节点且高度为 j 的生成树的最小代价.枚举 i 的所有真子集 S ,将其中的节点作为生成树前 $(j-1)$ 层的节点,剩余的节点作为第 j 层的节点.

求生成树的第 j 层的所有节点到 S 中的节点的最短边,将这些边的边权和乘 j 后加到 $dp[S][j-1]$ 上即可求得 $dp[i][j]$.

[证] 设这样求出的答案为 $dp'[i][j]$.

$dp[i][j]$ 中代价最小的生成树能被 $dp'[i][j]$ 枚举到,则 $dp[i][j] \geq dp'[i][j]$.

若 $dp'[i][j]$ 中生成树第 j 层中用到的某条边 $\langle a, b \rangle$ 应在 $1 \sim (j-1)$ 层,设 a 是集合 S 中的节点, b 是生成树第 j 层的节点,则在枚举 $S + \{b\}$ 时会得到更小的花费,即 $dp'[i][j]$ 枚举到的所有代码 $\geq$ 某个合法生成树的代价,即 $dp'[i][j] \geq dp[i][j]$.

综上, $dp[i][j] = dp'[i][j]$.

n 个元素取 k 个构成的集合有 C_n^k 个,每个集合有 2^k 个子集,则共 $C_n^k 2^k$ 个子集. k 枚举 $0 \sim n$,共 $\sum_{k=0}^n C_n^k 2^k = (1+2)^n = 3^n$ 个子集.

每个子集求剩余点到子集中的点的最短边时间复杂度 $O(n^2)$,总时间复杂度 $O(n^2 \cdot 3^n)$.

代码

```

1  const int MAXN = 12, MAXM = 1 << MAXN;
2  int n, m; // 节点数、边数
3  int dis[MAXN][MAXN]; // dis[u][v]表示节点u与v间的距离
4  int connect[MAXM]; // 二进制数connect[S]表示从所有节点的子集S中的节点出发能到达的节点
5  int dp[MAXM][MAXN]; // dp[i][j]表示建立所有包含状态i中的节点且高度为j的生成树的最小代价
6
7  void solve() {
8      cin >> n >> m;
9
10     memset(dis, INF, so(dis));
11     for (int i = 0; i < n; i++) dis[i][i] = 0; // 初始化
12
13     while (m--) {
14         int a, b, c; cin >> a >> b >> c;
15         a--, b--; // 下标从0开始
16         dis[a][b] = dis[b][a] = min(dis[a][b], c); // 重边保留最短边
17     }
18

```



```

19 // 预处理出从所有节点的子集S中的节点出发能到达的节点
20 for (int i = 1; i < 1 << n; i++) { // 枚举所有子集
21     for (int j = 0; j < n; j++) { // 枚举每个二进制数位
22         if (i >> j & 1) { // 该子集中有该节点
23             for (int k = 0; k < n; k++) // 枚举每个节点
24                 if (dis[j][k] != INF) connect[i] |= 1 << k;
25         }
26     }
27 }
28
29 memset(dp, INF, so(dp));
30 for (int i = 0; i < n; i++) dp[1 << i][0] = 0; // 赞助商免费打通的代价为0
31
32 for (int i = 1; i < 1 << n; i++) { // 枚举所有子集S
33     for (int j = i - 1; j; j = (j - 1) & i) {
34         if ((connect[j] & i) == i) { // 集合j中包含集合i中的所有节点
35             int remain = i ^ j; // 在生成树第j层的节点
36             int cost = 0;
37             for (int k = 0; k < n; k++) { // 枚举remain的每个二进制数位
38                 if (remain >> k & 1) {
39                     int tmp = INF;
40                     for (int u = 0; u < n; u++) // 求生成树第j层中的所有节点到S的所有节点的最短边
41                         if (j >> u & 1) tmp = min(tmp, dis[k][u]);
42                     cost += tmp;
43                 }
44             }
45
46             for (int k = 1; k < n; k++) // 枚举从起点到当前节点的路径上的节点数
47                 dp[i][k] = min(dp[i][k], dp[j][k - 1] + cost * k);
48         }
49     }
50 }
51
52 int ans = INF;
53 for (int i = 0; i < n; i++) // 枚举生成树的高度
54     ans = min(ans, dp[(1 << n) - 1][i]);
55 cout << ans;
56 }
57
58 int main() {
59     solve();
60 }

```

10.7 树形DP

10.7.1 没有上司的舞会

题意

某大学有编号为 $1 \sim n$ 的 n ($1 \leq n \leq 6000$)名职员, 他们的关系像一棵以校长为根的树, 父节点是子节点的直接上司. 每个职员有一个快乐指数, 用整数 h_i ($-128 \leq h_i \leq 127$)表示. 现要召开一场宴会, 没有职员愿意和直接上司一起参会. 在满足该前提的条件下, 主办方希望邀请一部分职员参会, 使得所有参会职员的快乐指数之和最大. 求所有参会职员的快乐指数之和的最大值.

先输入一个整数 n , 接下来 n 行中第 i 行表示第 i 个职员的快乐指数 h_i . 接下来 $(n - 1)$ 行每行输入两个整数 u, v ($1 \leq u, v \leq n, u \neq v$), 表示 v 是 u 的直接上司.

思路

$dp[u][0]$ 表示在以节点 u 为根节点的子树中选取且不选节点 u 的最大值, $dp[u][1]$ 表示在以节点 u 为根节点的子树中选取且选节点 u 的最大值. 因快乐指数可能为负, 则初始状态 $dp[u][0] = 0$, 即都不选; $dp[u][1] = happy[u]$. 最终答案 $\max\{dp[root][0], dp[root][1]\}$.

①计算 $dp[u][\]$ 时, 应先计算出节点 u 的儿子节点的状态 $dp[s_1][\], dp[s_2][\], \dots$.

②计算 $dp[u][0]$ 时, 要使其最大, 应让儿子节点的状态最大. u 的两棵子树既可选根节点, 也可不选根节点, 则 $dp[u][0] = \max\{dp[s_1][0], dp[s_1][1], dp[s_2][0], dp[s_2][1], \dots\}$.

③计算 $dp[u][1]$ 时, 因选了 u , 则不能选其儿子节点, 故 $dp[u][1] = \sum_i dp[s_i][0]$, 其中 s_i 为 u 的子树中的非根节点.

综上, 状态转移方程 $dp[u][0] = \max_{v \in sons_u} \{dp[v][0], dp[v][1]\}$, $dp[u][1] = \sum_{v \in sons_u} dp[v][0]$.

n 个节点, 每个节点2个状态, 计算每个状态时需枚举其儿子节点的数量次, 所有儿子节点的数量即树中边的数量($n - 1$), 总复杂度 $O(n)$.

输入中未给出根节点是谁, 需先求出根节点, 即无父亲节点的节点.

代码

```
1 void solve() {
2     int n; cin >> n;
3     vector<int> w(n + 1);
4     for (int i = 1; i <= n; i++) cin >> w[i];
5
6     vector<vector<int>> edges(n + 1);
7     vector<bool> isRoot(n + 1, true);
8     for (int i = 1; i < n; i++) {
9         int u, v; cin >> u >> v;
10        edges[v].push_back(u);
11        isRoot[u] = false;
12    }
13
14    // dp[u][0]表示以节点u为根节点的子树中选取且不选节点u的最大值
15    // dp[u][1]表示在以节点u为根节点的子树中选取且选节点u的最大值
16    vector<vector<int>> dp(n + 1, vector<int>(2));
17
18    function<void(int)> dfs = [&](int u) {
19        dp[u][1] = w[u];
20
21        for (auto v : edges[u]) {
22            dfs(v);
23
24            dp[u][0] += max(dp[v][0], dp[v][1]);
25            dp[u][1] += dp[v][0];
26        }
27    };
28
29    int root = 1;
30    while (!isRoot[root]) root++;
31
32    dfs(root);
33    cout << max(dp[root][0], dp[root][1]) << endl;
34 }
35
36 int main() {
37     solve();
38 }
```

10.7.2 树的直径

题意

给定一棵包含编号 $1 \sim n$ 的 n ($1 \leq n \leq 1e4$)个节点的树, 边权 w 满足 $|w| \leq 1e5$. 求一条路径, 使得该路径的两端点距离最远, 输出最长路径长度. 注意路径中可只包含一个点.

思路

有权树中, 两点间的最长路径称为树的直径. 树的直径可能不唯一.

有权树求直径的方法:

树形DP. 将每条直径按其经过的节点中高度最高的节点分类(以高度最高的点为直径的代表元素), 则只需枚举所有点, 求以该点为代表元素的直径长度的最大值.

以节点 u 为代表元素的直径分为两类:

① u 是直径端点之一, 则只需DFS求最长路径长度即可.

②直径穿过 u 点, 则枚举其以儿子节点为起点的路径长度最大值和次大值, 则最大值 + 次大值 + u 到两路径起点的距离之和即为直径长度的最大值.

代码

```
1  const int MAXN = 1e4 + 5, MAXM = MAXN << 1;
2  int n;
3  vii edges[MAXN];
4  int ans;
5
6  int dfs(int u, int fa) { // 求每个节点向下走的最长距离:当前节点、前驱节点
7      int dis = 0; // 从节点u出发的路径长度的最大值,初始化为0即可忽略负数
8      int d1 = 0, d2 = 0; // 经过节点u的路径长度的最大值、次大值,初始化为0即可忽略负数
9      for (auto [v, w] : edges[u]) {
10         if (v == fa) continue;
11
12         int d = dfs(v, u) + w;
13         dis = max(dis, d);
14
15         if (d >= d1) d2 = d1, d1 = d; // 更新最大值和次大值
16         else if (d > d2) d2 = d; // 更新次大值
17     }
18     ans = max(ans, d1 + d2);
19     return dis;
20 }
21
22 void solve() {
23     cin >> n;
24     for (int i = 0; i < n - 1; i++) {
25         int a, b, c; cin >> a >> b >> c;
26         edges[a].push_back({ b, c }), edges[b].push_back({ a, c });
27     }
28
29     dfs(1, -1); // 从根节点开始搜,根节点无前驱节点
30
31     cout << ans;
32 }
33
34 int main() {
35     solve();
```

10.7.3 树的中心

题意

给定一棵包含 $1 \sim n$ 的 n 个节点的树, 每条边上有一个权值. 求一个节点, 使得该节点到其他节点的最远距离最小.

第一行输入一个整数 n ($1 \leq n \leq 1e4$). 接下来 $(n - 1)$ 行每行输入三个整数 a, b, c ($1 \leq a, b \leq n, 1 \leq c \leq 1e5$), 表示节点 a 与 b 间存在一条权值为 c 的边.

输出所求点到其他树中节点的最远距离.

思路

枚举每个节点到其他节点的距离, 对每个节点的最远距离取 $\min$ 即可.

对每个节点 u , 将其到其他节点的路径分为两类: ①以 u 为起点向下走, 直接DFS即可; ②以 u 为起点向上走, 设 u 的父亲节点为 $fa[u]$, 考察以 $fa[u]$ 为起点的路径, 同理将其分为向下走和向上走. 对向下走的情况, 若最长路径不经过 u , 则最长路径长度即为 $fa[u]$ 到其他节点的最远距离; 否则次长路径长度即为 $fa[u]$ 到其他节点的最远距离.

两遍树形DP: ①第一遍向下走, 用子节点的信息更新父节点的信息, 记录最长、次长路径长度; ②第二遍向上走, 用父节点信息更新子节点信息, 求每个节点向上走的最远距离.

代码

```
1  const int MAXN = 1e4 + 5, MAXM = MAXN << 1;
2  int n;
3  vii edges[MAXN];
4  int down1[MAXN], down2[MAXN]; // 每个节点向下走的路径长度的最大值、次大值
5  int up[MAXN]; // 每个节点向上走的路径长度最大值
6  int nxt[MAXN]; // 最长路径中起点的后继
7  bool is_leaf[MAXN]; // 记录每个节点是否是叶子节点
8
9  int dfs_down(int u, int fa) { // 求每个节点向下走的最长距离: 当前节点、父亲节点
10     down1[u] = down2[u] = -INF; // 初始化
11     for (auto [v, w] : edges[u]) {
12         if (v == fa) continue;
13
14         int d = dfs_down(v, u) + w;
15         if (d >= down1[u]) {
16             down2[u] = down1[u], down1[u] = d; // 更新最大值和次大值
17             nxt[u] = v; // 记录后继节点
18         }
19         else if (d > down2[u]) down2[u] = d;
20     }
21
22     if (down1[u] == -INF) { // 未被更新过说明该节点无儿子节点, 即为叶子节点z
23         down1[u] = down2[u] = 0;
24         is_leaf[u] = true;
25     }
26
27     return down1[u];
28 }
29
30 void dfs_up(int u, int fa) { // 求每个节点向上走的最远距离: 当前节点、前驱节点
31     for (auto [v, w] : edges[u]) {
32         if (v == fa) continue;
33
34         if (nxt[u] == v) up[v] = max(up[u], down2[u]) + w; // 最长路径经过节点v
35         else up[v] = max(up[u], down1[u]) + w; // 最长路径不经过节点v
```

```

36     dfs_up(v, u);
37 }
38 }
39 }
40
41 void solve() {
42     cin >> n;
43     for (int i = 0; i < n - 1; i++) {
44         int a, b, c; cin >> a >> b >> c;
45         edges[a].push_back({ b, c }), edges[b].push_back({ a, c });
46     }
47
48     dfs_down(1, -1); // 从根节点向下搜,根节点无前驱节点
49     dfs_up(1, -1); // 从根节点向上搜,根节点无前驱节点
50
51     int ans = down1[1];
52     for (int i = 2; i <= n; i++) {
53         if (is_leaf[i]) ans = min(ans, up[i]); // 叶子节点只能向上走
54         else ans = min(ans, max(down1[i], up[i])); // 树中节点到其他节点的最远距离为向上走和向下走取max
55     }
56     cout << ans;
57 }
58
59 int main() {
60     solve();
61 }

```

10.7.4 数字变换

题意

若一个数 x 的约数之和 y 满足 $y < x$,则 x 与 y 可以互相变换.限定所有数字变换在不超过 n ($1 \leq n \leq 5e4$)的正整数范围内进行,求不断进行数字变换的次数的最大值.

思路

若数 x 可变换为数 y ($y < x$),则将 y 作为 x 的父节点.因每个数的约数之和唯一,故每个节点只有一个父节点,故所有数字变换构成一片森林.显然每种数字变换的序列与某棵树上的一条路径——对应,问题转化为对森林中的每一棵树求最长直径.

考虑如何快速求出每个数的约数之和.若用试除法求每个数的约数,则预处理的时间复杂度 $O(n\sqrt{n})$,可能TLE.考虑对每个数 x ,枚举哪些数是 x 的倍数,时间复杂度 $O(n \log n)$.

代码

```

1  const int MAXN = 5e4 + 5, MAXM = MAXN << 1;
2  int n;
3  vi edges[MAXN];
4  int sum[MAXN]; // 每个数的约数之和
5  bool vis[MAXN]; // 每个节点是否已被遍历过
6  int ans;
7
8  int dfs(int u) { // 当前节点
9      vis[u] = true;
10
11     int d1 = 0, d2 = 0; // 从节点u向下走的路径长度的最大值、次大值
12     for (auto v : edges[u]) {
13         if (!vis[v]) {
14             int d = dfs(v) + 1;
15             if (d >= d1) d2 = d1, d1 = d;

```

```

16     else if (d > d2) d2 = d;
17     }
18 }
19
20 ans = max(ans, d1 + d2);
21 return d1;
22 }
23
24 void solve() {
25     cin >> n;
26
27     // 预处理出所有每个数的约数之和
28     for (int i = 1; i <= n; i++)
29         for (int j = 2; j <= n / i; j++) sum[i * j] += i;
30
31     for (int i = 2; i <= n; i++) // 注意i从2开始枚举,因为sum[1]=0,而数字变换范围在正整数
32         if (sum[i] < i) edges[sum[i]].push_back(i);
33
34     for (int i = 1; i <= n; i++)
35         if (!vis[i]) dfs(i);
36
37     cout << ans;
38 }
39
40 int main() {
41     solve();
42 }

```

10.7.5 二叉苹果树

题意

有一二叉苹果树,树枝要么没有分叉,要么必有两分叉.树包含编号 $1 \sim n$ 的 n 个节点,其中节点1为根节点.用一根树枝两端连接的节点编号描述一根数值的位置.一棵苹果树的树枝过多,现需剪枝,但一些树枝上有苹果.给定需保留(与根节点连通)的树枝数,求最多能保留多少苹果.

第一行输入两整数 n, m ($1 \leq m < n \leq 100$),分别表示树的节点数和要保留的树枝数.接下来 $(n - 1)$ 行每行输入三个整数,前两个数为该树枝连接的节点编号,最后一个数为该树枝上的苹果数,不超过 $3e4$.

输出最多能保留多少苹果.

思路

将每个节点视为一个物品,以每个节点为根节点的子树视为一个物品组,则节点依赖于其父亲节点,转化为有依赖的背包问题.将每条边的权值设为该边所连的深度较大的节点的权值.

$dp[i][j]$ 表示在以 i 为根节点子树中保留 j 条边所能保留的苹果数的最大值.对节点 i ,枚举其儿子节点(若存在) s_1 和 s_2 .若需保留以 s_1 为根节点的子树中的边,则应保留边 $\langle i, s_1 \rangle$.枚举以 s_1 为根节点的子树中保留 $0, 1, \dots, (j - 1)$ 条边,它们能保留的苹果树的最大值分别为 $dp[s_1][0], dp[s_1][1], \dots, dp[s_2][j - 1]$.

代码

```

1  const int MAXN = 105, MAXM = MAXN << 1;
2  int n, m;
3  vii edges[MAXN];
4  int dp[MAXN][MAXM];
5
6  int dfs(int u, int fa) { // 当前节点、前驱节点
7      for (auto [v, w] : edges[u]) {
8          if (v == fa) continue;
9

```

```

10     dfs(v, u);    // 递归求子树的信息
11
12     for (int j = m; j >= 1; j--) {    // 枚举物品
13         for (int k = 0; k + 1 <= j; k++)    // 枚举体积
14             dp[u][j] = max(dp[u][j], dp[u][j - k - 1] + dp[v][k] + w);
15     }
16 }
17 }
18
19 void solve() {
20     cin >> n >> m;
21     for (int i = 0; i < n - 1; i++) {
22         int a, b, c; cin >> a >> b >> c;
23         edges[a].push_back({ b, c }), edges[b].push_back({ a, c });
24     }
25
26     dfs(1, -1);    // 从根节点开始搜,根节点无前驱节点
27
28     cout << dp[1][m];
29 }
30
31 int main() {
32     solve();
33 }

```

10.7.6 战略游戏

题意

现要保护一座城市,其道路构成一棵树.每个节点上的士兵可观察到所有与该节点相连的边.为保证所有边都能被观察到,求放置的士兵的最小数量.

有多组测试数据.每组测试数据第一行输入一个整数 n ($1 \leq n \leq 1500$),表示树的节点数,节点编号 $0 \sim (n - 1)$.接下来 n 行每行以[节点编号:(子节点数目) 子节点 子节点 $\dots$]的形式描述一个节点,每个节点的子节点数不超过10,每条边在输入数据中只出现一次.数据保证所有测试数据的 n 之和不超过300650.

对每组测试数据,输出放置的士兵的最小数量.

思路

显然每条边上至少选一个节点.

$dp[i][j]$ 表示只在以 i 为根节点的子树中选,且 i 的状态为 j 的方案所需士兵数的最小值,其中 $j = 0$ 表示不选节点 i , $j = 1$ 表示选节点 i .

设节点 i 的儿子节点为 $s_1, s_2, \dots$.有如下两种情况:

①若不选节点 i ,即 $j = 0$ 时,必选节点 $s_1, s_2, \dots$,此时 $dp[i][0] = dp[s_1][1] + dp[s_2][1] + \dots$.

②若选节点 i ,即 $j = 1$ 时,节点 $s_1, s_2, \dots$ 可选可不选,此时

$dp[i][1] = \min\{\min\{dp[s_1][0] + dp[s_1][1]\} + \min\{dp[s_2][0] + dp[s_2][1]\} + \dots\}$.

代码

```

1  const int MAXN = 1505;
2  int n;
3  vi edges[MAXN];
4  int dp[MAXN][2];    // dp[i][j]表示只在以i为根节点的子树中选,且i的状态为j的方案所需士兵数的最小值
5  bool vis[MAXN];
6
7  int dfs(int u) {    // 当前节点、前驱节点
8      dp[u][0] = 0, dp[u][1] = 1;    // 初始化选节点u和不选节点u所需的士兵数
9

```

```

10     for (auto v : edges[u]) {
11         dfs(v);
12
13         dp[u][0] += dp[v][1]; // 不选节点u
14         dp[u][1] += min(dp[v][0], dp[v][1]); // 选节点u
15     }
16 }
17
18 void solve() {
19     while (cin >> n) {
20         for (int i = 0; i < n; i++) edges[i].clear();
21         memset(vis, false, so(vis));
22
23         for (int i = 0; i < n; i++) {
24             int u, cnt; scanf("%d:%d", &u, &cnt);
25             while (cnt--) {
26                 int v; cin >> v;
27                 edges[u].push_back(v);
28                 vis[v] = true;
29             }
30         }
31
32         int root = 0;
33         while (vis[root]) root++; // 找到根节点
34         dfs(root); // 从根节点开始搜
35
36         cout << min(dp[root][0], dp[root][1]) << endl;
37     }
38 }
39
40 int main() {
41     solve();
42 }

```

10.7.7 皇宫看守

题意

皇宫各个宫殿的分布呈树状,宫殿可视为树中节点,两宫殿间的道路视为树中的边.在一个宫殿镇守的首位能观察到本宫殿和与本宫殿相连的其他宫殿的情况.每个宫殿全天需有人看守,不同宫殿看守所需费用不同.在能看守所有宫殿的前提下,求最小花费.

第一行输入一个整数 n ($1 \leq n \leq 1500$),表示树的节点数,节点编号 $1 \sim n$.接下来 n 行每行描述一个节点,依次为:该宫殿节点的编号 i ,在该宫殿安排看守所需费用 k ,该节点的儿子节点数 m ,接下来 m 个数 $r_1, \dots, r_m$ 分别表示该节点儿子节点的编号.

输出最小花费.

思路

$dp[i][0]$ 表示节点 i 处不放守卫,且 i 被父节点看到的最小花费, $dp[i][1]$ 表示节点 i 处不放守卫,且 i 被子节点看到的最小花费, $dp[i][2]$ 表示在节点 i 处放守卫的最小花费.

①若节点 i 处不放守卫,且 i 被父节点看到,则 i 的子节点只能放守卫或不放守卫且被其子节点看到,

$$\text{即 } dp[i][0] = \sum_{\text{edge} \langle i, j \rangle} \min\{dp[j][1], dp[j][2]\}.$$

②若节点 i 处放守卫,则 i 的子节点放不放守卫都可以,即 $dp[i][2] = \sum_{\text{edge} \langle i, j \rangle} \min\{dp[j][0], dp[j][1], dp[j][2]\}.$

③若节点 i 处不放守卫,且 i 被其子节点 k 看到,则节点 k 处只能放守卫, i 的其他子节点放不放守卫都可以,

$$\text{即 } dp[i][1] = \min_k \left\{ dp[k][2] + \sum_{\substack{\text{edge} \langle i,j \rangle \\ j \neq k}} \min\{dp[j][1], dp[j][2]\} \right\}.$$

最终答案 $ans = \min\{dp[root][1], dp[root][2]\}$.

代码

```

1  const int MAXN = 1505;
2  int n;
3  int w[MAXN]; // w[u]表示在节点u处放守卫的花费
4  vi edges[MAXN];
5  int dp[MAXN][3];
6  bool vis[MAXN];
7
8  int dfs(int u) { // 当前节点、前驱节点
9      dp[u][2] = w[u]; // 在节点u处放守卫
10
11     int sum = 0; // 记录子树中min{dp[v][1], dp[v][2]}之和
12     for (auto v : edges[u]) {
13         dfs(v);
14
15         dp[u][0] += min(dp[v][1], dp[v][2]); // 节点u处不放守卫,且u被父节点看见
16         dp[u][2] += min({ dp[v][0], dp[v][1], dp[v][2] }); // 在节点u处放守卫
17         sum += min(dp[v][1], dp[v][2]);
18     }
19
20     dp[u][1] = INF; // 节点u处不放首位,且节点u被子节点看见
21     for (auto v : edges[u])
22         dp[u][1] = min(dp[u][1], sum - min(dp[v][1], dp[v][2]) + dp[v][2]);
23 }
24
25 void solve() {
26     cin >> n;
27     for (int i = 1; i <= n; i++) {
28         int u, cnt; cin >> u >> w[u] >> cnt;
29         while (cnt--) {
30             int v; cin >> v;
31             edges[u].push_back(v);
32             vis[v] = true;
33         }
34     }
35
36     int root = 1;
37     while (vis[root]) root++; // 找到根节点
38
39     dfs(root); // 从根节点开始搜
40
41     cout << min(dp[root][1], dp[root][2]) << endl;
42 }
43
44 int main() {
45     solve();
46 }

```

10.7.8 Removing Leaves

原题指路:<https://codeforces.com/contest/1385/problem/F>

题意

给定一棵包含编号 $1 \sim n$ 的 n 个节点的无根树.每次操作可选择同一树中节点的 k 个叶子节点,将叶子节点和它们与父亲节点所连的边删去.求最多能进行几次操作.

有 t ($1 \leq t \leq 2e4$)组测试数据.每组测试数据第一行输入两个整数 n, k ($2 \leq n \leq 2e5, 1 \leq k \leq n$).接下来 $(n - 1)$ 行每行输入两个整数 u, v ($1 \leq u, v \leq n$),表示节点 u 与 v 间存在边.

思路

显然应优先删叶子节点数量多的树中节点,则可按叶子节点的个数将树中节点降序排列.注意到每次删除后可能原本的树中节点变为新的叶子节点,即使不产生新的叶子节点,原本的树中节点的顺序也可能改变,可用set存边,操作前先将节点从set中删除,进行操作后再将节点插回set中.当叶子节点数最多的树中节点的叶子节点数 $< k$ 时操作结束.

代码

```

1  const int MAXN = 2e5 + 5;
2  int n, k;
3  set<int> edges[MAXN];
4  set<int> leaves[MAXN]; // 存每个节点的叶子节点
5  struct compare {
6      bool operator()(const int x, const int y) const { // 按叶子节点的个数降序排列
7          return leaves[x].size() != leaves[y].size() ? leaves[x].size() > leaves[y].size() : x < y;
8      }
9  };
10 set<int, compare> que; // 充当一个队列
11
12 void solve() {
13     cin >> n >> k;
14     for (int i = 1; i <= n; i++) edges[i].clear(), leaves[i].clear();
15     que.clear();
16
17     for (int i = 1; i < n; i++) {
18         int u, v; cin >> u >> v;
19         edges[u].insert(v), edges[v].insert(u);
20     }
21
22     for (int u = 1; u <= n; u++) // 求树中节点的叶子节点
23         if (edges[u].size() == 1) leaves[*edges[u].begin()].insert(u);
24
25     for (int u = 1; u <= n; u++) que.insert(u); // 所有节点入队
26
27     int ans = 0;
28     while (true) {
29         int u = *que.begin();
30         if (leaves[u].size() < k) break;
31
32         for (int i = 0; i < k; i++) { // 删除k个节点
33             int v = *leaves[u].begin(); // 队首
34
35             // 删边
36             edges[u].erase(v), edges[v].erase(u);
37             if (leaves[u].count(v)) leaves[u].erase(v);
38             if (leaves[v].count(u)) leaves[v].erase(u);
39
40             que.erase(u), que.erase(v); // 先删除
41
42             if (edges[u].size() == 1) { // 原本的树中节点变为新的叶子节点
43                 int tmp = *edges[u].begin();

```

```

44     que.erase(tmp); // 先删除
45     leaves[tmp].insert(u);
46     que.insert(tmp); // 再插入
47 }
48
49     que.insert(u), que.insert(v); // 再插入
50 }
51     ans++;
52 }
53     cout << ans << endl;
54 }
55
56 int main() {
57     CaseT // 单测时注释掉该行
58     solve();
59 }

```

思路II

注意到对任一删除的方案,最终都至少存在一个节点无法删除,不妨以它为根节点.

考虑换根DP. $valid[u]$ 表示以节点 u 为根节点的子树能否删除到只剩下节点 u , $dp[u]$ 表示以 u 为根节点的子树最多能删多少次,则

$$ans = \max_{1 \leq u \leq n} dp[u].$$

设 $cnt[u]$ 表示节点 u 的儿子节点中 $valid[]$ 值为true的节点数.状态转移方程:

$$\begin{cases} valid_u = [cnt_u = |son_u| \cdot k \mid cnt_u] \\ dp[u] = \sum_{v \in son_u} dp[v] + \left\lfloor \frac{cnt_u}{k} \right\rfloor \end{cases}.$$

代码II

```

1  const int MAXN = 2e5 + 5;
2  int n, k;
3  vi edges[MAXN];
4  bool valid[MAXN]; // valid[u]表示以节点u为根节点的子树能否删除到只剩下节点u
5  int cnt[MAXN]; // cnt[u]表示节点u的儿子节点中valid为true的节点数
6  int dp[MAXN]; // dp[u]表示以u为根节点的子树最多能删多少次
7  int ans;
8
9  void dfs1(int u, int fa) { // 预处理出valid[], cnt[], dp[]:当前节点、前驱节点
10     cnt[u] = 0, valid[u] = true, dp[u] = 0;
11
12     for (auto v : edges[u]) {
13         if (v == fa) continue;
14
15         dfs1(v, u);
16         cnt[u] += valid[v], valid[u] &= valid[v], dp[u] += dp[v];
17     }
18
19     valid[u] &= cnt[u] % k == 0;
20     dp[u] += cnt[u] / k;
21 }
22
23 void dfs2(int u, int fa) {
24     ans = max(ans, dp[u]);
25
26     for (auto v : edges[u]) {
27         if (v == fa) continue;
28
29         // 备份
30         bool valid_u = valid[u], valid_v = valid[v];
31         int cnt_u = cnt[u], cnt_v = cnt[v];

```

```

32     int dp_u = dp[u], dp_v = dp[v];
33
34     // 换根
35     cnt[u] -= valid[v];
36     valid[u] = (cnt[u] == edges[u].size() - 1) && (cnt[u] % k == 0);
37     dp[u] = dp[u] - dp[v] - (cnt[u] + valid[v]) / k + cnt[u] / k;
38     cnt[v] += valid[u];
39     valid[v] = (cnt[v] == edges[v].size() - 1) && (cnt[v] % k == 0);
40     dp[v] = dp[v] + dp[u] - (cnt[v] - valid[u]) / k + cnt[v] / k;
41     dfs2(v, u);
42
43     // 恢复
44     valid[u] = valid_u, valid[v] = valid[v];
45     cnt[u] = cnt_u, cnt[v] = cnt_v;
46     dp[u] = dp_u, dp[v] = dp_v;
47 }
48 }
49
50 void solve() {
51     cin >> n >> k;
52
53     ans = 0;
54     for (int i = 1; i <= n; i++) edges[i].clear();
55
56     for (int i = 1; i < n; i++) {
57         int u, v; cin >> u >> v;
58         edges[u].push_back(v), edges[v].push_back(u);
59     }
60
61     dfs1(1, -1); // 从1号节点开始搜,其无前驱节点
62     dfs2(1, -1); // 从1号节点开始搜,其无前驱节点
63     cout << ans << endl;
64 }
65
66 int main() {
67     CaseT // 单测时注释掉该行
68     solve();
69 }

```

10.7.9 Vertex Deletion

题意

给定一棵包含编号 $1 \sim n$ 的 n 个节点的树,其中节点1为根节点.每次操作可删除一个节点.好的操作定义为删除节点后没有孤立的节点,即每个节点至少存在另一个节点与之连通.求好的操作的个数,答案对998244353取模.

有 t 组测试数据.每组测试数据第一行输入一个整数 n ($1 \leq n \leq 1e6$).接下来 $(n - 1)$ 行每行输入两个整数 u, v ($1 \leq u, v \leq n, u \neq v$),表示节点 u 与 v 间存在边.数据保证所有测试数据的 n 之和不超过 $1e6$.

思路

$dp[u][0]$ 表示删除节点 u ; $dp[u][1]$ 表示不删除节点 u , 但删去所有儿子节点; $dp[u][2]$ 表示不删除节点 u , 且至少保留一个儿子节点.最终答案 $dp[1][0] + dp[1][2]$.

对 $dp[u][0]$, 无需考虑节点 u 是否符合条件; 对 $dp[u][1]$, 为使得节点 u 符合条件, 需保留 u 的父节点; 对 $dp[u][2]$, 节点 u 始终满足条件.

设节点 u 的一个子节点为 v .

① 若删除 u , 为使得 v 符合条件, 可删除 v 或至少保留 v 的一个子节点, 状态转移方程: $dp[u][0] = \prod_{edge < u, v >} (dp[v][0] + dp[v][2])$.

②若不删除 u 且删除所有儿子节点,则 v 一定被删除,状态转移方程: $dp[u][1] = \prod_{edge \langle u,v \rangle} dp[v][0]$.

③若不删除 u 且至少保留一个儿子节点,考虑容斥,状态转移方程:

$$dp[u][2] = \prod_{edge \langle u,v \rangle} (dp[v][0] + dp[v][1] + dp[v][2]) - dp[u][1].$$

代码

```

1  const int MAXN = 2e5 + 5;
2  const int MOD = 998244353;
3  int n;
4  vi edges[MAXN];
5  // dp[u][0]表示删除节点u
6  // dp[u][1]表示不删除节点u,但删去所有儿子节点
7  // dp[u][2]表示不删除节点u,且至少保留一个儿子节点
8  int dp[MAXN][3];
9
10 void dfs(int u, int fa) { // 当前节点、前驱节点
11     dp[u][0] = dp[u][1] = dp[u][2] = 1; // 乘法,初始化为1
12
13     for (auto v : edges[u]) {
14         if (v == fa) continue;
15
16         dfs(v, u); // 递归求子树信息
17
18         dp[u][0] = ((1ll)dp[v][0] + dp[v][2]) % MOD * dp[u][0] % MOD;
19         dp[u][1] = (1ll)dp[u][1] * dp[v][0] % MOD;
20         dp[u][2] = ((1ll)dp[v][0] + dp[v][1] + dp[v][2]) % MOD * dp[u][2] % MOD;
21     }
22     dp[u][2] = ((dp[u][2] - dp[u][1]) % MOD + MOD) % MOD;
23 }
24
25 void solve() {
26     cin >> n;
27
28     for (int i = 1; i <= n; i++) edges[i].clear();
29
30     for (int i = 1; i < n; i++) {
31         int u, v; cin >> u >> v;
32         edges[u].push_back(v), edges[v].push_back(u);
33     }
34
35     dfs(1, -1); // 从根节点开始搜,根节点无前驱节点
36
37     cout << (dp[1][0] + dp[1][2]) % MOD << endl;
38 }
39
40 int main() {
41     CaseT // 单测时注释掉该行
42     solve();
43 }
```

10.7.10 Werewolves

题意

给定一棵包含编号 $1 \sim n$ 的 n 个节点的树,其中第 i ($1 \leq i \leq n$)个节点的颜色为 c_i .好的连通子图满足:子图中的节点中有一个颜色的数量严格大于该子图的节点数的一半,称该颜色为该子图的主要颜色.求该树的好连通子图的个数,答案对998244353取模.

第一行输入一个整数 n ($1 \leq n \leq 3000$).第二行输入 n 个整数 $c_1, \dots, c_n$ ($1 \leq c_i \leq n, 1 \leq i \leq n$).接下来 $(n-1)$ 行每行输入两个整数 u, v ($1 \leq u, v \leq n, u \neq v$),表示节点 u 与 v 间存在边.

思路

一个好的连通子图中的主要颜色唯一.

[证] 若不然,设某好的连通子图有两个主要颜色.

因主要颜色的节点数严格大于子图的节点数的一半,则两种主要颜色的节点数之和 $>$ 该子图的节点数,矛盾.

对每个颜色,统计以它为主要颜色的好的连通子图的个数.

固定一个主要颜色,将与该颜色相同的节点的权值置为1,颜色不同的节点的权值置为-1,问题转化为求点权之和 ≥ 1 的子树的个数. $dp[u][sum]$ 表示以 u 为根节点的子树中点权之和为 sum 的方案数.暴力转移时间复杂度 $O(n^4)$,会TLE.

考虑优化.设颜色为 c 的节点数为 cnt_c .注意到当前状态只能转移到满足 $-cnt_c < sum \leq cnt_c$ 的状态,因为无法得到更大或更小的点权和.设当前子树大小为 siz .注意到当前状态只能转移到满足 $|sum| \leq siz$ 的状态,故转移只需考虑满足 $|sum| \leq \min\{cnt_c, siz\}$ 的情况.计算每个主要颜色的时间复杂度为 $O(n \cdot cnt_c)$,则总时间复杂度 $\sum_c O(n \cdot cnt_c) = O\left(n \cdot \sum_c cnt_c\right) = O(n^2)$,可过.

代码

```

1  const int MAXN = 3005;
2  const int MOD = 998244353;
3  int n, m; // 节点数、每种颜色的节点数
4  int c[MAXN]; // 颜色
5  vi edges[MAXN];
6  bool vis[MAXN]; // 记录每个颜色是否遍历过
7  int w[MAXN]; // 点权
8  int dp1[MAXN][MAXN], tmp1[MAXN][MAXN]; // dp1[u][sum]表示以u为根节点的子树中点权之和为sum(sum≥1)的方案数
9  int dp2[MAXN][MAXN], tmp2[MAXN][MAXN]; // dp2[u][sum]表示以u为根节点的子树中点权之和为-sum(sum≥1)的方案数
10 int dp3[MAXN], tmp3[MAXN]; // dp3[u]表示以u为根节点的子树中点权之和为0的方案数
11 int ans;
12
13 int dfs(int u, int fa) { // 当前节点、前驱节点
14     int res = 1; // 子树大小
15     if (w[u] == 1) dp1[u][1] = 1; // 相同颜色
16     else dp2[u][1] = 1; // 不同颜色
17
18     for (auto v : edges[u]) {
19         if (v == fa) continue;
20
21         int siz = dfs(v, u); // 递归求子树的信息
22
23         for (int i = 0; i <= min(res, m); i++) // 备份当前状态
24             tmp1[u][i] = dp1[u][i], tmp2[u][i] = dp2[u][i], tmp3[u] = dp3[u];
25
26         for (int j = 1; j <= min(siz, m); j++) { // 转移不超过当前子树大小与相同颜色数的最小值,下同
27             dp1[u][j] = ((1ll)dp1[u][j] + (1ll)tmp3[u] * dp1[v][j]) % MOD; // 总和为0,+j转移到总和为j
28             dp2[u][j] = ((1ll)dp2[u][j] + (1ll)tmp3[u] * dp2[v][j]) % MOD; // 总和为0,-j转移到总和为-j
29         }
30         dp3[u] = ((1ll)dp3[u] + (1ll)tmp3[u] * dp3[v]) % MOD; // 总和为0,+0转移到总和为0
31
32         for (int i = 1; i <= min(res, m); i++) {
33             dp1[u][i] = ((1ll)dp1[u][i] + (1ll)tmp1[u][i] * dp3[v]) % MOD; // 总和为i,+0转移到总和为i
34             dp2[u][i] = ((1ll)dp2[u][i] + (1ll)tmp2[u][i] * dp3[v]) % MOD; // 总和为-i,+0转移到总和为-i

```

```

35
36     for (int j = 1; j <= min(siz, m); j++) {
37         if (i + j <= m) {
38             dp1[u][i + j] = ((ll)dp1[u][i + j] + (ll)tmp1[u][i] * dp1[v][j]) % MOD; // 总和为i,+j转移
到总和为i+j
39             dp2[u][i + j] = ((ll)dp2[u][i + j] + (ll)tmp2[u][i] * dp2[v][j]) % MOD; // 总和为-i,-j转移
到总和为-(i+j)
40         }
41
42         if (i - j >= 1) {
43             dp1[u][i - j] = ((ll)dp1[u][i - j] + (ll)tmp1[u][i] * dp2[v][j]) % MOD; // 总和为i,-j转移
到总和为i-j
44             dp2[u][i - j] = ((ll)dp2[u][i - j] + (ll)tmp2[u][i] * dp1[v][j]) % MOD; // 总和为-i,+j转移
到总和为-(i-j)
45         }
46
47         if (j - i >= 1) {
48             dp1[u][j - i] = ((ll)dp1[u][j - i] + (ll)tmp2[u][i] * dp1[v][j]) % MOD; // 总和为-i,+j转移
到总和为j-i
49             dp2[u][j - i] = ((ll)dp2[u][j - i] + (ll)tmp1[u][i] * dp2[v][j]) % MOD; // 总和为i,-j转移
到总和为-(j-i)
50         }
51
52         if (i == j) {
53             dp3[u] = ((ll)dp3[u] + (ll)tmp1[u][i] * dp2[v][j] + (ll)tmp2[u][i] * dp1[v][j]) % MOD;
54             // ①总和为i,-j;②总和为-i,+j,转移到总和为0
55         }
56     }
57 }
58
59 res += siz; // 更新子树大小
60 }
61
62 for (int i = 1; i <= min(res, m); i++)
63     ans = ((ll)ans + dp1[u][i]) % MOD; // 更新使得子树点权和≥1的方案数
64
65 return res;
66 }
67
68 void solve() {
69     cin >> n;
70     for (int i = 1; i <= n; i++) cin >> c[i];
71     for (int i = 1; i < n; i++) {
72         int u, v; cin >> u >> v;
73         edges[u].push_back(v), edges[v].push_back(u);
74     }
75
76     for (int i = 1; i <= n; i++) { // 每种颜色做一次树上背包
77         if (vis[c[i]]) continue;
78
79         vis[c[i]] = true;
80         m = 0;
81         for (int j = 1; j <= n; j++) {
82             if (c[j] == c[i]) {
83                 m++;
84                 w[j] = 1; // 相同颜色权值为1
85             }
86             else w[j] = -1; // 不同颜色权值为-1
87         }
88
89         // 初始化
90         for (int j = 1; j <= n; j++)
91             for (int k = 0; k <= m; k++) dp1[j][k] = dp2[j][k] = dp3[j] = 0;

```

```

92
93     dfs(1, -1); // 从根节点开始搜,根节点无前驱节点
94 }
95 cout << ans;
96 }
97
98 int main() {
99     solve();
100 }

```

10.7.11 Tree Division

题意

给定一棵包含编号 $1 \sim n$ 的 n 个节点的树,其中第 i ($1 \leq i \leq n$) 个节点有权值 a_i .判断节点1是否有效,若有效则输出"YES",否则输出"NO".定义节点 t 有效,如果可将 n 个节点划分为两个集合 A 、 B ,使得:①对 $\forall u, v \in A$ ($u \neq v$),若 u 是从节点 t 到节点 v 的路径,则 $a_u < a_v$;②对 $\forall u, v \in B$ ($u \neq v$),若 u 是从节点 t 到节点 v 的路径,则 $a_u > a_v$.

第一行输入一个整数 n ($1 \leq n \leq 1e5$).第二行输入 n 个整数 a_i ($1 \leq a_i \leq n$).接下来 $(n - 1)$ 行每行输入两个整数 x, y ($1 \leq x, y \leq n$),表示节点 x 与节点 y 间存在边.

思路

注意到以节点1为根节点时,节点1有效的条件为:①对 $\forall u, v \in A$ ($u \neq v$),若节点 v 是以节点 u 为根节点的子树中的节点,则 $a_u < a_v$,递推得 a_v 是根节点到节点 v 的路径上权值最大的点,且之后放入集合 A 中的节点的权值 $> a_v$;②对 $\forall u, v \in B$ ($u \neq v$),若节点 v 是以节点 u 为根节点的子树中的节点,则 $a_u > a_v$,递推得 a_v 是根节点到节点 v 的路径上权值最小的点,且之后放入集合 B 中的节点的权值 $< a_v$.

贪心策略:将节点放入集合 A 时,保证集合 B 中的节点的最小值尽量大;将节点放入集合 B 中时,保证集合 A 中的节点的最大值尽量小. $dp[u][0]$ 表示将节点 u 放入集合 A 中时集合 B 中的节点权值的最小值的最大值, $dp[u][1]$ 表示将节点 u 放入集合 B 中时集合 A 中的节点权值的最大值的最大值.初始条件 $dp[1][0] = -INF$, $dp[1][1] = INF$,都表示不合法的状态.

设节点 u 的一个子节点为 v .对 $dp[u][0]$,若 v 能放入集合 A ,由两种情况:① a_v 严格大于根节点到节点 v 的路径上权值最大的节点(节点 u),即 $a_v > a_u$;②将 v 放入集合 B 中时集合 A 中的节点权值的最大值的最大值(即 $dp[v][1]$)严格大于 a_u ,即 $a_u < dp[v][1]$.同理可得 $dp[u][1]$ 的状态转移.

$$\text{状态转移方程:} \begin{cases} dp[u][0] = \max_{\text{edge} < u, v >} \{ \min \{ a_u < a_v ? dp[v][0] : -INF, a_u < dp[v][1] ? a_v : -INF \} \} \\ dp[u][1] = \min_{\text{edge} < u, v >} \{ \max \{ a_u > a_v ? dp[v][1] : -INF, a_u > dp[v][0] ? a_v : -INF \} \} \end{cases}$$

若更新后 $dp[1][0] = -INF$ 且 $dp[1][1] = -INF$,则节点1无效,否则有效.

代码

```

1  const int MAXN = 1e5 + 5;
2  int n;
3  int a[MAXN];
4  vi edges[MAXN];
5  vi son[MAXN]; // son[u]表示节点u的儿子节点
6  // dp[u][0]表示将节点u放入集合A中时集合B中的节点权值的最小值的最大值
7  int dp[MAXN][2]; // dp[u][1]表示将节点u放入集合B中时集合A中的节点权值的最大值的最大值
8
9  void dfs1(int u, int fa) { // 预处理出以节点1为根节点时每个节点的儿子节点:当前节点、前驱节点
10     for (auto v : edges[u]) {
11         if (v == fa) continue;
12
13         son[u].push_back(v);
14         dfs1(v, u);
15     }
16 }
17

```



```

18 void dfs2(int u) { // 树形DP
19     for (auto v : son[u]) {
20         dfs2(v); // 递归求以v为根节点的子树的信息
21
22         dp[u][0] = max(dp[u][0], min(a[u] < a[v] ? dp[v][0] : INF, a[u] < dp[v][1] ? a[v] : INF));
23         dp[u][1] = min(dp[u][1], max(a[u] > a[v] ? dp[v][1] : -INF, a[u] > dp[v][0] ? a[v] : -INF));
24     }
25 }
26
27 void solve() {
28     cin >> n;
29     for (int i = 1; i <= n; i++) {
30         cin >> a[i];
31         dp[i][0] = -INF, dp[i][1] = INF; // 初始化为不合法状态
32     }
33     for (int i = 0; i < n - 1; i++) {
34         int x, y; cin >> x >> y;
35         edges[x].push_back(y), edges[y].push_back(x);
36     }
37
38     dfs1(1, -1);
39     dfs2(1);
40
41     if (dp[1][0] == INF && dp[1][1] == -INF) cout << "NO";
42     else cout << "YES";
43 }
44
45 int main() {
46     solve();
47 }

```

10.7.12 Destruction of a Tree

原题指路:<https://codeforces.com/problemset/problem/963/B>

题意

给定一个包含 n 个节点的树.一个节点能被破坏当且仅当其度数为偶数,破坏一个节点同时会破坏所有与其相连的边.问所有节点是否都能被破坏.

第一行输入一个整数 n ($1 \leq n \leq 2e5$).第二行输入 n 个整数 $p_1, \dots, p_n$ ($0 \leq p_i \leq n$),若 $p_i \neq 0$,表示节点 i 与节点 p_i 间存在无向边.数据保证给定的是一棵树.

若不能破坏所有节点,输出"NO";否则输出"YES",并输出任一破坏节点的顺序.

思路

易想到BFS,先将所有度数为偶数的点入队,模拟破坏节点的过程.但注意到即使使用堆也不能保证不会出现因之前破坏了某个节点使得最优解中本应可以破坏的节点无法破坏的情况,故直接贪心地删除是错误的.注意到子树间是独立的,即可依次把各个子树破坏干净,且一棵子树能否被完整破坏不影响另一棵子树,故按节点的DFS序破坏即可保证不会出现上述情况.

代码

```

1  const int MAXN = 2e5 + 5;
2  int n;
3  vi edges[MAXN];
4  int d[MAXN]; // 节点的度数
5  int dfn[MAXN], idfn[MAXN], idx; // 节点与其dfs序间的双射
6  bool vis[MAXN];
7  int ans[MAXN];
8

```

```

9 void dfs(int u, int fa) {
10     dfn[u] = ++idx, idfn[idx] = u;
11     for (auto v : edges[u]) {
12         if (v == fa) continue;
13
14         dfs(v, u);
15     }
16 }
17
18 void solve() {
19     cin >> n;
20     for (int i = 1; i <= n; i++) {
21         int p; cin >> p;
22         if (p) {
23             edges[i].push_back(p), edges[p].push_back(i);
24             d[i]++, d[p]++;
25         }
26     }
27
28     dfs(1, -1); // 从节点1开始搜,其无前驱节点
29
30     pqe<int> que;
31     for (int i = 1; i <= n; i++) // 度数为偶数的节点入队
32         if (d[i] % 2 == 0) que.push(dfn[i]);
33
34     int cnt = 0; // 当前删除的节点数
35     while (que.size()) {
36         int u = idfn[que.top()]; que.pop();
37         if (vis[u] || d[u] & 1) continue;
38
39         vis[u] = true;
40         ans[++cnt] = u;
41         for (auto v : edges[u]) {
42             d[v]--;
43             if (d[v] % 2 == 0) que.push(dfn[v]);
44         }
45     }
46
47     if (cnt != n) cout << "NO";
48     else {
49         cout << "YES" << endl;
50         for (int i = 1; i <= n; i++) cout << ans[i] << endl;
51     }
52 }
53
54 int main() {
55     solve();
56 }

```

10.7.11 Xorcerer's Stones

原题指路:<https://codeforces.com/contest/1779/problem/F>

题意 (4 s)

给定一棵包含编号 $1 \sim n$ 的 n 个节点的树,其中1号节点是根节点,节点 i ($1 \leq i \leq n$) 有一个权值 a_i . 现有操作:选择一个节点 i ($1 \leq i \leq n$), 计算以 i 为根节点的子树中所有节点(含节点 i) 的权值的异或和 x , 令以 i 为根节点的子树中的所有节点的权值为 x . 若能通过至多 $2n$ 次操作, 使得所有节点的权值都为0, 则操作次数和方案, 注意无需最小化操作次数; 否则输出 -1 .

第一行输入一个整数 n ($2 \leq n \leq 2e5$). 第二行输入 n 个整数 $a_1, \dots, a_n$ ($0 \leq a_i \leq 31$). 第三行输入 $(n - 1)$ 个整数 $p_2, \dots, p_n$ ($1 \leq p_i \leq i - 1$), 其中 p_i 表示节点 i 的父亲节点. 数据保证输入构成一棵树.

若无解,输出 -1 ;否则第一行输出一个整数 q ($0 \leq q \leq 2n$),表示操作次数.第二行输出 q 个整数 $v_1, \dots, v_p$ ($1 \leq v_i \leq n$),其中 v_i ($1 \leq i \leq q$)表示第 i 次操作选择的节点.若有多组解,输出任一组.

思路

若能在至多 $(2n - 1)$ 步内使得所有节点的点权的异或和为0,则在根节点处再进行一次操作即可满足要求.

注意到操作子树大小为奇数的节点不改变子树中所有节点的点权的异或和,

(还没写完)

代码

```
1 |
```

10.8 记忆化搜索

10.8.1 滑雪

题意

给定一个 R 行 C 列($1 \leq R, C \leq 300$)的矩阵,表示一个举行网格滑雪场,其中 (i, j) 元素表示滑雪场第 i 行第 j 列区域的高度,范围是 $[0, 10000]$.某人从滑雪场的某个区域出发,每次可向上下左右任一方向滑动一个单位,能滑动到相邻区域的前提是目标区域的高度低于当前所在的区域.输出最长滑雪距离(能经过的最大区域数).

思路

$dp[i][j]$ 表示从区域 (i, j) 开始滑的最大滑雪距离.

按第一步向哪个方向划分,如向右滑到区域 $(i, j + 1)$,则只需先求得从 $(i, j + 1)$ 开始滑的最长滑雪距离,即 $dp[i][j] = dp[i][j + 1] + 1$,这类划分存在的条件是 $height[i][j + 1] < height[i][j]$.

因计算 $dp[i][j]$ 需用到其他状态,则状态图需为拓扑图,即不能出现环.由滑雪高度严格单调递减知不会出现环.

代码

```
1  const int MAXN = 305;
2  int n, m;
3  int height[MAXN][MAXN];
4  int dp[MAXN][MAXN];
5  int dx[4] = { -1,0,1,0 }, dy[4] = { 0,1,0,-1 };
6
7  int solve(int x, int y) {
8      int& v = dp[x][y]; // 简化代码
9      if (~v) return v;
10
11     v = 1;
12     for (int i = 0; i < 4; i++) {
13         int curx = x + dx[i], cury = y + dy[i];
14         if (curx >= 1 && curx <= n && cury >= 1 && cury <= m && height[x][y] > height[curx][cury])
15             v = max(v, solve(curx, cury) + 1);
16     }
17     return v;
```

```

18 }
19
20 int main() {
21     for (int i = 1; i <= n; i++) {
22         for (int j = 1; j <= m; j++)
23             cin >> height[i][j];
24     }
25
26     memset(dp, -1, sizeof(dp));
27     int ans = 0;
28     for (int i = 1; i <= n; i++) {
29         for (int j = 1; j <= m; j++)
30             ans = max(ans, solve(i, j));
31     }
32     cout << ans;
33 }

```

代码II:记录路径

```

1  const int MAXN = 305;
2  int n, m;
3  int height[MAXN][MAXN];
4  int dp[MAXN][MAXN]; // dp[i][j]表示从点(i,j)开始滑的最长长度
5  int dx[4] = { -1,0,1,0 }, dy[4] = { 0,1,0,-1 };
6  int pre[MAXN]; // pre[h]记录最长路中高度为h的点的前驱
7
8  int solve(int x, int y) {
9      int& v = dp[x][y]; // 简化代码
10     if (~v) return v;
11
12     v = 1;
13     for (int i = 0; i < 4; i++) {
14         int curx = x + dx[i], cury = y + dy[i];
15         if (curx >= 1 && curx <= n && cury >= 1 && cury <= m && height[x][y] > height[curx][cury]) {
16             int tmp = solve(curx, cury);
17             if (tmp + 1 > v) {
18                 v = tmp + 1;
19                 pre[height[x][y]] = height[curx][cury];
20             }
21         }
22     }
23     return v;
24 }
25
26 int main() {
27     memset(dp, -1, sizeof(dp)); // 初始化为未更新过
28
29     cin >> n >> m;
30     int maxh = 0;
31     for (int i = 1; i <= n; i++)
32         for (int j = 1; j <= m; j++) cin >> height[i][j];
33
34     int ans = 0;
35     int start = 0; // 最长路的初始高度
36     for (int i = 1; i <= n; i++) {
37         for (int j = 1; j <= m; j++) {
38             int tmp = solve(i, j);
39             if (tmp > ans) {
40                 ans = tmp;
41                 start = height[i][j];
42             }

```

```

43     }
44 }
45 cout << ans << endl;
46
47 cout << start << ' ';
48 while (pre[start]) {
49     cout << pre[start] << ' ';
50     start = pre[start];
51 }
52 }

```

10.8.2 Digital Path

题意

$n \times m$ ($1 \leq n, m \leq 1000$) 的矩阵中的元素 $a_{ij} \in [-1e7, 1e7]$, 输出其中满足下列四个条件的路径数, 答案对 $1e9 + 7$ 取模: ① 路径只能横着或竖着走; ② 路径是最长的, 即无法再向四个方向沿伸; ③ 路径长度至少为 4; ④ 路径经过的数每次严格 +1.

思路

$dp[i][j][k]$ ($k = 0, 1, 2, 3, 4$) 表示到点 (i, j) 的路径长度为 k 的路径数, $k = 4$ 时表示路径长度 ≥ 4 的路径数.

显然入度为 0 的点都是起点, 出度为 0 的点都是终点, 故可用 BFS, 一开始先将所有入度为 0 的点入队. 但若像常规情况, 更新出的下一个可行的点入队, 可能会导致路径重复计算, 因为更新出的点可能入度不为 0, 即它可能是多条路径经过的点, 一种可行的做法是每更新出一个可行的点, 将其入度 - 1, 直至其入度减为 0 时才入队.

从点 (x, y) 更新到点 $(curx, cury)$ 时, 注意 $dp[curx][cury][4]$ 要用 $dp[curx][cury][4]$ 、 $dp[x][y][3]$ 、 $dp[x][y][4]$ 更新,

代码

```

1  const int MAXN = 1e3 + 5, MOD = 1e9 + 7;
2  int n, m; // 地图行数、列数
3  int graph[MAXN][MAXN]; // 地图
4  int in[MAXN][MAXN], out[MAXN][MAXN]; // 每个点的入度和出度
5  bool vis[MAXN][MAXN]; // 记录每个点是否已访问
6  // dp[i][j][k] 表示到点(i,j)的路径长度为k的路径数, 其中dp[i][j][4]为长度≥4的路径数
7  int dp[MAXN][MAXN][5];
8  const int dx[4] = { 1, -1, 0, 0 }, dy[4] = { 0, 0, 1, -1 };
9
10 void init() { // 预处理出每个点的入度和出度
11     for (int x = 1; x <= n; x++) {
12         for (int y = 1; y <= m; y++) {
13             for (int k = 0; k < 4; k++) {
14                 int curx = x + dx[k], cury = y + dy[k];
15                 if (curx < 1 || curx > n || cury < 1 || cury > m) continue;
16                 if (graph[x][y] == graph[curx][cury] + 1) in[x][y]++;
17                 if (graph[x][y] == graph[curx][cury] - 1) out[x][y]++;
18             }
19         }
20     }
21 }
22
23 void bfs() {
24     queue<pii> que; // 队列元素是坐标
25     // 将入度为0的点(起点)的dp[][][1]置为1并入队
26     for (int x = 1; x <= n; x++) {
27         for (int y = 1; y <= m; y++) {
28             if (!in[x][y]) {
29                 dp[x][y][1] = 1;
30                 que.push(make_pair(x, y));
31             }

```

```

32     }
33 }
34
35 while (que.size()) {
36     int x = que.front().first, y = que.front().second; que.pop();
37     for (int k = 0; k < 4; k++) {
38         int curx = x + dx[k], cury = y + dy[k];
39         if (curx < 1 || curx > n || cury < 1 || cury > m) continue;
40
41         if (graph[x][y] + 1 == graph[curx][cury]) {
42             dp[curx][cury][2] = (dp[curx][cury][2] + dp[x][y][1]) % MOD;
43             dp[curx][cury][3] = (dp[curx][cury][3] + dp[x][y][2]) % MOD;
44             dp[curx][cury][4] = (dp[curx][cury][4] + dp[x][y][3] + dp[x][y][4]) % MOD;
45             if (--in[curx][cury] == 0) // 入度减为0才入队,防止路径重复计数
46                 que.push(make_pair(curx, cury));
47         }
48     }
49 }
50 }
51
52 int main() {
53     cin >> n >> m;
54     for (int i = 1; i <= n; i++)
55         for (int j = 1; j <= m; j++) cin >> graph[i][j];
56
57     init();
58
59     bfs();
60
61     int ans = 0;
62     for (int x = 1; x <= n; x++) {
63         for (int y = 1; y <= m; y++)
64             if (!out[x][y]) // 只对出度为0的点(终点)统计答案
65                 ans = (ans + dp[x][y][4]) % MOD;
66     }
67     cout << ans;
68 }

```

思路II

避免搜索过程中路径被重复计算的另一可行方法是从终点开始DFS(类似于0-1背包压维后内层从后往前枚举,防止一个物品多次被选择),初始时将所有出度为0的点 (x, y) 的 $dp[x][y][1]$ 置为1,最后只对入度为0的点(起点)统计答案。

代码II

```

1  const int MAXN = 1e3 + 5, MOD = 1e9 + 7;
2  int n, m; // 地图行数、列数
3  int graph[MAXN][MAXN]; // 地图
4  int in[MAXN][MAXN], out[MAXN][MAXN]; // 每个点的入度和出度
5  bool vis[MAXN][MAXN]; // 记录每个点是否已访问
6  // dp[i][j][k]表示到点(i,j)的路径长度为k的路径数,其中dp[i][j][4]为长度≥4的路径数
7  int dp[MAXN][MAXN][5];
8  const int dx[4] = { 1, -1, 0, 0 }, dy[4] = { 0, 0, 1, -1 };
9
10 void init() { // 预处理出每个点的入度和出度
11     for (int x = 1; x <= n; x++) {
12         for (int y = 1; y <= m; y++) {
13             for (int k = 0; k < 4; k++) {
14                 int curx = x + dx[k], cury = y + dy[k];
15                 if (curx < 1 || curx > n || cury < 1 || cury > m) continue;
16                 if (graph[x][y] == graph[curx][cury] + 1) in[x][y]++;

```

```

17         if (graph[x][y] == graph[curx][cury] - 1) out[x][y]++;
18     }
19 }
20 }
21 }
22
23 void dfs(int x, int y) {
24     if (vis[x][y]) return; // 搜过了
25
26     vis[x][y] = true;
27     if (!out[x][y]) dp[x][y][1] = 1; // 将终点的dp[][1]置为1
28
29     for (int k = 0; k < 4; k++) {
30         int curx = x + dx[k], cury = y + dy[k];
31         if (curx < 1 || curx > n || cury < 1 || cury > m) continue;
32
33         if (graph[x][y] + 1 == graph[curx][cury]) {
34             dfs(curx, cury); // 继续搜下一个点
35
36             dp[x][y][2] = (dp[x][y][2] + dp[curx][cury][1]) % MOD;
37             dp[x][y][3] = (dp[x][y][3] + dp[curx][cury][2]) % MOD;
38             dp[x][y][4] = (dp[x][y][4] + dp[curx][cury][3] + dp[curx][cury][4]) % MOD;
39         }
40     }
41 }
42
43 int main() {
44     cin >> n >> m;
45     for (int i = 1; i <= n; i++)
46         for (int j = 1; j <= m; j++) cin >> graph[i][j];
47
48     init();
49
50     for (int x = 1; x <= n; x++)
51         for (int y = 1; y <= m; y++) dfs(x, y);
52
53     int ans = 0;
54     for (int x = 1; x <= n; x++) {
55         for (int y = 1; y <= m; y++)
56             if (!in[x][y]) // 只对出度为0的点(终点)统计答案
57                 ans = (ans + dp[x][y][4]) % MOD;
58     }
59     cout << ans;
60 }
61 }

```

思路III

$dp1[i][j]$ 表示以点 (i, j) 为起点的最长路径长度, $dp2[i][j]$ 表示到点 (i, j) 的路径数, 用两个DFS分别求出两组。

代码III

```

1 const int MAXN = 1e3 + 5, MOD = 1e9 + 7;
2 int n, m; // 地图行数、列数
3 int graph[MAXN][MAXN]; // 地图
4 int dp1[MAXN][MAXN]; // dp1[i][j]表示以(i, j)为起点的最长路径长度
5 int dp2[MAXN][MAXN]; // dp2[i][j]表示到点(i, j)的路径数
6 const int dx[4] = { 1, -1, 0, 0 }, dy[4] = { 0, 0, 1, -1 };
7
8 void init() { // 预处理出每个点的入度和出度
9     memset(dp1, -1, so(dp1));

```

```

10  memset(dp2, -1, so(dp2));
11  }
12
13  bool check(int x, int y) { // 判断是否有经过点(x,y)的合法路径
14      if (dp1[x][y] < 4) return false; // 没有从该点出发的长度≥4的路径
15
16      for (int k = 0; k < 4; k++) {
17          int curx = x + dx[k], cury = y + dy[k];
18          if (curx<1 || curx>n || cury<1 || cury>m) continue;
19
20          if (dp1[curx][cury] == dp1[x][y] + 1) return false;
21      }
22      return true;
23  }
24
25  // dfs1(int x, int y) { // 用于求dp1[] []的DFS
26      // res = 1;
27      for (int k = 0; k < 4; k++) {
28          int curx = x + dx[k], cury = y + dy[k];
29          if (curx<1 || curx>n || cury<1 || cury>m) continue;
30
31          if (graph[curx][cury] == graph[x][y] - 1) {
32              if (dp1[curx][cury] == -1) dp1[curx][cury] = dfs1(curx, cury); // 搜下一个没有搜过的点
33              res = max(res, 1 + dp1[curx][cury]);
34          }
35      }
36      return res;
37  }
38
39  // dfs2(int x, int y) { // 用于求dp2[] []的DFS
40      // res = 0;
41      for (int k = 0; k < 4; k++) {
42          int curx = x + dx[k], cury = y + dy[k];
43          if (curx<1 || curx>n || cury<1 || cury>m) continue;
44
45          if (graph[curx][cury] == graph[x][y] - 1) {
46              if (dp2[curx][cury] == -1) dp2[curx][cury] = dfs2(curx, cury); // 搜下一个没有搜过的点
47              res = (res + dp2[curx][cury]) % MOD;
48          }
49      }
50      return res;
51  }
52
53  int main() {
54      init();
55
56      cin >> n >> m;
57      for (int i = 1; i <= n; i++)
58          for (int j = 1; j <= m; j++) cin >> graph[i][j];
59
60      // maxlen = 0; // 记录最长路径长度
61      for (int x = 1; x <= n; x++) {
62          for (int y = 1; y <= m; y++) {
63              if (dp1[x][y] == -1) dp1[x][y] = dfs1(x, y); // 更新未被更新过的点的dp1[] []
64              maxlen = max(maxlen, dp1[x][y]);
65          }
66      }
67
68      if (maxlen < 4) return cout << 0, 0; // 无合法解
69
70      for (int x = 1; x <= n; x++) {
71          for (int y = 1; y <= m; y++)
72              if (dp1[x][y] == 1) dp2[x][y] = 1; // 将所有终点的dp2[] []置为1

```



```

73     }
74
75     for (int x = 1; x <= n; x++) {
76         for (int y = 1; y <= m; y++)
77             if (dp2[x][y] == -1) dp2[x][y] = dfs2(x, y); // 更新未被更新过的点的dp2[][]
78     }
79
80     ll ans = 0;
81     for (int x = 1; x <= n; x++) {
82         for (int y = 1; y <= m; y++) {
83             if (check(x, y)) {
84                 for (int k = 0; k < 4; k++) {
85                     int curx = x + dx[k], cury = y + dy[k];
86                     if (curx < 1 || curx > n || cury < 1 || cury > m) continue;
87
88                     if ((graph[x][y] == graph[curx][cury] + 1) && dp1[curx][cury] >= 3)
89                         ans = (ans + dp2[curx][cury]) % MOD;
90                 }
91             }
92         }
93     }
94     cout << ans;
95 }

```

10.8.3 Xingqiu's Joke

题意 (2 s)

有 T ($1 \leq T \leq 300$)组测试数据,每组测试数据输入两相异整数 a, b ($1 \leq a, b \leq 1e9$),每次进行如下三种操作之一,直至 a, b 中至少有一个为1,输出最少操作次数.操作:①两数同时+1;②两数同时-1;③两数同除以它们的一个公共素因子.

思路

在变化中寻找不变量,同时+1和-1让人联想到年龄的增长,注意到年龄差不变,可以发现加减过程中 $c = abs(a - b)$ 不变,则每一组 (a, b) 的公共素因子也是 c 的素因子.可想到对 c 素因数分解,枚举 c 的所有素因子 d ,进行状态 (a, b, c) 到状态 $(\frac{a}{d}, \frac{b}{d}, \frac{c}{d})$ 的转移.显然状态的前二维可只保留一个,不妨设 $a < b$,则只需保留 (a, c) ,因为最坏的情况要做 $(\min\{a, b\} - 1)$ 次-1,从状态 (a, c) 开始记搜.

考虑如何记录状态,最直接的想法是用一个 $\text{map}\langle \text{pair}\langle \text{int}, \text{int} \rangle, \text{int} \rangle$ 来记数,担心超时可以用哈希表,用一个哈希函数来将每组 (a, c) 映射为一个特征值,如 $\text{hash}(a, c) = a * 1e9 + c$.

考察 $\text{dfs}(a, c)$ 的状态数.显然第二维的状态数即 c 的因子个数,由熟知结论:int范围内因数最多的数有1600个因子.考察第一维的状态数,即 $\frac{a}{d}$ 上下取整共会产生多少个数.注意到int范围内的数至多有30个素因子(即30个2),此时 $\frac{a}{d}$ 上取整或下取整分别产生的数的个数约 $\log_2 a$ 个,约30个,总共加起来约60个.考察 a 除以 d_i 的顺序不同会不会导致结果数不同,猜测不会,理由在补充部分说明,此处先用约 $60 \times 1600 \times 300 = 2.88e7$ 的时间复杂度玄学过题.

补充:

对整数 x 和素因子 d_i ,将 x 以任意顺序整除 d_i (无论上取整还是下取整),结果至多有两种,故总状态数依旧是 c 的因数的级别.理由:设 x 以任意顺序整除 d_i 的结果是 k_1 ,全部进行上取整的结果是 k_2 .可用数归证明:

$k_1 \prod_{i=1}^n d_i \leq x \leq (k_1 + 1) \prod_{i=1}^n d_i - 1, (k_2 - 1) \prod_{i=1}^n d_i + 1 \leq x \leq k_2 \prod_{i=1}^n d_i$.将上述两式视为长度为 $(\prod_{i=1}^n d_i - 1)$ 的两区间,则当且仅当 $k_2 + 1 = k_1$ 或 $k_2 = k_1$ 时两区间有交集.因全部进行上取整的结果最大,全部进行下取整的结果最小,则上下取整混合的结果在两者之间(类似于排序不等式:正序和 $\geq$ 偏序和 $\geq$ 反序和).

代码

```

1  vi divisor; // 存c=abs(a-b)的素因子
2  map<pii, int> dp; // dp[{a,c}]表示dfs(a,c)的结果
3
4  void get(int c) { // 将c素因数分解
5      for (int i = 2; i * i <= c; i++) {
6          if (c % i == 0) {
7              divisor.push_back(i);
8              while (c % i == 0) c /= i;
9          }
10     }
11     if (c > 1) divisor.push_back(c);
12 }
13
14 int dfs(int a, int c) {
15     if (a == 1) return 0; // 已开锁
16     if (c == 1) return a - 1; // 无法再使用操作3,只能不断-1直至a=1
17     if (dp[make_pair(a, c)]) return dp[make_pair(a, c)]; // 搜过的
18
19     int res = a - 1; // 至多需要不断-1直至a=1,注意不能初始化为INF
20     for (auto d : divisor) { // 枚举初始的abs(a-b)的因子
21         if (c % d == 0) {
22             int rest = a % d; // a差几步可以变为d的倍数
23             int tmp1 = rest + 1 + dfs(a / d, c / d); // 做rest次-1,再做一次操作3,再加上a=floor(a/d)后的次数
24             int tmp2 = (d - rest) + 1 + dfs(a / d + 1, c / d); // 做(d-rest)次+1,再做一次操作3,再加上
a=ceil(a/d)后的次数
25             res = min({ res, tmp1, tmp2 });
26         }
27     }
28     return dp[make_pair(a, c)] = res;
29 }
30
31 int main() {
32     CaseT{
33         divisor.clear();
34         dp.clear();
35
36         int a, b; cin >> a >> b;
37
38         int c = abs(a - b);
39         get(c); // 将c素因数分解
40
41         cout << dfs(min(a, b), c) << endl;
42     }
43 }

```

10.8.4 Chocolate Bar

原题指路:<https://codeforces.com/contest/598/problem/E>

题意 (2 s)

给定一块 $n \times m$ 的网格型巧克力,每次操作可将巧克力沿着网格线切割,切割的花费为切割线长度的平方.现需切割这块巧克力,使得切出的巧克力构成的集合中存在一个子集,使得子集中的巧克力恰包含 k 个网格,求最小花费.

有 t ($1 \leq t \leq 40910$) 组测试数据.每组测试数据第一行输入三个整数 n, m, k ($1 \leq n, m \leq 30, 1 \leq k \leq \min\{n \cdot m, 50\}$).

思路

$dp[i][j][k]$ 表示在 $i \times j$ 的网格型巧克力中切出一个恰包含 k 个网格的巧克力的集合所需的最小花费。

按下一次操作横着切还是竖着切和第三维的分界点分类,状态转移方程:

$$\textcircled{1} \text{横着切: } dp[i][j][k] = j^2 + \min_{\substack{1 \leq x \leq \lfloor \frac{i}{2} \rfloor \\ 0 \leq mid \leq \min\{x \cdot j, k\}}} (dp[x][j][mid] + dp[i-x][j][k-mid]).$$

$$\textcircled{2} \text{竖着切: } dp[i][j][k] = i^2 + \min_{\substack{1 \leq y \leq \lfloor \frac{j}{2} \rfloor \\ 0 \leq mid \leq \min\{i \cdot y, k\}}} (dp[i][y][mid] + dp[i][j-y][k-mid]).$$

上式中只需枚举 $1 \leq x \leq \lfloor \frac{i}{2} \rfloor$ 和 $1 \leq y \leq \lfloor \frac{j}{2} \rfloor$ 是因为 $dp[n][m][k] = dp[m][n][k]$.

初始条件除 $dp[i][j][0] = dp[i][j][i \cdot j] = 0$ 外,其余 $dp[i][j][k] = INF$.最终答案 $ans = dp[n][m][k]$.

代码I: 迭代实现

```

1  const int MAXN = 30, MAXM = 50;
2  int dp[MAXN + 5][MAXN + 5][MAXM + 5];
3
4  void init() {
5      memset(dp, INF, sizeof(dp));
6      for (int i = 1; i <= MAXN; i++) {
7          for (int j = 1; j <= MAXN; j++) {
8              dp[i][j][0] = 0;
9              if (i * j <= MAXM) dp[i][j][i * j] = 0;
10         }
11     }
12
13     for (int i = 1; i <= MAXN; i++) {
14         for (int j = 1; j <= MAXN; j++) {
15             int tot = min(i * j, 50);
16             for (int k = 1; k <= tot; k++) { // 枚举切的次数
17                 for (int x = 1; x <= i / 2; x++) { // 横着切,枚举上半部分的高
18                     int cnt = min(x * j, k);
19                     for (int mid = 0; mid <= cnt; mid++) // 枚举分界点
20                         dp[i][j][k] = min(dp[i][j][k], dp[x][j][mid] + dp[i-x][j][k-mid] + j * j);
21                 }
22
23                 for (int y = 1; y <= j / 2; y++) { // 竖着切,枚举左半部分的宽
24                     int cnt = min(i * y, k);
25                     for (int mid = 0; mid <= cnt; mid++) // 枚举分界点
26                         dp[i][j][k] = min(dp[i][j][k], dp[i][y][mid] + dp[i][j-y][k-mid] + i * i);
27                 }
28             }
29         }
30     }
31 }
32
33 void solve() {
34     int n, m, k; cin >> n >> m >> k;
35     cout << dp[n][m][k] << endl;
36 }
37
38 int main() {
39     init();
40     CaseT
41     solve();
42 }

```

代码II: 记搜实现

```

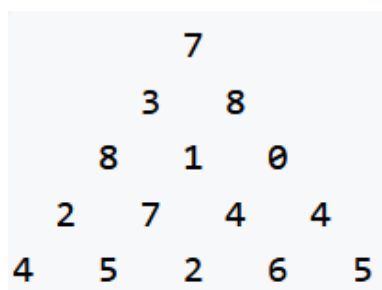
1  const int MAXN = 30, MAXM = 50;
2  int dp[MAXN + 5][MAXN + 5][MAXM + 5];
3
4  int cal(int n, int m, int k) { // 求dp[n][m][k]
5      if (~dp[n][m][k]) return dp[n][m][k];
6      if (!k || n * m == k) return 0;
7
8      int res = INF;
9      for (int i = 1; i <= n / 2; i++) { // 横着切
10         for (int j = 0; j <= k; j++) // 枚举分界点
11             res = min(res, m * m + cal(i, m, j) + cal(n - i, m, k - j));
12     }
13
14     for (int i = 1; i <= m / 2; i++) { // 竖着切
15         for (int j = 0; j <= k; j++) // 枚举分界点
16             res = min(res, n * n + cal(n, i, j) + cal(n, m - i, k - j));
17     }
18     return dp[n][m][k] = res;
19 }
20
21 void solve() {
22     memset(dp, -1, sizeof(dp));
23
24     CaseT{
25         int n, m, k; cin >> n >> m >> k;
26         cout << cal(n, m, k) << endl;
27     }
28 }
29
30 int main() {
31     solve();
32 }

```

10.9 数字三角形模型

10.9.1 数字三角形

题意



如上图, 给定 n ($1 \leq n \leq 500$)行的数字三角形, 每个数字的范围是 $[-10000, 10000]$. 从顶部出发, 在每一节点处可选择移动到其左下或右下的节点, 一直走到底层. 求一条路径使得路径上的数字和最大, 输出最大数字和.

思路

共有 2^{n-1} 条路径, 暴搜 $O(2^n)$, 最大 2^{499} , 会TLE.

考虑DP, 考虑从上到下还是从下到上. 若是从上到下, 则需特判边界, 故用从下到上.

$dp[i][j]$ 表示从下到上走到坐标 (i, j) 的路径中数字和的最大值.

坐标 (i, j) 只能来自 $(i + 1, j)$ 或 $(i + 1, j + 1)$. 从下经 $(i + 1, j)$ 走到 (i, j) 的路径可分为两部分: ①从下走到 $(i + 1, j)$, 此路径数字和的最大值为 $dp[i + 1][j]$; ②从 $(i + 1, j)$ 走到 (i, j) , 此路径的数字和即 $nums[i][j]$. 同理, 从下经 $(i + 1, j + 1)$ 走到 (i, j) 的路径数字和的最大值为 $dp[i + 1][j + 1] + nums[i][j]$.

状态转移方程:

$dp[i][j] = \max\{dp[i + 1][j] + nums[i], dp[i + 1][j + 1] + nums[i]\} = \max\{dp[i + 1][j], dp[i + 1][j + 1]\} + nums[i]$, 状态数 $O(n^2)$, 转移 $O(1)$, 总时间复杂度 $O(n^2)$.

代码I

```
1 void solve() {
2     int n; cin >> n;
3     vector<vector<int>> a(n + 1, vector<int>(n + 1));
4     for (int i = 1; i <= n; i++)
5         for (int j = 1; j <= i; j++) cin >> a[i][j];
6
7     // dp[i][j]表示从下到上走到点(i, j)的路径中数字和的最大值
8     vector<vector<int>> dp(n + 1, vector<int>(n + 1));
9     for (int i = 1; i <= n; i++) dp[n][i] = a[n][i]; // 初始条件
10
11    for (int i = n - 1; i; i--) {
12        for (int j = 1; j <= i; j++)
13            dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + a[i][j];
14    }
15    cout << dp[1][1] << endl;
16 }
17
18 int main() {
19     solve();
20 }
```

思路II

$dp[i][j]$ 未更新前存原三角形的数字, 更新后存数字和的最大值, 则只需开一个数组.

代码II

```
1 void solve() {
2     int n; cin >> n;
3     // dp[i][j]表示从下到上走到点(i, j)的路径中数字和的最大值
4     vector<vector<int>> dp(n + 1, vector<int>(n + 1));
5     // 初始时dp[][]存原矩阵
6     for (int i = 1; i <= n; i++)
7         for (int j = 1; j <= i; j++) cin >> dp[i][j];
8
9     for (int i = n - 1; i; i--) {
10        for (int j = 1; j <= i; j++)
11            dp[i][j] += max(dp[i + 1][j], dp[i + 1][j + 1]);
12    }
13    cout << dp[1][1] << endl;
14 }
15
16 int main() {
17     solve();
18 }
```

10.9.2 摘花生

题意

有一片网格形的花生地,其中每个格点处都有一定量的花生.某人从左上角的格点出发,每次只能向右走或向下走,他到达一个格点时可得到该格点处的所有花生.问他最多能得到多少花生.

有 t ($1 \leq t \leq 100$)组测试数据.每组测试数据第一行输入两个整数 r, c ($1 \leq r, c \leq 100$),分别表示网格的行数和列数.接下来输入一个 $r \times c$ 的整型矩阵描述格点处的花生数,每个格点处的花生数范围为 $[0, 1000]$.

对每组测试数据,输出他最多能得到多少花生.

思路

$dp[i][j]$ 表示从点 $(1, 1)$ 走到点 (i, j) 的所有方案中能得到的花生数的最大值.

代码

```
1  const int MAXN = 105;
2  int n, m;
3  int w[MAXN][MAXN];
4  int dp[MAXN][MAXN]; // dp[i][j]表示从点(1,1)走到点(i,j)的所有方案中能得到的花生数的最大值
5
6  void solve() {
7      cin >> n >> m;
8      for (int i = 1; i <= n; i++)
9          for (int j = 1; j <= m; j++) cin >> w[i][j];
10
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= m; j++)
13             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) + w[i][j];
14     }
15     cout << dp[n][m] << endl;
16 }
17
18 int main() {
19     CaseT // 单测时注释掉该行
20     solve();
21 }
```

10.9.3 最低通行费

题意

某人要穿过一个 $n \times n$ 的网格.他从左上角的方格出发,每次只能上下左右移动,途中每经过1个方格都需花费1个单位时间,他需在不超过 $(2n - 1)$ 个单位时间内到达右下角的方格.经过每个方格时需花费一定的费用,他希望在规定时间内用最小的费用到达右下角,求最小费用.

第一行输入一个整数 n ($1 \leq n \leq 100$).接下来输入一个 $n \times n$ 的整型矩阵,代表每个方格的费用,其中元素范围为 $[1, 100]$.

思路

注意到从左上角到达右下角的最小费用为 $(2n - 1)$,而时间限制为 $(2n - 1)$,则每次只能向右或向下走.

思路同**10.9.2 摘花生**,注意特判边界:①起点处不能由其他方向转移过来;②第一行不能由上边走过来;③第一列不能由左边走过来.

代码

```

1  const int MAXN = 105;
2  int n;
3  int w[MAXN][MAXN];
4  int dp[MAXN][MAXN]; // dp[i][j]表示从点(1,1)走到点(i,j)的所有方案中花费的最小值
5
6  void solve() {
7      cin >> n;
8      for (int i = 1; i <= n; i++)
9          for (int j = 1; j <= n; j++) cin >> w[i][j];
10
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= n; j++) {
13             if (i == 1 && j == 1) dp[i][j] = w[i][j]; // 特判左上角
14             else {
15                 dp[i][j] = INF; // 初始化为INF
16                 if (i > 1) // 只有不在第一行时才能从上边走过来
17                     dp[i][j] = min(dp[i][j], dp[i - 1][j] + w[i][j]);
18                 if (j > 1) // 只有不在第一列时才能从左边走过来
19                     dp[i][j] = min(dp[i][j], dp[i][j - 1] + w[i][j]);
20             }
21         }
22     }
23     cout << dp[n][n] << endl;
24 }
25
26 int main() {
27     solve();
28 }

```

10.9.4 方格取数

题意

有一 $n \times n$ 的网格,在其中某些方格中填入正整数,其余方格填入0.某人从左上角的方格 A 出发,每次可向右走或向下走,最终到达右下角的方格 B .途中经过某个方格时,他会取走方格中的数(取走后方格中的数变为0).他从 A 到 B 走了两次,求两条路径使得取到的数字之和最大.

第一行输入一个整数 n ($1 \leq n \leq 10$).接下来若干行每行输入三个整数 i, j, v ($1 \leq i, j \leq n$),表示在 (i, j) 处填了数 v .最后一行输入0 0 0表示输入结束.

输出一个整数,表示两路径上取得的数字之和的最大值之和.

思路

$dp[i_1][j_1][i_2][j_2]$ 表示从 $(1, 1)$ 出发,分别走到 (i_1, j_1) 和 (i_2, j_2) 的路径和的最大值.

考虑如何处理同一方格中的数不能重复取的问题.考虑两路径何时会经过同一方格,显然当且仅当 $i_1 + j_1 = i_2 + j_2$ 时两路径会经过同一方格.令 $k = i_1 + j_1 = i_2 + j_2$,则状态表示可优化为: $dp[k][i_1][i_2]$ 表示从 $(1, 1)$ 出发,分别走到 $(i_1, k - i_1)$ 和 $(i_2, k - i_2)$ 的路径和的最大值,其中 k 为当前走到的格子的横纵坐标之和.

以两路径的最后一步分类:①路径一向下走,路径二向下走;②路径一向下走,路径二向右走;③路径一向右走,路径二向下走;④路径一向右走,路径二向右走.

$$\text{对①, } dp[k][i_1][i_2] = \max\{dp[k-1][i_1-1][j_1], dp[k-1][i_2-1][j_2]\} + \begin{cases} w[i_1][j_1], (i_1, i_2) = (j_1, j_2) \\ w[i_1][j_1] + w[i_2][j_2], (i_1, i_2) \neq (j_1, j_2) \end{cases}$$

类似可得②③④的状态转移方程,四种情况取max即可.

代码

```

1  const int MAXN = 15;
2  int n;
3  int w[MAXN][MAXN];
4  int dp[MAXN][MAXN][MAXN]; // dp[k][i1][i2]表示从(1,1)出发,分别走到(i1,k-i1)和(i2,k-i2)的路径和的最大值
5
6  void solve() {
7      cin >> n;
8
9      int a, b, c;
10     while (cin >> a >> b >> c, a || b || c) w[a][b] = c;
11
12     for (int k = 2; k <= n + n; k++) {
13         for (int i1 = 1; i1 <= n; i1++) {
14             for (int i2 = 1; i2 <= n; i2++) {
15                 int j1 = k - i1, j2 = k - i2;
16                 if (j1 >= 1 && j1 <= n && j2 >= 1 && j2 <= n) {
17                     int t = w[i1][j1] + (i1 != i2) * w[i2][j2];
18                     int& x = dp[k][i1][i2];
19                     x = max(x, dp[k - 1][i1 - 1][i2 - 1] + t); // 下、下
20                     x = max(x, dp[k - 1][i1 - 1][i2] + t); // 下、右
21                     x = max(x, dp[k - 1][i1][i2 - 1] + t); // 右、下
22                     x = max(x, dp[k - 1][i1][i2] + t); // 右、右
23                 }
24             }
25         }
26     }
27
28     cout << dp[2 * n][n][n];
29 }
30
31 int main() {
32     solve();
33 }

```

10.9.5 传纸条

题意

同学坐成一个 $m \times n$ 的矩阵,A坐在左上角 $(1, 1)$ 处,B坐在右下角 (m, n) 处,他们希望通过中间的同学传纸条交流.A给B的纸条只能向下或向右传递,B给A的纸条只能向上或向左传递.每个同学都能帮忙传递纸条,但只会帮一次,即若某同学在A传给B时帮忙,则他在B传给A时不帮忙,反之亦然.每个同学的好心程度不同(A和B的好心程度无定义,输入时用0表示),可用 $[0, 100]$ 的自然数表示,数值越大表示越好心.A和B希望尽可能地请好心程度高的同学帮忙传纸条,即找到来回两条传递路径,使得两条路径上的同学的好心程度之和最大.

第一行输入整数 m, n ($1 \leq n, m \leq 50$).接下来输入一个 $m \times n$ 的整型矩阵表示每个同学的好心程度,其元素范围为 $[0, 100]$.

输出一个整数,表示来回两路径上参与传纸条的同学的好心程度之和的最大值.

思路

$dp[i_1][j_1][i_2][j_2]$ 表示从 $(1, 1)$ 出发,分别走到 (i_1, j_1) 和 (i_2, j_2) 的路径好心程度之和的最大值.同10.9.4 方格取数,优化为 $dp[k][i_1][i_2]$ 表示从 $(1, 1)$ 出发,分别走到 $(i_1, k - i_1)$ 和 $(i_2, k - i_2)$ 的路径好心程度之和的最大值,其中 k 为当前走到的格子的横纵坐标之和.状态表示 $2n \cdot n \cdot n$,状态转移时有四种情况,总时间复杂度 $O(n^3)$.

枚举 i_1 的范围需满足 $1 \leq i_1 \leq n, 1 \leq k - i_1 \leq m$,则 $\max\{1, k - m\} \leq i_1 \leq \min\{k - 1, n\}$,同理这也是 i_2 的范围.

代码

```

1  const int MAXN = 55;
2  int n, m;
3  int w[MAXN][MAXN];
4  int dp[MAXN << 1][MAXN][MAXN]; // dp[k][i1][i2]表示从(1,1)出发,分别走到(i1,k-i1)和(i2,k-i2)的路径好心程度之和的最大值
5
6  void solve() {
7      cin >> n >> m;
8      for (int i = 1; i <= n; i++)
9          for (int j = 1; j <= m; j++) cin >> w[i][j];
10
11     for (int k = 2; k <= n + m; k++) {
12         for (int i1 = max(1, k - m); i1 <= min(k-1, n); i1++) {
13             for (int i2 = max(1, k - m); i2 <= min(k - 1, n); i2++) {
14                 int t = w[i1][k - i1] + (i1 != i2) * w[i2][k - i2];
15                 for (int a = 0; a <= 1; a++) {
16                     for (int b = 0; b <= 1; b++)
17                         dp[k][i1][i2] = max(dp[k][i1][i2], dp[k - 1][i1 - a][i2 - b] + t);
18                 }
19             }
20         }
21     }
22
23     cout << dp[n + m][n][n];
24 }
25
26 int main() {
27     solve();
28 }

```

10.10 最长上升子序列模型

10.10.1 怪盗基德的滑翔翼

题意

城市中有 n 座建筑物排成一条线,每座的高度不同.初始时,怪盗基德可从任一座建筑物的顶端出发,选择一个方向逃跑,中途不能改变方向,且他只能向下滑行,即只能从较高的建筑滑翔到较低的建筑.求他最多可经过多少座建筑物的顶端(含出发点的建筑物).

有 t ($1 \leq t \leq 100$)组测试数据.每组测试数据第一行输入一个整数 n ($1 \leq n \leq 100$).第二行输入 n 个相异的整数 h ($0 < h < 1000$),依次代表建筑物的高度.

对每组测试数据,输出一个整数,表示他最多可经过多少座建筑物的顶端.

思路

从 $h[i]$ 出发:①向左走,则答案为从左往右看的以 $h[i]$ 结尾的LIS的长度;②向右走,则答案为从右往左看的以 $h[i]$ 结尾的LIS的长度.

代码

```

1  const int MAXN = 105;
2  int n;
3  int h[MAXN];
4  int dp[MAXN]; // dp[i]表示以a[i]结尾的LIS的长度
5
6  void solve() {
7      cin >> n;
8      for (int i = 0; i < n; i++) cin >> h[i];

```

```

9
10 int ans = 0;
11 for (int i = 0; i < n; i++) { // 枚举结尾
12     dp[i] = 1;
13     for (int j = 0; j < i; j++) // 向左走
14         if (h[i] < h[j]) dp[i] = max(dp[i], dp[j] + 1);
15     ans = max(ans, dp[i]);
16 }
17
18 memset(dp, 0, so(dp)); // 清空
19
20 for (int i = n - 1; i >= 0; i--) { // 枚举结尾
21     dp[i] = 1;
22     for (int j = n - 1; j > i; j--) // 向右走
23         if (h[i] < h[j]) dp[i] = max(dp[i], dp[j] + 1);
24     ans = max(ans, dp[i]);
25 }
26
27 cout << ans << endl;
28 }
29
30 int main() {
31     CaseT // 单测时注释掉该行
32     solve();
33 }

```

10.10.2 登山

题意

山上有编号 $1 \sim n$ 的 n 个景点.现需按顺序浏览这些景点,要求不连续浏览海拔相等的两景点,且一旦开始下山就不再向上走.求最多能浏览多少景点.

第一行输入整数 n ($2 \leq n \leq 1000$),表示经典数.第二行输入 n 个整数,表示每个景点的海拔.

思路

按顺序浏览表明浏览的景点是所有景点的一个子序列.最优方案是严格上升,再严格下降.

从前往后和从后往前各做一遍LIS即可.

代码

```

1  const int MAXN = 1005;
2  int n;
3  int h[MAXN];
4  int dp1[MAXN], dp2[MAXN]; // dp[i]表示以a[i]结尾的LIS的长度
5
6  void solve() {
7      cin >> n;
8      for (int i = 0; i < n; i++) cin >> h[i];
9
10     for (int i = 0; i < n; i++) { // 枚举最高景点
11         dp1[i] = 1;
12         for (int j = 0; j < i; j++) // 从前往后
13             if (h[i] > h[j]) dp1[i] = max(dp1[i], dp1[j] + 1);
14     }
15
16     for (int i = n - 1; i >= 0; i--) { // 枚举最高景点
17         dp2[i] = 1;
18         for (int j = n - 1; j > i; j--) // 从后往前

```

```

19     if (h[i] > h[j]) dp2[i] = max(dp2[i], dp2[j] + 1);
20 }
21
22 int ans = 0;
23 for (int i = 0; i < n; i++) ans = max(ans, dp1[i] + dp2[i] - 1); // 分界点重复算了一次
24 cout << ans;
25 }
26
27 int main() {
28     solve();
29 }

```

10.10.3 合唱队形

题意

n 个同学排成一排,老师要让其中的 $(n - k)$ 位同学出列,使得剩下的 k 位同学排成合唱队形.合唱队形满足:设 k 位同学从左往右依次编号为 $1, \dots, k$, 他们的身高 $h_1, \dots, h_k$ 满足 $h_1 < \dots < h_i > h_{i+1} > \dots > h_k$ ($1 \leq i \leq k$). 求最少需让几位同学出列,能使得剩下的同学排成合唱队形.

第一行输入一个整数 n ($2 \leq n \leq 100$) 表示同学个数.第二行输入 n 个整数 $h_1, \dots, h_n$ ($130 \leq h_i \leq 230$).

思路

同10.10.2 登山.

代码

```

1  const int MAXN = 1005;
2  int n;
3  int h[MAXN];
4  int dp1[MAXN], dp2[MAXN]; // dp[i]表示以a[i]结尾的LIS的长度
5
6  void solve() {
7      cin >> n;
8      for (int i = 0; i < n; i++) cin >> h[i];
9
10     for (int i = 0; i < n; i++) {
11         dp1[i] = 1;
12         for (int j = 0; j < i; j++)
13             if (h[i] > h[j]) dp1[i] = max(dp1[i], dp1[j] + 1);
14     }
15
16     for (int i = n - 1; i >= 0; i--) {
17         dp2[i] = 1;
18         for (int j = n - 1; j > i; j--)
19             if (h[i] > h[j]) dp2[i] = max(dp2[i], dp2[j] + 1);
20     }
21
22     int ans = 0;
23     for (int i = 0; i < n; i++) ans = max(ans, dp1[i] + dp2[i] - 1);
24     cout << n - ans;
25 }
26
27 int main() {
28     solve();
29 }

```

10.10.4 友好城市

题意

有一条贯穿东西的大河,河有笔直的南北两岸,岸上各位置有互不相同的 n 个城市.北岸的每个城市有且仅有一个友好城市在南岸,且不同城市的友好城市不同.现每对友好城市都想开辟一条航线连接两城市,要求任意两航线间不交叉.求最多能让多少对友好城市有航线相连.

第一行输入一个整数 n ($1 \leq 5000$),表示城市数.接下来 n 行每行输入两个整数 x, y ($0 \leq x, y \leq 1e4$),分别表示北岸和南岸的一对友好城市的坐标.

思路

用一个pair来表示一对友好城市的坐标,将其升序排列.从左往右考察南岸城市及其对应的北岸城市,注意到若航线不交叉,则北岸城市坐标递增,则南岸的每种合法方案对应一个北岸城市坐标的LIS,而每个北岸城市坐标的LIS对应南岸的一种合法方案,故它们一一对应.

代码

```
1  const int MAXN = 5005;
2  int n;
3  pii cities[MAXN];
4  int dp[MAXN]; // dp[i]表示以a[i]结尾的LIS的长度
5
6  void solve() {
7      cin >> n;
8      for (int i = 0; i < n; i++) cin >> cities[i].first >> cities[i].second;
9
10     sort(cities, cities + n);
11
12     int ans = 0;
13     for (int i = 0; i < n; i++) {
14         dp[i] = 1;
15         for (int j = 0; j < i; j++)
16             if (cities[i].second > cities[j].second) dp[i] = max(dp[i], dp[j] + 1);
17         ans = max(ans, dp[i]);
18     }
19     cout << ans;
20 }
21
22 int main() {
23     solve();
24 }
```

10.10.5 最大上升子序列和

题意

给定一个长度为 n ($1 \leq n \leq 1000$)的元素范围为 $[0, 1e4]$ 的整型序列 $a[]$,求其最长上升子序列和.

思路

$dp[i]$ 表示以 $a[i]$ 结尾的最大上升子序列和,则 $dp[i] = \max_{1 \leq k \leq n} dp[k] + a[i]$.

代码

```
1  const int MAXN = 1005;
2  int n;
3  int a[MAXN];
4  int dp[MAXN]; // dp[i]表示以a[i]结尾的最大上升子序列和
```

```

5
6 void solve() {
7     cin >> n;
8     for (int i = 0; i < n; i++) cin >> a[i];
9
10    int ans = 0;
11    for (int i = 0; i < n; i++) {
12        dp[i] = a[i]; // 初始化
13        for (int j = 0; j < i; j++)
14            if (a[i] > a[j]) dp[i] = max(dp[i], dp[j] + a[i]);
15        ans = max(ans, dp[i]);
16    }
17    cout << ans;
18 }
19
20 int main() {
21     solve();
22 }

```

10.10.6 拦截导弹

题意

有一套导弹拦截系统,其第一发炮弹能到达任意高度,但后面的每一发炮弹都不能高于前一发的高度.现有若干导弹来袭,求:①一套导弹拦截系统最多能拦截多少导弹;②要拦截所有导弹至少需多少套导弹拦截系统.

输入若干个整数,表示每个导弹的高度,高度是不超过 $3e4$ 的正整数,导弹数不超过1000.

思路

①是求序列的最长不升子序列的长度,②是求序列至少用多少个最长不升子序列能覆盖.

对②,依次枚举每个高度,当前高度有两种选择:①若不超过现有子序列的最低高度,则可接到现有的子序列之后;②若超过现有子序列的最低高度,只能新建一个子序列;③其余情况两种选择都可以.注意到任一情况都会使得当前高度是某个子序列的结尾.故②的答案即最长上升子序列的方案数.

考虑贪心,对当前高度 h ,若 $\exists$ 子序列的结尾元素 $\geq h$,则将其接到结尾高度 $\geq h$ 的最小高度之后;否则新建一个子序列.

[证] 设 A 是贪心得到的子序列个数, B 是最优解,则 $B \leq A$.

设最优解的方案与贪心得到的方案不同,考察第一个不同的数 x .

设贪心得到的方案中将 x 接到了子序列1的元素 a 之后,最优解的方案将 x 接到了子序列2的元素 b 之后,则 $b \geq a$.

考察交换两方案中 x 及其之后的序列,则子序列数不变.

因两方案的子序列数有限,则贪心的方案可通过有限次调整变为最优解的方案,故 $A \leq B$.综上 $A = B$.

代码

```

1  const int MAXN = 1005;
2  int n;
3  int h[MAXN];
4  int dp[MAXN]; // dp[i]表示以a[i]结尾的LIS的长度
5  int group[MAXN]; // 每个不升子序列的结尾元素,即每套系统拦截的导弹
6
7  void solve() {
8      string line;
9      getline(cin, line);
10     stringstream ss(line);
11     while (ss >> h[n]) n++;

```

```

12
13     int ans = 0, cnt = 0;
14     for (int i = 0; i < n; i++) {
15         dp[i] = 1;
16         for (int j = 0; j < i; j++)
17             if (h[i] <= h[j]) dp[i] = max(dp[i], dp[j] + 1);
18         ans = max(ans, dp[i]);
19
20         int idx = 0;
21         while (idx < cnt && group[idx] < h[i]) idx++; // 找到第一个结尾元素≥h[i]的子序列
22         if (idx == cnt) group[cnt++] = h[i]; // 新开一个子序列
23         else group[idx] = h[i]; // 接到现有的子序列后
24     }
25     cout << ans << endl << cnt;
26 }
27
28 int main() {
29     solve();
30 }

```

10.10.7 导弹拦截系统

题意

一套导弹拦截系统的拦截高度要么严格单调上升,要么严格单调下降.依次给定一系列导弹的高度,求至少需要几套导弹拦截系统才能将它们全部拦截.

有多组测试数据.每组测试数据第一行输入一个整数 n ($1 \leq n \leq 50$),表示导弹数.第二行输入 n 个相异的整数,表示导弹高度.输入 $n = 0$ 时表示输入结束,该数据无需处理.

对每组测试数据,输出至少需要几套导弹拦截系统才能将导弹全部拦截.

思路

在10.10.6 拦截导弹中,每个高度在哪个子序列后是唯一确定的,但在本题中,可选择将每个高度接在上升子序列后还是接在下降子序列后,可用DFS枚举两种方案,更新全局最小值.

代码

```

1  const int MAXN = 55;
2  int n;
3  int h[MAXN];
4  int up[MAXN], down[MAXN]; // 严格上升子序列的结尾、严格下降子序列的结尾
5  int ans;
6
7  // 当前枚举到的高度、当前上升子序列的个数、当前下降子序列的个数
8  void dfs(int pos, int upcnt, int downcnt) {
9      if (upcnt + downcnt >= ans) return;
10
11     if (pos == n) {
12         ans = min(ans, upcnt + downcnt);
13         return;
14     }
15
16     // 将当前高度接到上升子序列后
17     int k = 0;
18     while (k < upcnt && up[k] >= h[pos]) k++;
19     int tmp = up[k]; // 备份,后面要恢复现场
20     up[k] = h[pos];
21     if (k < upcnt) dfs(pos + 1, upcnt, downcnt); // 接到现有的上升子序列后
22     else dfs(pos + 1, upcnt + 1, downcnt); // 新建一个上升子序列

```

```

23     up[k] = tmp;    // 恢复现场
24
25     // 将当前高度接到下降子序列后
26     k = 0;
27     while (k < downcnt && down[k] <= h[pos]) k++;
28     tmp = down[k];  // 备份,后面要恢复现场
29     down[k] = h[pos];
30     if (k < downcnt) dfs(pos + 1, upcnt, downcnt); // 接到现有的下降子序列后
31     else dfs(pos + 1, upcnt, downcnt + 1); // 新建一个下降子序列
32     down[k] = tmp;  // 恢复现场
33 }
34
35 void solve() {
36     while (cin >> n, n) {
37         for (int i = 0; i < n; i++) cin >> h[i];
38
39         ans = n;
40         dfs(0, 0, 0);
41         cout << ans << endl;
42     }
43 }
44
45 int main() {
46     solve();
47 }

```

10.10.8 最长公共上升子序列

题意

对两序列 $a = [a_1, \dots, a_n]$ 和 $b = [b_1, \dots, b_m]$, 若它们都包含一段位置未必连续但数值严格递增的子列, 则称该子列是两序列的公共上升子序列, 所有的公共上升子序列中最长的称为两序列的最长公共上升子序列.

给定两长度为 n ($1 \leq n \leq 3000$)的整数序列 $a = [a_1, \dots, a_n]$ ($0 \leq a_i \leq 2^{31} - 1$)和 $b = [b_1, \dots, b_n]$ ($0 \leq b_i \leq 2^{31} - 1$), 求 $a[]$ 与 $b[]$ 的LCIS的长度.

思路

$dp[i][j]$ 表示由 $a[]$ 的前 i 个元素和 $b[]$ 的前 j 个元素构成的、以 $b[j]$ 结尾的LCIS的长度.

按LCIS是否包含 $a[i]$ 分类:

(1)不包含 $a[i]$ 的公共上升子序列中长度的最大值即由 $a[]$ 的前 $(i - 1)$ 个元素和 $b[]$ 的前 j 个元素构成的、以 $b[j]$ 结尾的LCIS的长度, 即 $dp[i - 1][j]$.

(2)对包含 $a[i]$ 的公共上升子序列, 因序列以 $b[j]$ 结尾, 则 $a[i] = b[j]$. 按倒数第二个数是空, $b[1], \dots, b[j - 1]$ 分类:

①倒数第二个数为空时, LCIS的长度最大值为0.

②倒数第二个数是 $b[k]$ 时, 当前的LCIS为由 $a[]$ 的前 $(i - 1)$ 个元素和 $b[]$ 的前 k 个元素构成的、以 $b[k]$ 结尾的LCIS最后接上 $a[i]$, 即 $dp[i - 1][k] + 1$.

朴素做法第一层循环枚举 $a[i]$, 第二层循环枚举 $b[j]$, $a[i] = b[j]$ 时, 第三层循环枚举LCIS的倒数第二个数.

当 $a[]$ 与 $b[]$ 的元素都相等时, 总时间复杂度 $O(n^3)$, 会TLE.

代码I (TLE)

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1), b(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5     for (int i = 1; i <= n; i++) cin >> b[i];
6
7     // dp[i][j]表示由a[]的前i个元素和b[]的前j个元素构成的、以b[j]结尾的LCIS的长度
8     vector<vector<int>> dp(n + 1, vector<int>(n + 1));
9     for (int i = 1; i <= n; i++) {
10         for (int j = 1; j <= n; j++) {
11             dp[i][j] = dp[i - 1][j]; // LCIS不含a[i]
12
13             if (a[i] == b[j]) { // LCIS含a[i]
14                 dp[i][j] = max(dp[i][j], 1); // LCIS倒数第二个数为空
15                 for (int k = 1; k < j; k++) { // LCIS倒数第二个数为b[k]
16                     if (b[k] < b[j])
17                         dp[i][j] = max(dp[i][j], dp[i - 1][k] + 1);
18                 }
19             }
20         }
21     }
22     cout << *max_element(all(dp[n])) << endl;
23 }
24
25 int main() {
26     solve();
27 }

```

思路II

```

1 for (int k = 1; k < j; k++)
2     if (b[k] < b[j]) dp[i][j] = max(dp[i][j], dp[i - 1][k] + 1);

```

考虑优化. 注意到上述代码中 $b[j] = a[i]$, 而 $a[i]$ 与 j 的循环无关, 可将其移到外面, 此时内层循环变为与 $dp[][1..j]$ 的最大值取max, 可用一个变量记录 $dp[][1..j]$ 的最大值, 总时间复杂度 $O(n^2)$.

代码II

```

1 void solve() {
2     int n; cin >> n;
3     vector<int> a(n + 1), b(n + 1);
4     for (int i = 1; i <= n; i++) cin >> a[i];
5     for (int i = 1; i <= n; i++) cin >> b[i];
6
7     // dp[i][j]表示由a[]的前i个元素和b[]的前j个元素构成的、以b[j]结尾的LCIS的长度
8     vector<vector<int>> dp(n + 1, vector<int>(n + 1));
9     for (int i = 1; i <= n; i++) {
10         int maxLength = 1; // LCIS的长度, 初始时只包含a[i]
11         for (int j = 1; j <= n; j++) {
12             dp[i][j] = dp[i - 1][j]; // LCIS不含a[i]
13
14             if (a[i] == b[j]) // LCIS含a[i], 且LCIS的倒数第二个数为空
15                 dp[i][j] = max(dp[i][j], maxLength);
16
17             if (b[j] < a[i]) // LCIS含a[i], 且LCIS的倒数第二个数为b[k]
18                 maxLength = max(maxLength, dp[i - 1][j] + 1);
19         }
20     }

```



```

21     cout << *max_element(all(dp[n])) << endl;
22 }
23
24 int main() {
25     solve();
26 }

```

10.11 倍增优化DP

10.11.1 开车旅行

题意

A和B决定去编号 $1 \sim n$ 的 n 个城市,其中编号较小的城市在编号较大的城市的西边.各城市的海拔高度不同,其中第 i 个城市的海拔高度为 h_i .城市 i 与 j 间的距离定义为它们的海拔高度之差的绝对值,即 $d[i][j] = |h_i - h_j|$.A和B轮流开车,第一天A开,之后每天换一次.每人每天均会从一个城市出发到达另一城市.他们计划以一个城市 s 为起点,一直向东行驶,至多行驶 x 公里结束旅行.A与B的驾驶风格不同,B总沿前进方向选择一个最近的城市作为目的地,A总沿前进方向选择第二近的城市作为目的地,若当前城市到两城市的距离相等,则认为海拔较低的城市更近.若其中一人无法按自己的原则选择目的地,或到达目的地会使得行驶距离超过 x 公里,则他们结束旅行.

现有两个问题:(1)对给定的 $x = x_0$,从哪个城市出发可使得A开车行驶的总路程与B开车行驶的总路程之比最小(若B开车行驶距离为0,则比值视为 $+\infty$,且两个 $+\infty$ 视为相等).若从多个城市出发,A开车行驶的总路程与B开车行驶的总路程之比都最小,则输出海拔最高的城市;(2)对给定的 $x = x_i$ 和出发城市 s_i ,分别求A和B各自开车行驶的总路程.

第一行输入一个整数 n ($1 \leq n \leq 1e5$),表示城市数目.第二行输入 n 个相异的整数 $h_1, \dots, h_n$ ($-1e9 \leq h_i \leq 1e9$).第三行输入一个整数 x_0 ($0 \leq x_0 \leq 1e9$).第四行输入一个整数 m ($1 \leq m \leq 1e4$),表示询问的个数.接下来 m 行每行输入两个整数 s 和 x ($1 \leq s \leq n, 0 \leq x \leq 1e9$),表示从城市 s 出发,至多行驶 x 公里.

第一行输出一个整数 s_0 ,表示对给定的 x_0 ,从编号 s_0 的城市出发,A开车行驶的路程总数与B开车行驶的路程总数之比最小.接下来 m 行每行输出两个整数,分别表示从城市 s 出发,至多行驶 x 公里时,A和B各自开车行驶的总路程.

思路

注意到在题目条件下,两两城市间的距离都确定,则行驶的路线确定.

代码

```

1 |

```

10.11.2 开车旅行

题意

思路

代码

10.12 单调队列优化DP

单调队列可优化DP的状态转移中涉及到的求窗口中的最值的部分.

10.12.1 Jump and Treasure

原题指路:<https://codeforces.com/gym/103743/problem/C>

题意 (2 s)

x 轴上有编号 $0 \sim n$ 的 $(n + 1)$ 个柱子,玩家从第0号柱子出发,终点为 $[n + 1, +\infty)$ 的平台,每次只能跳到柱子上,且跳跃距离不超过 p .除了0号柱子,其他柱子上有价值为 a_i (可为负数)的物品,玩家到达对应的柱子后必须取走.游戏有 n 个等级,其中第 i 个等级中玩家只能跳到编号是 i 的倍数的柱子上.若玩家能在等级 x 的游戏中到达终点,求其能获得的物品价值之和的最大值.

第一行输入三个整数 n, q, p ($2 \leq p \leq n \leq 1e6, 1 \leq q \leq 1e6$),分别表示柱子数、询问数、最大跳跃距离.第二行输入 n 个整数 $a_1, \dots, a_n$ ($|a_i| \leq 1e9$).接下来 q 行每行输入一个整数 x ($1 \leq x \leq n$),表示询问若玩家能在等级 x 的游戏中到达终点,其能获得的物品价值之和的最大值.

对每组询问,若玩家能到达终点,输出其能获得的物品价值之和的最大值;否则输出"Noob".

思路

先考察第1个等级. $dp[i]$ 表示到达第 i 个柱子时的最大收益,则显然有 $dp[i] = \max_{i-j \leq p} dp[j] + a[i]$.暴力转移时间复杂度 $O(n^2)$,会TLE,显然可用单调队列优化至 $O(n)$.

对第 k 个等级,能跳的柱子的编号为 $0, k, 2k, \dots$,共 $\left(\left\lfloor \frac{n}{k} \right\rfloor + 1\right)$ 个位置,则所有等级中能跳的柱子的数量为 $\sum_{k=1}^n \left(\frac{n}{k} + 1\right)$,时间复杂度 $O(n \log n)$.

代码

```
1  const int MAXN = 1e6 + 5;
2  int n, q, p; // 柱子数、询问数、最大跳跃距离
3  int a[MAXN];
4  ll dp[MAXN]; // dp[i]表示到达第i个柱子时的最大收益
5
6  void solve() {
7      cin >> n >> q >> p;
8      for (int i = 1; i <= n; i++) cin >> a[i];
9      while (q--) {
10         int x; cin >> x;
11
12         if (x > p) {
13             cout << "Noob" << endl;
14             continue;
15         }
16
17         vi pos; // 能跳的柱子的编号
18         for (int i = 0; i <= n; i += x) pos.push_back(i);
19         pos.push_back(n + 1);
20
21         deque<int> que;
22         que.push_back(0);
23         dp[0] = 0;
```

```

24     for (auto i : pos) {
25         while (que.size() && i - que.front() > p) que.pop_front(); // 超出最大跳跃距离
26         dp[i] = dp[que.front()] + a[i];
27
28         // 维护单调队列
29         while (que.size() && dp[que.back()] < dp[i]) que.pop_back();
30         que.push_back(i);
31     }
32
33     cout << dp[n + 1] << endl;
34 }
35 }
36
37 int main() {
38     solve();
39 }

```

10.12.2 最大连续子段和

题意

给定一个长度为 n ($1 \leq n \leq 3e5$)的整数序列,从中求一段长度不超过 m ($1 \leq m \leq 3e5$)的非空连续子段,使得其中的元素之和最大.

思路

$dp[i]$ 表示以 a_i 结尾的、长度不超过 m 的非空连续子段中元素之和的最大值.以结尾元素 a_k 分类,状态转移方程

$$dp[k] = \max_{1 \leq j \leq m} (pre[k] - pre[k - j]) = pre[k] - \max_{1 \leq j \leq m} pre[k - j],$$
其中 $pre[]$ 为 $a[]$ 的前缀和数组.

暴力转移时间复杂度 $O(n^2)$.考虑优化,注意到状态转移方程中 $\max_{1 \leq j \leq m} pre[k - j]$ 表示求长度为 m 的区间中的最大值,可用单调队列优化至 $O(1)$ 转移.

代码

```

1  const int MAXN = 3e5 + 5;
2  int pre[MAXN];
3
4  void solve() {
5      int n, m; cin >> n >> m;
6      for (int i = 1; i <= n; i++) {
7          cin >> pre[i];
8          pre[i] += pre[i - 1];
9      }
10
11     int ans = -INF;
12     deque<int> que; // 存pre[]的下标
13     que.push_back(0); // pre[0]
14     for (int i = 1; i <= n; i++) {
15         if (que.front() < i - m) que.pop_front();
16         ans = max(ans, pre[i] - pre[que.front()]);
17         while (que.size() && pre[que.back()] >= pre[i]) que.pop_back();
18         que.push_back(i);
19     }
20     cout << ans << endl;
21 }
22
23 int main() {
24     solve();
25 }

```

10.12.3 修剪草坪

题意

有编号 $1 \sim n$ 的 n 个人, 编号 i 号人的效率为 e_i . 现安排若干人完成工作, 求不安排 k 个连续编号的人的前提下可达到的最大总效率.

第一行输入两个整数 n, k ($1 \leq n \leq 1e5$). 第二行输入 n 个整数 $e_1, \dots, e_n$ ($0 \leq e_i \leq 1e9$).

思路

$dp[i]$ 表示只考虑前 i 个人的合法方案所能达到的最大总效率. 以是否选第 i 个人分类: ①不选第 i 个人, 总效率为 $dp[i-1]$; ②选第 i 个人, 按最后一段选的人数 x 分类, 总效率为 $dp[i-x-1] + (pre[i] - pre[i-x])$, 其中 $pre[]$ 是 $e[]$ 的前缀和. 状态转移方程:

$dp[i] = \max\{dp[i-1], pre[i] + \max_{1 \leq j \leq k} (dp[i-j-1] - pre[i-j])\}$. 令 $g[i] = dp[i-1] - pre[i]$, 显然可用滑动窗口优化, 边界

条件 $i = j$ 时, 取 $g[0] = 0$.

代码

```
1  const int MAXN = 1e5 + 5;
2  int n, m;
3  ll pre[MAXN];
4  ll dp[MAXN]; // dp[i] 表示只考虑前 i 个人的合法方案所能达到的最大总效率
5
6  ll g(int i) {
7      return dp[max(i - 1, 0)] - pre[i];
8  }
9
10 void solve() {
11     cin >> n >> m;
12     for (int i = 1; i <= n; i++) {
13         cin >> pre[i];
14         pre[i] += pre[i - 1];
15     }
16
17     deque<int> que;
18     que.push_back(0);
19     for (int i = 1; i <= n; i++) {
20         if (que.front() < i - m) que.pop_front();
21         dp[i] = max(dp[i - 1], pre[i] + g(que.front()));
22         while (que.size() && g(que.back()) <= g(i)) que.pop_back();
23         que.push_back(i);
24     }
25     cout << dp[n] << endl;
26 }
27
28 int main() {
29     solve();
30 }
```

10.12.4 旅行问题

题意

环形公路上有 n 个加油站, 每个加油站有若干升汽油 (有些加油站无汽油), 每升汽油可让汽车行驶一公里. 某人计划行驶一圈该环形公路, 他从某加油站出发, 一直按顺时针或逆时针走遍所有加油站后回到起点, 初始时其油量为零. 他每路过一个加油站时会把该加油站的所有油带上, 行驶过程中不能没油. 判断以每个加油站为起点时能否成功行驶一圈该环形公路.

第一行输入一个整数 n ($3 \leq n \leq 1e6$). 接下来 n 行每行输入两个整数, 其中第 i ($1 \leq i \leq n$) 行输入两个整数 p_i, d_i ($0 \leq p_i, d_i \leq 2e9$), 表示 i 号加油站的存油量和它到顺时针的下一个加油站的距离.

输出 n 行,其中第 i ($1 \leq i \leq n$)行输出一个字符串,若从 i 号加油站出发,能一直按顺时针或逆时针走遍所有加油站后回到起点,则输出"TAK";否则输出"NIE".

思路

倍增将环转化为链.因顺时针与逆时针对称,下面讨论顺时针的做法.

设 $\text{delta}[i] = p[i] - d[i]$,则从某个起点开始可以行驶一圈等价于以该起点为左端点的长度为 n 的区间的每个前缀的 $\text{delta}[]$ 的前缀和 ≥ 0 ,只需保证该区间内前缀和的最小值 ≥ 0 ,可用单调队列优化.注意顺时针的过程需倒序枚举 i ,求窗口的最小值;逆时针的过程需顺序枚举 i ,求窗口的最大值.

顺时针做一遍上述过程,标记可作为起点的加油站,再逆时针做一遍即可.

代码

```
1  const int MAXN = (1e6 + 5) * 2; // 注意开两倍空间
2  int n;
3  int oil[MAXN], dis[MAXN];
4  ll pre[MAXN]; // pre[i]表示delta[j]=oil[j]-dis[j]的前缀和
5  bool ans[MAXN];
6
7  void solve() {
8      cin >> n;
9      for (int i = 1; i <= n; i++) cin >> oil[i] >> dis[i];
10
11     // 顺时针:倒序枚举i,单调队列维护区间最小值
12     for (int i = 1; i <= n; i++) pre[i] = pre[i + n] = oil[i] - dis[i];
13     for (int i = 1; i <= 2 * n; i++) pre[i] += pre[i - 1];
14
15     deque<int> que;
16     que.push_back(n * 2 + 1);
17     for (int i = n * 2; i >= 0; i--) {
18         if (que.front() > i + n) que.pop_front();
19         if (i < n && pre[que.front()] - pre[i] >= 0) ans[i + 1] = true;
20         while (que.size() && pre[que.back()] >= pre[i]) que.pop_back();
21         que.push_back(i);
22     }
23
24     // 逆时针:顺序枚举i,单调队列维护区间最大值
25     dis[0] = dis[n];
26     for (int i = 1; i <= n; i++) pre[i] = pre[i + n] = oil[i] - dis[i - 1];
27     for (int i = 1; i <= 2 * n; i++) pre[i] += pre[i - 1];
28
29     que.clear();
30     que.push_back(0);
31     for (int i = 1; i <= 2 * n; i++) {
32         if (que.front() < i - n) que.pop_front();
33         if (i > n && pre[i] - pre[que.front()] >= 0) ans[i - n] = true;
34         while (que.size() && pre[que.back()] <= pre[i]) que.pop_back();
35         que.push_back(i);
36     }
37
38     for (int i = 1; i <= n; i++) cout << (ans[i] ? "TAK" : "NIE") << endl;
39 }
40
41 int main() {
42     solve();
43 }
```

10.12.5 烽火传递

题意

两城市间有 n 座烽火台,每座烽火台发出信号有一定代价.为准确传递情报,连续 m 个烽火台中需至少有一个发出信号.求在两城市间准确传递情报的最小总代价.

第一行输入两个整数 n, m ($1 \leq m \leq n \leq 2e5$).第二行输入 n 个整数,其中第 i ($1 \leq i \leq n$)个整数 a_i ($0 \leq a_i \leq 1000$)表示第 i 座烽火台发出信号的代价.

思路

$dp[i]$ 表示使得前 i 座烽火台满足要求且第 i 座烽火台发出信号的最小总代价,状态转移方程 $dp[i] = \min_{i-m \leq j \leq i} dp[j] + a[i]$,初始条件 $dp[0] = 0$.显然可用单调队列优化至总时间复杂度 $O(n)$.

代码

```
1  const int MAXN = 2e5 + 5;
2  int n, m;
3  int a[MAXN];
4  int dp[MAXN]; // dp[i]表示使得前i座烽火台满足要求且第i座烽火台发出信号的最小总代价
5
6  void solve() {
7      cin >> n >> m;
8      for (int i = 1; i <= n; i++) cin >> a[i];
9
10     deque<int> que;
11     que.push_back(0);
12     for (int i = 1; i <= n; i++) {
13         if (que.front() < i - m) que.pop_front();
14         dp[i] = dp[que.front()] + a[i];
15         while (que.size() && dp[que.back()] >= dp[i]) que.pop_back();
16         que.push_back(i);
17     }
18
19     int ans = INF;
20     for (int i = n - m + 1; i <= n; i++) ans = min(ans, dp[i]);
21     cout << ans << endl;
22 }
23
24 int main() {
25     solve();
26 }
```

10.12.6 绿色通道

题意

有编号 $1 \sim n$ 的 n 道题目,完成第 i ($1 \leq i \leq n$)题需花费 a_i 分钟.现需花费不超过 t 分钟完成若干题目,使得连续的未完成的题目构成的区间的长度最小,输出最小长度.

第一行输入两个整数 n, t ($1 \leq n \leq 5e4, 1 \leq t \leq 1e8$).第二行输入 n 个整数 $a_1, \dots, a_n$ ($1 \leq a_i \leq 3000$).

思路

连续的未完成的题目构成的区间的长度越长,满足要求需花费的时间越小,故答案具有二段性,可二分.

对每个二分的长度 mid ,问题转化为求连续的未完成的题目构成的区间的长度不超过 mid 的情况下满足要求所需的最小时间,即

10.12.5.

代码

```

1  const int MAXN = 5e4 + 5;
2  int n, m;
3  int a[MAXN];
4  int dp[MAXN]; // dp[i]表示使得前i道题满足要求且完成第i道题所需的最小时间
5
6  bool check(int mid) {
7      deque<int> que;
8      que.push_back(0);
9      dp[0] = 0;
10
11     for (int i = 1; i <= n; i++) {
12         if (que.front() < i - mid - 1) que.pop_front(); // 注意-1
13         dp[i] = dp[que.front()] + a[i];
14         while (que.size() && dp[que.back()] >= dp[i]) que.pop_back();
15         que.push_back(i);
16     }
17
18     int res = INF;
19     for (int i = n - mid; i <= n; i++) res = min(res, dp[i]);
20     return res <= m;
21 }
22
23 void solve() {
24     cin >> n >> m;
25     for (int i = 1; i <= n; i++) cin >> a[i];
26
27     int l = 0, r = n;
28     while (l < r) {
29         int mid = l + r >> 1;
30         if (check(mid)) r = mid;
31         else l = mid + 1;
32     }
33     cout << l << endl;
34 }
35
36 int main() {
37     solve();
38 }

```

10.12.7 理想正方形

题意

给定一个 $a \times b$ ($2 \leq a, b \leq 1000$)的元素范围为 $[1, 1e9]$ 整型矩阵,求一个 $n \times n$ ($1 \leq n \leq \min\{a, b, 100\}$)的正方形,使得其中元素的最大值余最小值之差最小.

思路

考虑如何求 $n \times n$ 的正方形中的最值.先在每行中求所有长度为 n 的区间中的最值,将最值存到区间右端点的元素处,则求 $n \times n$ 的正方形中的最值等价于求最靠右的一条边上的最值,即在列方向上做相同操作.

用滑动窗口分别在行方向和列方向预处理即可.

代码

```

1  int k; // k为正方形的边长
2
3  // 求数组a[]中以每个元素为右端点的长度为len的区间的最值,答案存在数组b[]中
4  void get(vector<int>& a, vector<int>& b, int len, function<bool(int, int)> cmp) {
5      deque<int> que;
6      for (int i = 1; i <= len; i++) {
7          if (que.size() && que.front() <= i - k) que.pop_front();
8          while (que.size() && cmp(a[que.back()], a[i])) que.pop_back();
9          que.push_back(i);
10         b[i] = a[que.front()];
11     }
12 }
13
14 void solve() {
15     int n, m; cin >> n >> m >> k;
16     vector<vector<int>> w(n + 1, vector<int>(m + 1));
17     for (int i = 1; i <= n; i++)
18         for (int j = 1; j <= m; j++) cin >> w[i][j];
19
20     vector<vector<int>> rowMin(n + 1, vector<int>(m + 1)); // 行最小值
21     vector<vector<int>> rowMax(n + 1, vector<int>(m + 1)); // 行最大值
22     for (int i = 1; i <= n; i++) {
23         get(w[i], rowMin[i], m, [&](int a, int b) { return a >= b; });
24         get(w[i], rowMax[i], m, [&](int a, int b) { return a <= b; });
25     }
26
27     int ans = INF;
28     for (int i = k; i <= m; i++) {
29         vector<int> a(n + 1); // 每一列的行最值
30
31         for (int j = 1; j <= n; j++) a[j] = rowMin[j][i];
32         vector<int> colMin(n + 1);
33         get(a, colMin, n, [&](int a, int b) { return a >= b; });
34
35         for (int j = 1; j <= n; j++) a[j] = rowMax[j][i];
36         vector<int> colMax(n + 1);
37         get(a, colMax, n, [&](int a, int b) { return a <= b; });
38
39         for (int j = k; j <= n; j++) ans = min(ans, colMax[j] - colMin[j]);
40     }
41     cout << ans << endl;
42 }
43
44 int main() {
45     solve();
46 }

```

10.13 斜率优化DP

10.13.1 任务安排I

题意

有 n 个任务排成一个序列,依次在一台机器上等待执行.机器会将任务分为若干批,每一批包含连续的若干个任务.从时刻0开始,任务被分批处理,处理第 i 个任务花费时间 t_i .每批任务开始前,机器需 s 的启动时间,故完成一批任务的时间是启动时间 s 加上完成其中的每个任务所需的时间之和.一个任务执行后会在机器中等待,直至该批任务全部完成,即一批任务在同一时刻完成.每个任务所需的费用是其完成时刻乘费用系数 c_i .规划一个分批方案,使得总费用最小,输出最小总费用.

第一行输入两个整数 n, s ($1 \leq n \leq 5000, 1 \leq s \leq 50$).接下来 n 行每行输入两个整数,其中第 i ($1 \leq i \leq n$)行输入两个整数 t_i, c_i ($1 \leq t_i, c_i \leq 100$),表示单独完成第 i 个任务所需的时间和其费用系数.

思路

在每一批前启动机器所需的时间会对后面的所有任务的完成时间产生影响,故可将启动机器所需的时间对后面的任务的的时间的影响全部累加到这一批任务中.

$pret[]$ 表示 $t[]$ 的前缀和, $prec[]$ 表示 $c[]$ 的前缀和.

$dp[i]$ 表示对前 i 个任务的所有划分方案中总费用的最小值.按最后一批的前一个任务的编号分类,状态转移方程:

$dp[i] = \min_{0 \leq j \leq i-1} [dp[j] + pret[i] \cdot (prec[i] - prec[j]) + s \cdot (prec[n] - prec[j])]$,初始条件

$dp[0] = 0, dp[i] = INF$ ($1 \leq i \leq n$).

DP的状态数 $O(n)$,转移 $O(n)$,总时间复杂度 $O(n^2)$.

代码

```
1  const int MAXN = 5005;
2  int n, s; // s为启动机器所需的时间
3  int pret[MAXN], prec[MAXN]; // 任务所需的时间和费用系数的前缀和
4  ll dp[MAXN]; // dp[i]表示对前i个任务的所有划分方案中总费用的最小值
5
6  void solve() {
7      cin >> n >> s;
8      for (int i = 1; i <= n; i++) {
9          cin >> pret[i] >> prec[i];
10         pret[i] += pret[i - 1], prec[i] += prec[i - 1];
11     }
12
13     memset(dp, INF, sizeof(dp));
14     dp[0] = 0; // 初始条件
15     for (int i = 1; i <= n; i++) {
16         for (int j = 0; j < i; j++)
17             dp[i] = min(dp[i], dp[j] + (ll)pret[i] * (prec[i] - prec[j]) + (ll)s * (prec[n] - prec[j]));
18     }
19     cout << dp[n] << endl;
20 }
21
22 int main() {
23     solve();
24 }
```

10.13.2 任务安排II

题意

有 n 个任务排成一个序列,依次在一台机器上等待执行.机器会将任务分为若干批,每一批包含连续的若干个任务.从时刻0开始,任务被分批处理,处理第 i 个任务花费时间 t_i .每批任务开始前,机器需 s 的启动时间,故完成一批任务的时间是启动时间 s 加上完成其中的每个任务所需的时间之和.一个任务执行后会在机器中等待,直至该批任务全部完成,即一批任务在同一时刻完成.每个任务所需的费用是其完成时刻乘费用系数 c_i .规划一个分批方案,使得总费用最小,输出最小总费用.

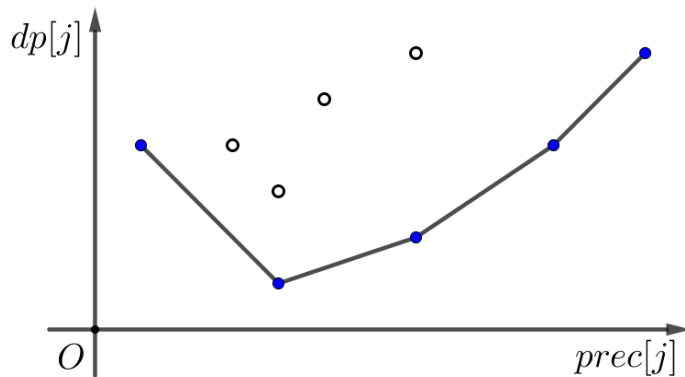
第一行输入两个整数 n, s ($1 \leq n \leq 3e5, 0 \leq s \leq 512$).接下来 n 行每行输入两个整数,其中第 i ($1 \leq i \leq n$)行输入两个整数 t_i, c_i ($1 \leq t_i, c_i \leq 512$),表示单独完成第 i 个任务所需的时间和其费用系数.

思路

$$\text{状态转移方程: } dp[i] = \min_{0 \leq j \leq i-1} [dp[j] + pret[i] \cdot (prec[i] - prec[j]) + s \cdot (prec[n] - prec[j])].$$

注意到求每个 $dp[i]$ 时,实际上是求一个 j 使得等式右边取得最小值,而等式右边只有 j 是变量,进而只有 $dp[j]$ 和 $prec[j]$ 是变量,将它们视为主元,整理得: $dp[i] = dp[j] - (pret[i] + s) \cdot prec[j] + pret[i] \cdot prec[i] + s \cdot prec[n]$.将 $dp[j]$ 视为 y , $prec[j]$ 视为 x ,则上式是一个直线方程,整理得: $dp[j] = (pret[i] + s) \cdot prec[j] + (dp[i] - pret[i] \cdot prec[i] - s \cdot prec[n])$,求 $dp[i]$ 的最小值转化为求直线截距的最小值.对固定的 i ,直线的斜率 $(pret[i] + s)$ 固定,又 $t_i \geq 1$,则斜率为正.

注意到每个 j 确定平面上的一个点 $(prec[j], dp[j])$,过该点作斜率为 $(pret[i] + s)$ 的直线,求其截距即可求得 $dp[i]$.因 $t_i, c_i \geq 1$,则截距越小, $dp[i]$ 越小.将一条斜率为 $(pret[i] + s)$ 的直线从截距 $-INF$ 开始上移直至穿过某个点 $(prec[j], dp[j])$,此时 $dp[i]$ 最小.



对不同的 i ,直线的斜率可能不同.如上图,对所有点 $(prec[j], dp[j])$ 求凸包,则只有凸包的下边界上的点可能是取得最小截距的点.类似单调队列优化DP的思路,可将冗余点删去,具体地,任选两个点 $(prec[j_1], dp[j_1])$ 和 $(prec[j_2], dp[j_2])$,过它们作一条直线,则直线上方(不含直线上)的点都可删去,即只需用队列维护凸包的下边界.

注意到凸包的下边界上的节点数最坏是 $O(n)$ 的级别,会TLE.考虑优化,对每个斜率,考察取得最小截距的点满足的性质.设凸包下边界的线段的斜率逆时针依次为 $k_1 < \dots < k_m$,则斜率为 k 的直线会在第一个 $> k$ 的斜率与上一个斜率的交界处取得最小截距.因凸包边界的斜率单调递增,问题转化为对每个 k ,在具有单调性的队列中找到第一个 $> k$ 的元素,这可用二分实现.

设队列 $que[]$ 的队首指针为 $head$,队尾指针为 $tail$.注意到 $t_i, c_i \geq 1$,则 $pret[]$ 和 $prec[]$ 单调递增,进而:

- ①直线斜率随 i 的增大而增大,故查询斜率 k 取得最小截距的点时,可删去队首中斜率 $< k$ 的所有点,即
$$\frac{dp[que[head+1]] - dp[que[head]]}{prec[que[head+1]] - prec[que[head]]} \leq pret[i] + s \text{ 时,删去队首元素.}$$
- ②点 $(prec[j], dp[j])$ 的横坐标随 j 的增大而增大,即只在队尾插入元素,故插入新点后将与其还在队列中的点连线后删去直线上方的点即可,即
$$\frac{dp[que[tail]] - dp[que[tail-1]]}{prec[que[tail]] - prec[que[tail-1]]} \geq \frac{dp[i] - dp[que[tail]]}{prec[i] - prec[que[tail]]} \text{ 时,删去队尾元素.}$$

综上,可用队列 $O(n)$ 地维护凸包的下边界,初始时0号点在队列中.

实现时可将不等式交叉相乘来避免精度问题,注意第二个不等式交叉相乘后可能爆long long.

代码

```

1  const double eps = 1e-8;
2  const int MAXN = 3e5 + 5;
3  int n, s; // 任务数、机器启动时间
4  int pret[MAXN], prec[MAXN]; // 任务耗时、费用系数的前缀和
5  ll dp[MAXN]; // dp[i]表示对前i个任务的所有划分方案中总费用的最小值
6  int que[MAXN];
7
8  void solve() {
9      cin >> n >> s;
10     for (int i = 1; i <= n; i++) {
11         cin >> pret[i] >> prec[i];
12         pret[i] += pret[i - 1], prec[i] += prec[i - 1];
13     }
14 }

```

```

15     int head = 0, tail = 0;
16     for (int i = 1; i <= n; i++) {
17         while (head < tail &&
18             cmp(((double)dp[que[head + 1]] - dp[que[head]]) / (prec[que[head + 1]] -
19             prec[que[head]]), pret[i] + s) <= 0) head++;
20
21         int j = que[head];
22         dp[i] = dp[j] - ((ll)pret[i] + s) * prec[j] + (ll)pret[i] * prec[i] + (ll)s * prec[n];
23
24         while (head < tail &&
25             cmp(((double)dp[que[tail]] - dp[que[tail - 1]]) / (prec[que[tail]] - prec[que[tail -
26             1]]),
27             ((double)dp[i] - dp[que[tail]]) / (prec[i] - prec[que[tail]])) >= 0) tail--;
28         que[++tail] = i;
29     }
30     cout << dp[n] << endl;
31 }
32
33 int main() {
34     solve();
35 }

```

10.13.3 任务安排III

题意

有 n 个任务排成一个序列,依次在一台机器上等待执行.机器会将任务分为若干批,每一批包含连续的若干个任务.从时刻0开始,任务被分批处理,处理第 i 个任务花费时间 t_i .每批任务开始前,机器需 s 的启动时间,故完成一批任务的时间是启动时间 s 加上完成其中的每个任务所需的时间之和.一个任务执行后会在机器中等待,直至该批任务全部完成,即一批任务在同一时刻完成.每个任务所需的费用是其完成时刻乘费用系数 c_i .规划一个分批方案,使得总费用最小,输出最小总费用.

第一行输入两个整数 n, s ($1 \leq n \leq 3e5, 0 \leq s \leq 512$).接下来 n 行每行输入两个整数,其中第 i ($1 \leq i \leq n$)行输入两个整数 t_i, c_i ($-512 \leq t_i \leq 512, 0 \leq c_i \leq 512$),表示单独完成第 i 个任务所需的时间和其费用系数.

思路

因 t_i 可为负,则直线斜率无单调性,进而不能像**10.13.2**一样直接删去小于当前斜率的所有点,需在所有点中二分.

因 $c_i \geq 0$,则 $prec[]$ 单调递增,进而新加的点横坐标单调递增,故维护凸包的过程与**10.13.2**相同.

总时间复杂度 $O(n \log n)$.

若 c_i 可为负,即新加入的点的横坐标未必单调递增,则需用平衡树动态维护有序序列.

代码

```

1  const double eps = 1e-8;
2  const int MAXN = 3e5 + 5;
3  int n, s; // 任务数、机器启动时间
4  int pret[MAXN], prec[MAXN]; // 任务耗时、费用系数的前缀和
5  ll dp[MAXN]; // dp[i]表示对前i个任务的所有划分方案中总费用的最小值
6  int que[MAXN];
7
8  void solve() {
9      cin >> n >> s;
10     for (int i = 1; i <= n; i++) {
11         cin >> pret[i] >> prec[i];
12         pret[i] += pret[i - 1], prec[i] += prec[i - 1];
13     }
14 }

```

```

15  int head = 0, tail = 0;
16  for (int i = 1; i <= n; i++) {
17      int l = head, r = tail;
18      while (l < r) {
19          int mid = l + r >> 1;
20          if (cmp(((double)dp[que[mid + 1]] - dp[que[mid]]))
21              / (prec[que[mid + 1]] - prec[que[mid]]), pret[i] + s) > 0) r = mid;
22          else l = mid + 1;
23      }
24
25      int j = que[l];
26      dp[i] = dp[j] - ((ll)pret[i] + s) * prec[j] + (ll)pret[i] * prec[i] + (ll)s * prec[n];
27
28      while (head < tail &&
29          cmp(((double)dp[que[tail]] - dp[que[tail - 1]]) / (prec[que[tail]] - prec[que[tail -
30  1]]),
31              ((double)dp[i] - dp[que[tail]]) / (prec[i] - prec[que[tail]])) >= 0) tail--;
32      que[++tail] = i;
33  }
34  cout << dp[n] << endl;
35 }
36
37 int main() {
38     solve();
39 }

```

10.13.4 Cats Transport

原题指路:<https://codeforces.com/problemset/problem/311/B>

题意

有编号 $1 \sim m$ 的 m 只猫和 p 个饲养员.有一条笔直的路,路边有编号 $1 \sim n$ 的 n 座山,其中 i ($i \geq 2$)号山与 $(i - 1)$ 号山的距离为 d_i ,饲养员住在1号山.现猫出去玩,其中 i 号猫去山 h_i 玩,玩到时刻 t_i 后在原地等待饲养员.饲养员的行走速度为1米/单位时间,他们需收回所有的猫,饲养员在山上收猫的时间忽略,每个饲养员可携带任意多只猫.现需规划每个饲养员从1号山出发的时间,使得所有猫的等待时间之和最小,输出所有猫的等待时间之和.注意饲养员的出发时间可为负.

第一行输入三个整数 n, m, p ($2 \leq n \leq 1e5, 1 \leq m \leq 1e5, 1 \leq p \leq 100$).第二行输入 $(n - 1)$ 个整数 $d_2, \dots, d_n$ ($1 \leq d_i \leq 1000$).接下来 m 行每行输入两个整数,其中第 i ($1 \leq i \leq m$)行输入两个整数 h_i, t_i ($1 \leq h_i \leq n, 0 \leq t_i \leq 1e9$).

思路

对 $d[]$ 求前缀和 $pred[]$,则 $pred_i$ 表示1号山到 i 号山的距离.设 i ($1 \leq i \leq p$)号饲养员在时刻 s_i 出发,则他到 j ($1 \leq j \leq m$)号猫所在山的时刻为 $s_i + pred_j$,进而他能接到 j 号猫的充要条件是: $s_i + pred_j \geq t_j$.若 i 号饲养员在时刻 $(s_i + \Delta t)$ ($\Delta t \geq 0$)出发,则 j 号猫的等待时间为 $s_i + \Delta t - (t_j - pred_j)$.令 $td_i = t_i - pred_i$,它表示饲养员接 i 号猫的最早时间.

将猫按 td_i 值非降序排列,则每个饲养员出发后将接到连续的若干只猫,且其最早的出发时间是其所要接的最后一只猫的 td_i 值.设 $td[]$ 的前缀和为 $pre[]$.以接到排序后的1, 2, 3号猫为例,饲养员最早在 td_3 时刻出发,这三只猫的等待时间之和为 $(a_3 - a_1) + (a_3 - a_2) = 3a_3 - pre_3 - pre_0$.问题转化为:将猫按 td_i 值非降序排列得到的序列分为至多 p 段,求所有段的代价之和的最小值.

$dp[i][j]$ 表示将前 i 只猫分为 j 段时所有段的代价之和的最小值.因DP转移时外层循环 j ,内层循环 i ,故将状态定义为 $dp[j][i]$ 时访问的内存连续,效率更高,故状态定义改为 $dp[j][i]$ 表示让前 j 个饲养员接前 i 只小猫时所有猫的等待时间之和的最小值.以最后一个饲养员接哪一段小猫分类,状态转移方程 $dp[j][i] = \min_{1 \leq k \leq i-1} \{dp[j-1][k] + td[i] \cdot (i - k) - (pre[i] - pre[k])\}$,初始条件除

$dp[i][0] = 0$ ($0 \leq i \leq p$)外,其余状态为 $dp[j][i] = INF$.

考察使得状态转移方程的等式右边取得最小值的 k ,以 $dp[j-1][k]$ 、 pre_k 、 k 为主元整理得:

$dp[j-1][k] + pre[k] = td[i] \cdot k + dp[j][i] - td[i] \cdot i + pre[i]$.因 $td_i > 0$,故直线的斜率为正,且直线的截距越小, $dp[j][i]$ 越小.斜率优化即可做到 $O(mp)$ 的时间复杂度,注意状态转移时 $dp[j][i]$ 需要用到第 $(j - 1)$ 层的状态,即队列维护第 $(j - 1)$ 层选取的点.

代码

```

1  const double eps = 1e-8;
2  const int MAXN = 1e5 + 5, MAXM = 1e5 + 5, MAXP = 105;
3  int n, m, p; // 山数、猫数、饲养员数
4  int pred[MAXN]; // 山的距离的前缀和
5  int t[MAXN]; // 猫玩的时刻
6  ll td[MAXN], pre[MAXN]; // td[i]=t[i]-d[i],pre[]是td[]的前缀和
7  ll dp[MAXP][MAXM]; // dp[j][i]表示让前j个饲养员接前i只小猫时所有猫的等待时间之和的最小值
8  int que[MAXM];
9
10 ll cal(int k, int j) {
11     return dp[j - 1][k] + pre[k];
12 }
13
14 void solve() {
15     cin >> n >> m >> p;
16     for (int i = 2; i <= n; i++) {
17         cin >> pred[i];
18         pred[i] += pred[i - 1];
19     }
20     for (int i = 1; i <= m; i++) {
21         int h; cin >> h >> t[i];
22         td[i] = t[i] - pred[h];
23     }
24
25     sort(td + 1, td + m + 1);
26     for (int i = 1; i <= m; i++) pre[i] = pre[i - 1] + td[i];
27
28     memset(dp, INF, sizeof(dp));
29     for (int i = 0; i <= p; i++) dp[i][0] = 0; // 初始条件
30
31     for (int j = 1; j <= p; j++) { // 枚举饲养员
32         int head = 0, tail = 0; // 初始时队列有元素0
33
34         for (int i = 1; i <= m; i++) { // 枚举猫
35             while (head < tail &&
36                 cmp(((double)cal(que[head + 1], j) - cal(que[head], j))
37                     / (que[head + 1] - que[head]), td[i]) <= 0) head++;
38
39             int k = que[head];
40             dp[j][i] = dp[j - 1][k] - td[i] * k + pre[k] + td[i] * i - pre[i];
41
42             while (head < tail &&
43                 cmp(((double)cal(que[tail], j) - cal(que[tail - 1], j)) / (que[tail] - que[tail - 1])
44                     , ((double)cal(i, j) - cal(que[tail], j)) / (i - que[tail])) >= 0) tail--;
45             que[++tail] = i;
46         }
47     }
48     cout << dp[p][m] << endl;
49 }
50
51 int main() {
52     solve();
53 }

```

